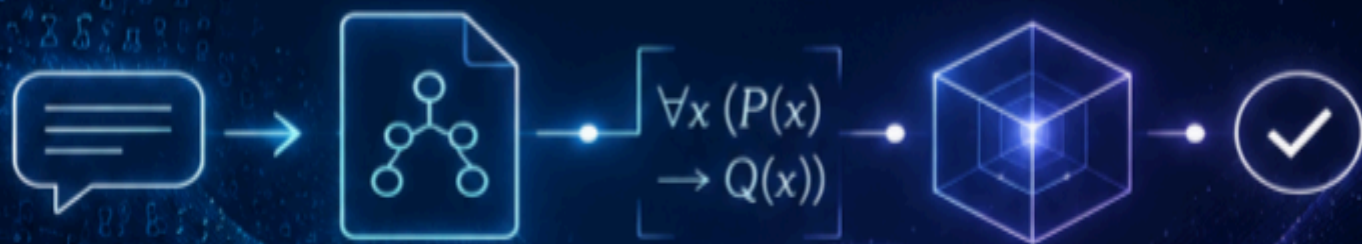


LANGUAGE NATUREL EXÉCUTABLE

Étude critique de la sémantique formelle, de
l'interprétation abstraite, de l'exécution symbolique
et des architectures LLM à fondement symbolique



LANGAGE NATUREL EXÉCUTABLE

Étude critique de la sémantique formelle, de l'interprétation abstraite, de l'exécution symbolique et des architectures LLM à fondement symbolique

Vers un runtime neuro-symbolique pour documents longs, règles et systèmes agentiques



Copyright © [2026] Axiologic Research

Tous droits réservés.

KDP droits d'édition détenus par Outfinity SRL (Iași, Roumanie).

Disponible gratuitement sur le site Axiologic (www.axiologic.net).

Note de l'auteur

Ce travail a été créé sous l'égide de Axiologic Research dans le cadre de deux programmes de recherche interconnectés, l'un dédié à l'intelligence artificielle et l'autre à l'outfinitisme, tous deux dirigés par Sînică Alboaie. Pour explorer les détails de la façon dont ces deux programmes se croisent et s'appuient l'un sur l'autre, nous invitons les lecteurs à visiter la page Outfinity Venture Studio à www.outfinity.ch. Alors que le

s modèles de langage d'intelligence artificielle ont été utilisés pour aider à la génération de texte, la philosophie fondamentale, la structure thématique et l'approche analytique découlent exclusivement des décennies d'étude dédiée de l'auteur sur les phénomènes sociaux, les technologies sociales et les véritables technologies dures résultant de divers projets de recherche européens et auto-financés. Récemment, dans le cadre du projet de recherche Achille, nous avons étudié en profondeur la littérature générée par AI et sa vérification automatisée à l'aide d'approches neuro-concentriques; tous les livres gratuits mis à la disposition du public font partie de la suite d'œuvres que nous utilisons pour tester nos dernières technologies.

Table des matières

Résumé.....	5
Préface : portée, méthode et position critique.....	8
1. Du langage courant à la sémantique exécutable.....	11
1.1 Ce qu'exige l'exécution et où doit se situer la frontière de confiance.....	11
1.2 L'écart de formalisation : pourquoi un solveur correct peut encore produire la mauvaise réponse.....	15
2. La lignée intellectuelle du langage exécutable.....	18
2.1 Langues contrôlées, sémantique formelle, discours et logique naturelle.....	18
2.2 Analyse sémantique, requêtes exécutables, induction de programmes et premiers systèmes neuro-symboliques.....	22
3. Interprétation abstraite et exécution symbolique pour le langage.....	26
3.1 Interprétation abstraite comme une sémantique de l'incertitude bornée.....	26
3.2 Exécution symbolique, résolution de contraintes, vérification de modèle et raffinement sémantique.....	30
4. Ce que l'ère LLM a réellement atteint.....	35
4.1 Autoformalisation, raisonnement assisté par programme, logique, spécifications temporelles et planification.....	35
4.2 Analyse de programmes assistée par LLM, exploration symbolique, transformateurs abstraits et NLP certifié.....	39
5. Pourquoi les systèmes de langage exécutable échouent.....	43
5.1 Ancrage, identité, formalisations fallacieuses, ontologies et connaissance du monde...	43
5.2 Pragmatique, non-monotonie, échelle, sécurité et échec de l'évaluation.....	47
6. Familles d'architectures pour le langage naturel exécutable.....	51
6.1 Conceptions concurrentes : représentations universelles, portefeuilles de formalismes, compilation contrôlée et systèmes query-first.....	51
6.2 Un pipeline de référence du texte aux symboles, à l'exécution et aux réponses en langage naturel contraint.....	56
7. LongTextJS, CircuitJS et nllAgent: une étude de cas critique.....	60
7.1 Ce que l'architecture publique met en œuvre et pourquoi ses limites comptent.....	61
7.2 Évaluation indépendante: points forts, problèmes graves actuels et évolution requise..	65
8. Construire et valider un runtime neuro-symbolique.....	69
8.1 Stratégie de mise en œuvre: un système de langage exécutable minimal mais scientifiquement honnête.....	69
8.2 Questions de recherche, repères et programme de publication.....	76
Conclusion : de la forme probabiliste à la conséquence symbolique.....	83
Références.....	87

Résumé

Les grands modèles de langage ont fait du langage naturel une interface efficace avec le calcul, mais ils n'ont pas rendu le langage naturel exécutable au sens fort dans lequel un programme, une requête de base de données, un système de transition ou une théorie logique est exécutable. Un modèle peut générer une explication convaincante tout en changeant la portée d'un quantificateur, en omettant une exception, en inventant une prémisse ou en appliquant la même règle différemment en deux endroits. Les outils externes améliorent la fiabilité uniquement lorsque le modèle a bien traduit la requête. Un théorème peut prouver parfaitement la mauvaise formalisation; un interprète Python peut exécuter un programme qui répond accidentellement à l'exemple tout en déformant la question; un exécuteur symbolique peut exposer un chemin réel par le biais d'un logiciel même si la règle de langage naturel utilisée pour construire la condition de chemin a été mal traduite. Le problème difficile n'est donc pas seulement d'ajouter un solveur après un LLM. Il s'agit de construire une chaîne vérifiable du langage source aux symboles fondés, des symboles fondés aux objets formels exécutables, et des résultats symboliques au langage naturel sans permettre au modèle de langage final de modifier ce qui a été établi.

Ce livre examine les traditions pertinentes comme un problème de recherche connexe. Les fondements antérieurs comprennent les langages naturels contrôlés, la sémantique de la théorie des modèles et du discours, la logique naturelle, l'analyse sémantique, la programmation logique, les systèmes de texte à requête, la synthèse de programmes à partir d'exemples et de langage, les moteurs de règles, les tables de décision, les logiques temporelles, la planification automatisée et les systèmes neuro-symboliques. Les bases d'analyse de programme comprennent l'interprétation abstraite, l'exécution symbolique et concolique, l'analyse de flux de données, la vérification du modèle, la résolution SAT/SMT/CSP, la résolution de Datalog et l'évaluation des points fixes, les automates et la vérification de la production de preuves. La littérature récente LLM ajoute le raisonnement assisté par programme, l'incitation du solveur déclaratif, l'autoformalisation, la traduction de la logique du premier ordre, la traduction temporelle-logique, la planification formelle, l'auto-réparation guidée par solveur, l'analyse de programme assistée par LLM, l'exploration symbolique guidée par LLM et la synthèse de transformateurs abstraits vérifiés. Dans ces domaines de travail, les résultats les plus solides apparaissent constamment lorsque le formalisme cible est suffisamment étroit pour avoir une sémantique explicite et que la production acceptée est vérifiée par un composant indépendant du LLM.

L'enquête accorde une attention particulière à la distinction entre deux utilisations de l'interprétation abstraite. Le premier, déjà établi dans NLP, vérifie un modèle neuronal sur une famille formelle de perturbations textuelles. Les domaines abstraits d'intervalle et relationnels peuvent certifier que la prédiction d'un classificateur ne change pas sous chaque substitution autorisée par un ensemble contradictoire spécifié. Il s'agit d'une véritable garantie formelle, mais elle concerne la stabilité du modèle dans une relation de perturbation construite; elle ne certifie pas que les paraphrases humaines arbitraires préservent le sens, ni que la réponse originale du modèle est vraie. La deuxième utilisation est plus spéculative et plus pertinente pour le langage naturel exécutable: traiter l'ensemble d'interprétations recevables d'un document comme une sémantique collective et l'informatique conservatrice peut, doit, ne peut pas, et des conclusions inconnues sur cet ensemble. Cela nécessite des domaines explicites

pour l'identité, le temps, la modalité, la quantité, le statut de source, la force normative et la couverture. Il nécessite également une garantie conditionnelle honnête: l'analyse n'est solide que par rapport à l'ensemble d'interprétation, l'ontologie, la politique de mise à la terre et les règles de transfert qui lui sont fournies.

L'exécution symbolique est complémentaire. L'interprétation abstraite peut à moindre coût montrer qu'une violation est impossible sous l'abstraction actuelle ou qu'une violation reste possible. L'exécution symbolique peut alors tenter de construire une interprétation concrète et un chemin qui la réalise. Si un tel chemin n'existe pas, le témoin raté peut révéler quelle abstraction ou choix sémantique était trop grossier. Cela suggère un modèle de recherche appelé raffinement sémantique guidé par contre-exemple: le contre-exemple d'un solveur, le noyau insatisfaisant, ou la preuve ratée est cartographié aux passages sources exacts et aux engagements sémantiques qui l'ont généré; le LLM ou un humain est invité à ne réviser que ces engagements; l'analyse symbolique est rejouée. Le LLM est utilisé pour rechercher l'espace des formalisations, tandis que le noyau symbolique détermine ce qui découle de chaque formalisation acceptée.

Les systèmes récents montrent des fragments importants de cette vision. PAL délègue un calcul procédural à Python. SatLM génère des contraintes déclaratives pour un moteur de satisfaction. Logic-LM et LINC traduisent les tâches de raisonnement en langage naturel en logique formelle et s'appuient sur des solveurs symboliques ou des preuves de théorème pour la déduction. NL2TL et nl2spec traduisent les exigences en spécifications temporelles, nl2spec prenant explicitement en charge la correction interactive au niveau de la sous-formule. Le travail de planification formelle a montré de grands gains lorsque LLMs construit des instances de planification qui sont ensuite résolues et vérifiées par des outils sonores. SymGPT traduit les règles en langage naturel ERC en une grammaire restreinte et utilise l'exécution symbolique pour trouver des violations concrètes du contrat intelligent, signalant des milliers de violations de règles et des voies d'attaque explicites. Dans l'autre sens, AutoBug décompose l'analyse du programme en chemins symboliques gérés par LLM mais reste approximatif; SAIL utilise LLMs pour rechercher des transformateurs abstraits tandis que les contrôles sémantiques et syntaxiques durs préservent la solidité globale. Ces résultats sont la preuve que LLMs sont des mécanismes de proposition et de recherche efficaces autour des noyaux formels, et non la preuve que la génération probabiliste elle-même est devenue un raisonnement formel.

Une récente revue systématique de NLP neuro-symbolique soutient que les architectures fédératives, dans lesquelles les modules neuronaux et symboliques restent autonomes et communiquent à travers des interfaces structurées, montrent souvent des gains plus importants que les architectures qui ne font qu'injecter des biais symboliques dans les réseaux de neurones. Ses auteurs mettent également en garde contre le fait que l'avantage apparent n'est pas encore concluant parce que les systèmes fédératifs et injectifs sont généralement évalués sur différents types de tâches. Ce livre adopte la même position disciplinée. Le cas architectural de la séparation est solide parce qu'il clarifie la limite de confiance, permet une relecture indépendante et maintient la composante symbolique inspectable; l'affirmation empirique selon laquelle une famille surpasse universellement une autre reste non prouvée [Chatzikyriakidis and Lappin 2026].

Le livre fournit également une étude de cas critique de l'architecture publique NaturalLanguageLinterAgent, ou nllAgent, et de ses composants LongTextJS et CircuitJS. La mise en œuvre sépare un programme immuable côté document d'un programme réutilisable

côté théorie. Il préserve les ancrs sources exactes, les types d'observation versionnés, les identités de producteurs, la couverture, les lacunes et le statut épistémique. La compilation du circuit tire les observations effectivement exigées par une règle, la publication vérifie si des producteurs compatibles existent, des tests de compatibilité de temps d'exécution si le document concret a donné suffisamment d'observations et de couverture, et un vérificateur indépendant contrôle si une conclusion candidate est publiée. L'apprentissage se fait dans un espace de travail jetable et ne peut pas publier automatiquement. Ce sont des décisions de gouvernance inhabituellement sérieuses pour un système d'analyse de documents assisté par LLM. Le dépôt public enregistre également des limitations non résolues plutôt que de les cacher. Son registre actuel de questions graves identifie l'incrémentalité du planificateur borné, la propagation incomplète d'alternatives d'interprétation à travers chaque chemin d'exécution, l'analyse de compatibilité conservatrice pour un comportement procédural opaque, la planification minimale de la requête et l'indexation temporelle, la génération volontairement restreinte contrôlée et le soutien de mutation qui reste spécifique au scénario. Ce sont des limites de recherche crédibles: elles concernent le substrat sémantique et d'exécution, et non pas seulement le vernis d'interface utilisateur manquant.

Aucune évaluation indépendante évaluée par des pairs ou examen technique par des tiers de nllAgent n'a été localisée dans les recherches effectuées pour cette édition. En conséquence, l'évaluation de l'étude de cas est explicitement la propre revue architecturale de l'auteur, fondée sur le dépôt public, la documentation, la portée exécutable et les questions graves autoproclamées. Le jugement est favorable à la séparation des preuves, à la théorie, à la vérification et à la publication, mais prudent sur la complétude sémantique. Les allégations de mise en œuvre les plus solides en vigueur concernent des contrôles structurels et contrôlés-anglais. La résolution d'identité générale, la sémantique des événements riches, les connaissances du monde actuel, le transfert de domaine large et les allégations de qualité statistiquement prises en charge restent un travail de recherche plutôt qu'une capacité établie.

La proposition centrale du livre est un runtime multi-formalisme en langage naturel-exécutable. Un LLM propose d'abord un petit ensemble de structures sémantiques candidates, chacune liée à des travées de sources exactes et tapées contre une ontologie sélectionnée. Un noyau symbolique de mise à la terre n'admet que des objets dont l'identité, la provenance, la portée, le temps, la modalité et les preuves satisfont des contrats explicites. Un routeur sélectionne une ou plusieurs cibles formelles en fonction de la question: évaluation relationnelle ou ZX0000QXZ pour la fermeture et les requêtes de documents; tableaux de décision pour les politiques; SAT/SMT/CSP pour les contraintes; automates pour les modèles ordonnés; modélisation de vérification du comportement temporel; exécution symbolique pour les témoins de chemin; et assistants de preuve ou fournisseurs de théorème pour les réclamations déductives. L'exécution formelle produit un résultat canonique contenant des faits acceptés, des candidats rejetés, de l'incertitude, des témoins, des objets de preuve, des limites de couverture et des hypothèses. Un dernier LLM peut expliquer ce résultat, mais chaque phrase de l'explication doit être autorisée par la structure canonique et traçable à la preuve ou à une dérivation vérifiée.

Le langage naturel exécutable ne devrait donc pas être annoncé comme l'exécution d'une prose sans restriction. Il devrait être défini comme un processus contrôlé de compilation, d'ancrage, d'exécution et de réalisation plutôt que sur un ou plusieurs candidats sémantiques explicites. La garantie est conditionnelle mais utile: compte tenu de la révision de la source, de l'ontologie, de la mise à la terre, de l'ensemble d'interprétation, de la sémantique formelle et des

limites d'exécution, le résultat rapporté suit et peut être rejoué. Le principal problème de recherche ouverte n'est pas le solveur. C'est la construction, la comparaison, le raffinement et la validation des formalisations sur lesquelles le solveur opère.

Préface : portée, méthode et position critique

L'ambition d'un langage naturel exécutable est facile à exprimer et facile à surestimer. Un utilisateur écrit une règle, une politique, une hypothèse scientifique, une procédure ou une question dans un langage ordinaire; un système le comprend; le calcul suit; le résultat est renvoyé comme langage ordinaire. Les modèles de langage contemporain font paraître le début et la fin de ce pipeline résolu. Ils peuvent lire la prose sans restriction, générer du code et des formules, invoquer des outils et expliquer couramment les sorties. L'illusion dangereuse est que le milieu a disparu. En fait, le milieu contient presque tous les problèmes difficiles: décider quel sens compte, identifier des objets à travers les passages, distinguer les preuves explicites des hypothèses de fond, préserver la modalité et les exceptions, sélectionner un formalisme approprié, prouver que l'artefact généré est bien formé, et empêcher l'explication finale de dire plus que l'exécution établie.

Une étude de synthèse utile ne peut pas être un catalogue de petits systèmes présentés dans de courtes sous-sections isolées. Le champ est fragmenté précisément parce que les mêmes mots sont utilisés pour différentes opérations. «Exécution symbolique» peut signifier l'exploration de chemin de style King sur des entrées de programme symboliques, l'exécution d'une forme logique générée sur une base de connaissances, ou simplement l'utilisation d'un interprète externe. La «vérification formelle» peut se référer à l'exactitude d'un résultat de solveur, à l'exactitude d'un programme compilé, à la fidélité de la traduction en langue naturelle ou à la robustesse d'un réseau de neurones sous une famille de perturbations prédéfinie. «Plantage» peut signifier connecter des mots à des colonnes de base de données, des entités à des portées de documents, des propositions à la connaissance du monde ou des instructions de robot à des objets perceptifs. Un traitement sérieux doit montrer comment ces couches se rapportent et où les garanties s'arrêtent.

La méthode d'enquête suit donc les mécanismes plutôt que les catégories de marketing. Pour chaque ligne de travail, l'analyse pose cinq questions. Quel artefact est produit à partir du langage? Quelle sémantique cet artefact a-t-il indépendamment du LLM? Comment l'artefact est-il ancré dans la source ou l'environnement? Quel composant détermine réellement le résultat? Qu'est-ce qui est et n'est pas garanti lorsque le système réussit? Les papiers sont traités comme des preuves de leurs tâches déclarées, et non comme une preuve d'une architecture générale. Des résultats numériques solides sont inclus lorsqu'ils clarifient le mécanisme, mais ils ne sont pas transférés dans des domaines avec une structure sémantique différente. L'horizon de l'enquête est le 2 août 2026 et comprend la littérature évaluée par des pairs, les procédures officielles, les documents de recherche primaires et le dépôt public nllAgent et la documentation.

Le terme «revue» est utilisé avec soin. Plusieurs enquêtes générales existent maintenant, mais chacune ne couvre qu'une partie du problème de bout en bout. L'enquête en langage naturel contrôlé de Kuhn organise des langues en fonction de la précision, de l'expressivité, du naturel et de la simplicité [Kuhn 2014]. Hamilton et ses collègues évaluent si le neuro-symbole NLP a effectivement fourni le raisonnement, le transfert, l'interprétabilité, l'efficacité des données et la généralisation hors distribution, tandis que des enquêtes plus récentes classifient les systèmes neuro-symboliques axés sur les tâches et axés sur les données et les connaissances [Hamilton et al. 2024; Delvecchio et al. 2025; Wang et al. 2025b]. Les enquêtes sur l'exécution symbolique et l'analyse du programme assistée par LLM décrivent les limites

d'ingénierie de l'exploration de trajectoire et le rôle croissant des composants appris [Baldoni et al. 2018; Wang et al. 2025a]. Une revue dédiée étudie LLMs en tant que formalisateurs pour la planification automatisée [Tantakoun et al. 2025], et l'enquête d'autoformalisation se concentre sur la traduction des mathématiques informelles en formes vérifiables par machine [Weng et al. 2025]. Une feuille de route bidirectionnelle pour l'ingénierie des exigences examine à la fois l'utilisation de LLMs pour rendre les méthodes formelles accessibles et l'utilisation de méthodes formelles pour limiter les exigences [Hamilton et al. 2024; Delvecchio et al. 2025; Wang et al. 2025b]-produites [Ferrari and Spoletini 2025]. Une étude de synthèse ciblée sur trente-cinq cartes LLM-assistée des exigences et des spécifications du logiciel, y compris le travail ciblant Dafny, C et Java; son champ d'application confirme que le support de traduction augmente plus rapidement que l'assurance sémantique de bout en bout [Beg et al. 2025]. La revue systématique 2026 Frontiers développe une nouvelle taxonomie pour les NLP neuro-symboliques et plaide pour une plus grande attention aux systèmes fédératifs [Chatzikyriakidis and Lappin 2026].

Aucun examen étudié unifie le langage contrôlé, l'ancrage sémantique, l'interprétation abstraite, l'exécution symbolique, la sélection du formalisme, les résultats de la preuve et la régénération limitée comme une seule architecture de confiance. Cette conclusion est une synthèse de la littérature plutôt qu'une revendication de non-existence exhaustive: la terminologie diffère d'un domaine à l'autre et les documents pertinents sont distribués en fonction de la linguistique computationnelle, des langages de programmation, des bases de données, des méthodes formelles, de l'ingénierie logicielle, de la planification et de la robotique. Lorsqu'il n'existe pas d'examen indépendant, en particulier pour nllAgent, le texte qualifie ses jugements d'analyse originale plutôt que d'impliquer l'approbation externe.

La position du livre est opinionnée mais conservatrice. Il rejette l'idée qu'un grand modèle devrait être considéré comme le dernier raisonneur simplement parce que sa prose ressemble à une preuve. Il rejette également l'idée opposée selon laquelle seul un analyseur sémantique complet écrit à la main peut soutenir une exécution formelle utile. Dans les domaines ouverts, un LLM est souvent le meilleur mécanisme disponible pour proposer des événements, des relations, des schémas formels et des cartographies candidates. La question architecturale est de savoir comment limiter ce pouvoir. La réponse la plus défendable est un système fédéré avec des artefacts intermédiaires explicites, des interfaces tapées et versionnées, une exécution symbolique indépendante, des états inconnus conservateurs et des preuves rejouables.

Cette position a une conséquence importante. Le but du système n'est pas toujours de produire une formalisation définitive. Parfois, l'artefact correct est un ensemble d'alternatives. Un pronom peut se référer à deux acteurs possibles; une expression juridique peut admettre des portées concurrentes; une exigence peut être incomplète; deux sources peuvent être en désaccord; un document peut ne pas dire si une exception s'applique. Une durée symbolique qui choisit arbitrairement une interprétation est moins rigoureuse qu'un LLM qui énonce candidement l'ambiguïté. L'exécutabilité devient précieuse lorsqu'elle peut calculer ce qui suit à chaque lecture admissible, ce qui suit sous au moins un, quelles conclusions dépendent d'un choix contesté et quelles preuves supplémentaires résoudraient le choix.

L'architecture proposée traite donc l'incertitude structurellement. Les scores de confiance peuvent classer les candidats, mais ils ne transforment pas une interprétation proposée en un fait certifié. L'état d'un article enregistre comment il est devenu connu: directement extrait d'un

analyseur déterministe, proposé par un modèle, assumé pour un scénario, dérivé par une règle, vérifié par un vérificateur, ou confirmé par une personne. La couverture indique si un domaine a été entièrement examiné, échantillonné, récupéré ou non. Une conclusion négative n'est autorisée que lorsque la recherche pertinente est clôturée en vertu d'une politique déclarée. C'est la différence entre un système symbolique fiable et un classificateur fluide avec une sortie en forme de JSON.

La discussion reste accessible plutôt que la preuve lourde. L'interprétation abstraite, l'exécution symbolique, la vérification des modèles et la sémantique logique sont expliquées par leur essence opérationnelle et les obligations qu'ils créent. La notation formelle n'apparaît que lorsqu'elle clarifie une distinction que la prose peut s'estomper. L'objectif n'est pas de présenter des mathématiques décoratives. Il s'agit de donner aux concepteurs de systèmes, aux examinateurs et aux chercheurs suffisamment de précision conceptuelle pour décider où une garantie proposée est valide, où elle est conditionnelle et où elle est simplement ambitieuse.

La thèse résultante peut être clairement énoncée. LLMs devrait découvrir des formes formelles plausibles; les composants déterministes et formellement contraints devraient contrôler la mise à la terre, la compilation, l'exécution et l'acceptation; LLMs peut alors réaliser le résultat canonique comme langage, mais seulement dans le cadre d'un contrat qui empêche les ajouts non pris en charge. Ce n'est pas une solution complète au sens. Il s'agit d'une voie crédible allant du traitement du langage probabiliste au raisonnement borné, exécutable et auditable.

1. Du langage courant à la sémantique exécutable

1.1 Ce qu'exige l'exécution et où doit se situer la frontière de confiance

Le mot «exécuter» porte un engagement plus fort que «interpréter», «répondre» ou «raison». Un programme est exécutable car ses opérations ont un effet défini sur un état. Une requête de base de données est exécutable car les opérateurs relationnels déterminent quels tuples sont sélectionnés, joints, groupés ou exclus. Un problème de planification est exécutable parce que les actions ont des conditions préalables et des effets, et un planificateur recherche une séquence qui atteint un objectif. Une théorie logique est exécutable dans un sens plus large parce qu'une procédure d'inférence détermine si une conclusion suit, si un modèle existe ou si les contraintes sont satisfaisantes. Dans chaque cas, l'opération décisive est extérieure à la rhétorique utilisée pour décrire la tâche. Un résultat peut être reproduit à partir d'un artefact et d'une sémantique.

Une réponse LLM est différente. Le modèle cartographie un contexte à une distribution sur les continuations. Il peut internaliser des modèles qui ressemblent à des algorithmes, et il peut résoudre de nombreuses tâches correctement, mais la règle appliquée à une étape particulière n'est pas inspectable séparément ou garanti d'être utilisée de manière cohérente. L'incitation à la chaîne de pensée peut améliorer les performances, mais une chaîne de phrases n'est pas un objet de preuve simplement parce qu'elle est organisée comme des étapes. L'autocohérence échantillonne plusieurs continuations, et non plusieurs exécutions formellement définies. L'utilisation de l'outil peut rendre une valeur exacte tout en laissant l'interprétation qui a produit l'appel d'outil non vérifié. La distinction cruciale est entre un modèle qui propose un artefact exécutable et un modèle dont la propre génération est traitée comme une exécution.

Le langage naturel exécutable commence donc par la matérialisation. Le système doit créer un objet avec une grammaire déclarée, un système de type, une sémantique et une carte source. Cet objet peut être une requête relationnelle, une théorie logique, une machine d'état, un ensemble de contraintes temporelles, une table de décision, un programme Datalog, un automate fini ou un graphique typé avec des opérateurs enregistrés. Le choix particulier dépend de la tâche. Ce qui compte, c'est que le comportement de l'objet ne soit plus déterminé par la distribution du modèle de langage. Il peut être rejeté, exécuté, testé, comparé et rejoué indépendamment.

Cette exigence est plus forte que la production structurée ordinaire. Un modèle peut émettre des JSON syntaxiquement valides qui contiennent des entités inventées ou des champs mutuellement incohérents. Un schéma valide la forme, pas la vérité. De même, un modèle peut générer du SQL légal qui rejoint les mauvaises colonnes, une logique de premier ordre valide qui inverse un quantificateur, ou une formule temporelle qui exprime la relation de préséance opposée. Une frontière de confiance utile n'est donc pas simplement «après la génération JSON». Il se trouve après la mise à la terre de la source, la vérification de type, la compatibilité sémantique et la compilation formelle. Chaque symbole qui affecte la réponse doit avoir une origine admissible et un rôle défini.

L'architecture sépare naturellement quatre autorités. L'autorité source détermine quel texte, quelles données ou observations ont été effectivement fournies. L'autorité sémantique détermine quels types, relations, modalités et politiques d'identité sont autorisés. L'autorité de règle détermine quelles contraintes ou transformations constituent la théorie en cours

d'exécution. L'autorité d'exécution détermine si un résultat candidat suit cette théorie. Un LLM peut aider les quatre, mais il ne devrait pas les fusionner silencieusement. Lorsque le même modèle invente une prémisse, sélectionne une ontologie, applique une règle et écrit le verdict, il n'y a pas de séparation significative entre les preuves et la conclusion.

L'autorité de la source commence par des octets et des portées stables. Un système d'analyse de documents devrait préserver la révision, la structure et l'emplacement exacts de chaque fragment cité. Il devrait distinguer une citation d'un résumé et un résumé d'une inférence. Lorsqu'un modèle propose qu'une phrase exprime une obligation, l'observation doit indiquer la phrase et conserver le modèle, l'invite, le schéma et les solutions de rechange qui ont produit la classification. Si la citation ne se trouve pas dans la source, l'observation est rejetée plutôt que acceptée comme une reconstruction plausible. Cela ressemble à une ingénierie de provenance banale, mais c'est une condition préalable pour chaque garantie ultérieure.

L'autorité sémantique est généralement répartie sur une ontologie supérieure et des paquets de domaine. Une ontologie universelle est attrayante parce qu'elle promet l'interopérabilité, mais elle tend soit à devenir trop abstraite pour trancher des questions utiles, soit trop grande et controversée pour gouverner. Une meilleure conception définit une petite couche commune pour les portées de source, les mentions, les entités, les événements, le temps, les mondes, le statut, la provenance, les alternatives, la couverture et les lacunes. Les forfaits de domaine définissent ensuite des concepts tels que l'intervention clinique, l'obligation réglementaire, l'événement d'objets narratifs, l'appel de logiciel API ou la transaction financière. Chaque paquet doit indiquer non seulement les noms de terrain, mais aussi les engagements et les tests sémantiques. Un type appelé «homologation» n'a pas de sens à moins que le système ne précise s'il désigne un document, un acte, un état, une décision d'autorité ou une combinaison.

L'autorité de la règle devrait être mise en version séparément de l'interprétation du document. Une politique, une théorie du linter ou un modèle scientifique change à un rythme différent de la source qu'il analyse. Si les règles sont incorporées de manière invisible dans les invites, il devient difficile de savoir si deux runs ont appliqué la même théorie. Les artefacts de règles versionnées permettent une publication de référence, des diffs sémantiques et un rollback. Ils permettent également plusieurs théories sur la même représentation de document. Un manuscrit peut être vérifié pour la cohérence de la terminologie, la cohérence causale et le support de citation sans le recompiler en une invite opaque différente pour chaque règle.

L'autorité d'exécution doit être aussi petite et mécanique que pratique. Pour les requêtes relationnelles, il peut s'agir d'un évaluateur déterministe. Pour les contraintes, un solveur SMT ou CSP. Pour le comportement temporel, un modèle de vérificateur. Pour les propriétés de programme spécifiques au chemin, un exécuteur symbolique. Pour les obligations de preuve, un théorème d'essai plus un petit vérificateur de preuve. Pour un langage de circuit personnalisé, il peut s'agir d'un interprète restreint avec des opérateurs enregistrés et des vérificateurs indépendants. Le composant d'acceptation ne doit pas demander à un LLM si le résultat «semble correct». Il devrait vérifier un témoin contre la source et la théorie.

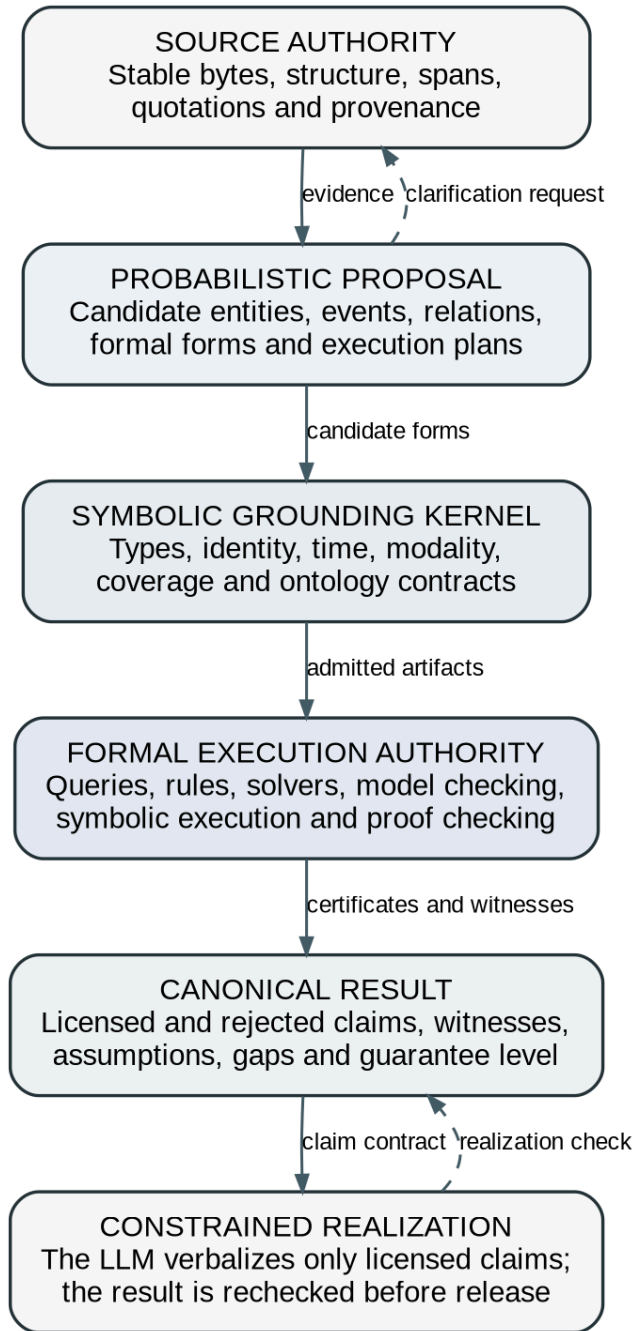


Figure 1. Frontière de confiance pour le langage naturel exécutable. Le LLM propose des formulaires sémantiques, tandis que la base de source, l'exécution formelle, l'acceptation et la licence de réclamation finale restent vérifiables de manière indépendante.

Un moyen utile de classer les systèmes est par l'emplacement de l'oracle final. Dans un système purement neuronal, le LLM est l'oracle. Dans un système à outils, un outil calcule une valeur, mais le LLM peut rester l'oracle pour savoir si l'appel d'outil représente la question. Dans un système soutenu par un solveur, le solveur est l'oracle des conséquences d'une formalisation générée. Dans un système fondé sur des preuves, le solvateur et le noyau de mise à la terre déterminent conjointement le résultat accepté. Dans un système de traitement de la preuve, même la sortie du solveur peut être vérifiée par un noyau de confiance plus petit. Chaque étape réduit l'espace dans lequel l'erreur probabiliste peut changer silencieusement le verdict.

Cela n'implique pas que chaque application a besoin d'une assurance au niveau du théorème. Un assistant d'écriture peut utiliser des circuits déterministes légers, tandis qu'un système de réglementation ou de sécurité peut nécessiter des artefacts de preuve formelle. Le principe important est que la garantie corresponde au mécanisme. Un contrôle régeux déterministe peut certifier la présence d'un motif littéral, et non l'absence de concept. Un extracteur d'événements assisté par modèle peut soutenir un avertissement de continuité proposé, et non un compte rendu mécaniquement certain de l'identité narrative. Un solveur peut établir l'insatisfiabilité des contraintes, pas l'exhaustivité de la traduction en langage naturelle. La rigueur consiste en grande partie à refuser de transférer la confiance au-delà de ces frontières.

L'architecture la plus forte est donc fédérative. La composante neurale n'est pas dissoute dans un réseau symbolique, et la composante symbolique n'est pas réduite à un régularisateur différentiable. Ils restent des systèmes distincts avec une interface typée. Le LLM est responsable de l'ampleur, de la génération d'hypothèses, de la normalisation sémantique et de la récupération à partir d'un langage inhabituel. Le noyau officiel est responsable de l'admissibilité, de l'exécution, des invariants, des témoins et de la reproductibilité. L'examen de 2026 par Chatzikyriakidis et Lappin appelle les systèmes liés à des systèmes fédérateurs et observe qu'ils montrent souvent de forts gains sur des tâches où la perception peut être séparée de l'inférence. Les auteurs soulignent également que les comparaisons restent confondues par les différences de tâches. L'argument architectural pour la fédération est néanmoins indépendant de la supériorité de référence: des modules distincts rendent les réclamations auditable et permettent à un composant de rejeter un autre [Chatzikyriakidis and Lappin 2026].

La production finale en langage naturel requiert sa propre frontière. Une fois qu'un solveur retourne un résultat, il est tentant de fournir la trace à un ZX0000QXZ et de demander une réponse polie. Sans contrainte, le modèle peut ajouter des explications de fond, déduire des motifs ou simplifier l'incertitude en certitude. Un résultat canonique devrait donc énumérer les allégations qui peuvent être verbalisées, leur statut, leurs preuves, leurs hypothèses et leur portée. Le renderer peut choisir le libellé et l'organisation, mais il doit être vérifié par rapport à la structure canonique. Une méthode pratique est la réalisation au niveau de la revendication: le modèle génère des phrases associées à des identifiants de résultat; un vérificateur s'assure que chaque phrase est entraînée par ou paraphrase une revendication autorisée; le matériel non étayé est retiré ou étiqueté comme commentaire. Dans les paramètres d'assurance élevée, le langage contrôlé gabarit peut être préférable à la prose sans restriction.

La limite de confiance proposée dans ce livre peut être résumée sans code. LLMs peut proposer des formes sémantiques, des cartographies ontologiques, des choix de formalisme, des réparations et des explications. Ils n'établissent pas de preuves, ne ferment pas une recherche en monde ouvert, ne certifient pas l'identité, ne décident pas de la satisfaction ou n'autorisent une réclamation finale. Ces fonctions appartiennent à des contrats symboliques explicites. Cette séparation préserve la valeur unique des modèles de langage tout en empêchant leur maîtrise de la preuve.

1.2 L'écart de formalisation : pourquoi un solveur correct peut encore produire la mauvaise réponse

La faiblesse centrale de la plupart des systèmes de raisonnement neuro-symbolique n'est pas le moteur d'inférence. Les solveurs SAT, les solveurs SMT, les moteurs de base de données, les provers de théorème, les vérificateurs de modèles et les exécuteurs symboliques sont souvent beaucoup plus fiables que le modèle de langage qui les précède. La faiblesse réside dans l'écart de formalisation: la différence entre ce que la source signifie dans le contexte et l'objet formel soumis à l'exécution. Un système peut être parfaitement sain de l'entrée formelle à la sortie formelle tout en étant malsain du langage naturel à la réponse.

Envisager une simple phrase de politique: «Un examinateur ne peut approuver une libération qu'après que le rapport de sécurité est terminé, à moins que le commandant de l'incident n'autorise une exception d'urgence.» Un formalisateur doit conserver au moins l'acteur, l'action, l'objet, la condition temporelle, l'état d'achèvement, l'exception, l'autorité et la portée de «seulement après». Il doit distinguer la permission de l'obligation. Il doit décider si l'exception supprime uniquement la condition temporelle ou autorise l'approbation en général. Il doit relier «le rapport de sécurité» à l'artefact correct et «le commandant de l'incident» à une identité ou un rôle. Si l'un de ces choix est faux, un solveur peut appliquer correctement la mauvaise politique.

La portée du quantificateur est une source d'erreur classique. «Chaque chercheur a examiné un article» peut signifier que chaque chercheur a examiné au moins un article, pas nécessairement le même. «Un article a été examiné par chaque chercheur» a une structure différente. La négation crée des problèmes similaires: «tous les échantillons ne sont pas passés» ne signifie pas qu'aucun échantillon n'est passé. Les conditionnels, les comparaisons, la modalité, les relations temporelles, l'anaphore et le présupposé créent tous des distinctions que la traduction au niveau de la surface peut effacer. Les traditions sémantiques formelles ont développé des mécanismes sophistiqués pour ces phénomènes, mais le langage à domaine ouvert ajoute des connaissances mondiales, des conventions de genre et des objectifs implicites qu'aucune représentation ne capture complètement.

L'écart apparaît également dans l'analyse sémantique des dénotations. Un système formé uniquement sur une question et une réponse finale peut trouver un programme qui produit la bonne réponse sur l'environnement de formation. De tels programmes fallacieux sont un problème connu dans l'analyse sémantique faiblement supervisée [Pasupat and Liang 2016; Guu et al. 2017; Goldman et al. 2018]. L'exécution ne distingue pas le programme prévu d'un programme accidentel si les deux donnent la même dénotation. Les systèmes Text-to-SQL font face au même risque: une requête peut renvoyer les bonnes lignes en raison des valeurs de données actuelles tout en contenant la mauvaise jointure ou le mauvais filtre. Le décodage guidé par l'exécution supprime les programmes qui bloquent ou contredisent les contraintes intermédiaires, mais il ne peut pas prouver la fidélité sémantique [Wang et al. 2018].

L'écart de formalisation est mieux décomposé en obligations distinctes. La mise à la terre lexicale demande quels prédicats ou champs correspondent à des mots et des phrases. L'échouement référentiel demande quelles mentions désignent la même entité. La mise à la terre structurelle demande quels arguments, portées et dépendances appartiennent ensemble. La mise à la terre temporelle relie les phrases aux instants, aux intervalles, aux calendriers et à la commande. L'ancrage modal distingue les faits, les possibilités, les obligations, les

autorisations, les intentions, les réclamations déclarées et les contre-factuels. La mise à la terre ontologique détermine si un concept appartient au modèle de domaine sélectionné. La mise à la terre de preuves relie chaque prémisse à une portée source, un capteur, une ligne de base de données ou une hypothèse déclarée. Chaque obligation peut échouer de manière indépendante.

Le système doit également décider de la signification à formaliser. Une représentation sémantique totale de la prose sans restriction n'est ni pratique ni nécessaire pour la plupart des tâches. La règle en cours d'exécution crée la demande. Un circuit terminologique peut nécessiter seulement des portées exactes et des termes normalisés. Une règle de conformité temporelle peut nécessiter des événements, des acteurs, des dates et des exceptions. Une analyse causale peut nécessiter des réclamations, des interventions, des résultats et de l'incertitude. La formalisation axée sur la demande réduit les coûts et limite le nombre d'engagements non soutenus. Il crée également un problème de couverture explicite: si une règle exige un concept que les producteurs disponibles ne peuvent pas se concrétiser, la course doit s'arrêter ou retourner inconnue plutôt que d'approcher silencieusement.

L'ambiguïté doit être représentée comme alternative, et non pas immédiatement effondrée. Supposons que «Alex ait dit à Jordan qu'ils devraient partir» apparaît dans une procédure. Le pronom peut faire référence à Alex, Jordan ou un groupe, selon le contexte et les conventions linguistiques. Un modèle peut classer les candidats, mais une grande confiance n'est pas une preuve. Si le verdict de la règle en aval est le même sous toutes les liaisons plausibles, l'ambiguïté n'est pas pertinente et l'exécution peut se poursuivre. Si le verdict change, le résultat devrait exposer la dépendance et demander des éclaircissements ou un contexte supplémentaire. C'est un endroit où l'interprétation abstraite devient conceptuellement utile: elle peut calculer les conclusions sur un ensemble d'interprétations sans énumérer chaque combinaison en détail.

Une cartographie complète est souvent impossible car le langage naturel est un monde ouvert. Un document peut omettre des faits qui sont vrais, qui assument des connaissances communes ou qui reposent sur la pratique institutionnelle. Le raisonnement du monde fermé traite les faits non enregistrés comme faux; le raisonnement du monde ouvert les traite comme inconnus. Les deux politiques sont appropriées dans des contextes différents. Un inventaire de base de données peut être fermé pour une table et une heure spécifiées. Une collection d'e-mails récupérés n'est généralement pas une preuve complète qu'aucune notification n'a été envoyée. La couche de mise à la terre doit donc fixer une couverture aux observations et aux fouilles. Les conclusions négatives nécessitent un jeton de couverture qui indique le domaine, la révision de la source, la portée, la méthode, les exclusions et la politique d'exhaustivité.

Les connaissances externes présentent d'autres complications. Un LLM général contient des informations étendues mais non versionnées et potentiellement obsolètes. Un système symbolique ne peut pas traiter la mémoire du modèle comme un fait intemporel. La connaissance du monde devrait entrer par des paquets de source, datés, spécifiques à la juridiction et au contenu. Un pack peut affirmer qu'une règle juridique était effective dans une juridiction particulière au cours d'un intervalle particulier ou qu'une constante scientifique a été tirée d'une source nommée. Les conflits entre les paquets doivent rester visibles. Les mondes fictifs ou hypothétiques doivent être sélectionnables sans contamination par les faits du monde actuel. La documentation nllAgent traite correctement de tels packs comme une limite plutôt que de prétendre que la mémoire d'un modèle de fondation fournit une autorité formelle.

La fidélité de la formalisation peut être testée, mais pas universellement prouvée. Une méthode est la vérification bidirectionnelle: traduire l'objet formel en langage contrôlé et le comparer avec la source. Un autre est le test contrastif: générer des changements textuels minimes qui devraient changer un élément formel et préserver le reste. Un troisième est une comparaison extensive: demandez des formalisations concurrentes pour générer des exemples distinctifs ou des résultats de requêtes. Un quatrième est la stabilité de la paraphrase: les formulations sémantiquement équivalentes devraient compiler à des artefacts équivalents ou probablement liés. Un cinquième est l'explication contre-exemple: lorsque le solveur produit un témoin, cartographiez chaque dépendance à la source et demandez si le scénario est linguistiquement admissible. Un sixième est l'examen humain axé sur le petit ensemble de choix qui affectent le résultat plutôt que l'ensemble du programme formel.

Les systèmes interactifs tels que nl2spec illustrent la valeur de la réparation locale. Au lieu de demander à l'utilisateur de comprendre ou de réécrire une formule temporelle entière, l'interface associe des sous-formules à des fragments sources, permettant de corriger un composant erroné [Cosler et al. 2023]. La rétroaction du solveur peut jouer un rôle similaire. Logic-LM utilise des erreurs de syntaxe et d'exécution pour inviter à la reformalisation [Pan et al. 2023]. Les systèmes plus avancés peuvent utiliser des cœurs ou des contre-exemples insatisfaisants plutôt que des messages d'erreur génériques. L'idée de base est que les commentaires formels devraient réduire la question linguistique.

L'écart de formalisation limite également les revendications de «sonorité» de bout en bout. Un système peut prouver un théorème de la forme: si chaque observation acceptée représente avec précision sa source et que l'ontologie et les règles sont correctes, alors le verdict suit. C'est précieux. Il ne doit pas être réécrit comme «le verdict est prouvé à partir du document» à moins que les obligations d'observation et de mise à la terre n'aient également été acquittées. La garantie doit nommer ses hypothèses et ses inconnues. Une enveloppe de solidité est plus honnête qu'une étiquette de confiance binaire.

Ce livre traite donc la formalisation comme un processus de compilation avec plusieurs artefacts intermédiaires plutôt qu'une seule invite. La source est segmentée et ancrée. Les observations des candidats sont produites dans le cadre de schémas. Les alternatives d'identité et de temps sont représentées. La compatibilité de l'ontologie est vérifiée. Un formalisme cible est sélectionné. Le compilateur produit un objet exécutable et une carte source. L'exécuteur produit un témoin, une preuve, un modèle ou un échec. Le résultat est rejoué et rendu. A chaque étape, une erreur a une catégorie spécifique et une réparation possible. L'architecture n'élimine pas l'incertitude sémantique; elle empêche l'incertitude d'être cachée à l'intérieur de la production fluide.

La conclusion de recherche la plus importante suit. Les systèmes LLM soutenus par un solveur doivent être évalués séparément sur la fidélité de formalisation et l'exécution formelle. Un score de réponse finale élevé peut dissimuler des programmes fallacieux. Un score faible peut être causé par une bonne formalisation avec un solveur inadéquat, ou par une mauvaise formalisation avec un solveur parfait. Les repères ont besoin d'or ou de structures formelles admissibles, d'alignements de preuves et de paires minimales sémantiques, et pas seulement de réponses. Jusqu'à ce que cette séparation devienne standard, de nombreuses affirmations sur le «raisonnement formel avec LLMs» resteront difficiles à interpréter.

2. La lignée intellectuelle du langage exécutable

2.1 Langues contrôlées, sémantique formelle, discours et logique naturelle

L'idée que le langage lisible par l'homme peut supporter l'inférence mécanique est antérieure aux grands modèles de langage par décennies. Les travaux antérieurs ont abordé le problème de la direction opposée: plutôt que de demander à un modèle statistique d'interpréter la prose sans restriction, les chercheurs ont conçu des langues ou des théories sémantiques qui limitaient suffisamment l'ambiguïté pour soutenir la traduction en logique. Ces traditions sont essentielles car elles révèlent ce qu'un système de langage exécutable doit préserver même lorsqu'un LLM effectue l'analyse frontale.

Les langues naturelles contrôlées sont le précédent d'ingénierie le plus clair. Ils restreignent le vocabulaire, la grammaire ou l'interprétation tout en conservant une surface qui ressemble au langage ordinaire. La tentative d'anglais contrôlé est l'exemple le plus connu. Les phrases sont traduites en structures de représentation du discours, puis en logique formelle, permettant des requêtes et une inférence tout en restant lisibles pour les non-logiciens [Fuchs and Schwitter 1996; Fuchs et al. 2008]. Les langages contrôlés ont été utilisés pour la représentation des connaissances, les spécifications techniques, les wikis sémantiques, les règles commerciales et les exigences. L'enquête de Kuhn montre que la famille est diversifiée: certaines langues sont conçues pour la clarté humaine plutôt que pour la précision formelle, tandis que d'autres échangent le naturel pour la sémantique déterministe [Kuhn 2014].

La leçon durable n'est pas que la langue sans restriction devrait être remplacée par un dialecte contrôlé. C'est qu'un système exécutable a besoin d'une couche de normalisation dont la sémantique est plus stricte que la prose ordinaire. Un LLM peut servir de traducteur de la langue ouverte à une forme sémantique contrôlée, mais la forme contrôlée doit avoir une grammaire et une interprétation indépendantes. Sans cette frontière, le «langage naturel contrôlé» devient simplement un style d'écriture préféré et ne fournit aucune garantie de calcul.

Les langues contrôlées exposent également l'importance des commentaires des auteurs. Si une phrase est en dehors du fragment contrôlé, un système traditionnel la rejette ou produit une ambiguïté explicite. Les interfaces modernes LLM ont tendance à faire le contraire: elles sélectionnent une lecture plausible et procèdent. Le rejet est souvent traité comme un défaut d'utilisabilité, mais dans un système de haute assurance, c'est une forme d'intégrité épistémique. Une interface de langue exécutable de nouvelle génération devrait combiner la flexibilité de la paraphrase LLM avec l'honnêteté d'un compilateur de langage contrôlé. Il peut proposer plusieurs lectures normalisées, expliquer les distinctions, et ne demander que lorsque la distinction change le résultat.

La sémantique formelle fournit une lignée théorique plus profonde. La sémantique théorique du modèle représente un sens dans des conditions dans lesquelles les expressions sont vraies dans un modèle. La compositionnalité relie l'interprétation d'une expression complexe aux significations de ses parties et à leur combinaison syntaxique. La sémantique de style Montague, les calculs lambda typés, les quantificateurs généralisés et les cadres connexes offrent des comptes précis de portée, de structure d'arguments et d'opérateurs logiques. Ils démontrent que la signification du langage naturel n'est pas réductible à une liste plate de

triples. Les quantificateurs, la négation, les verbes intensionnels, la modalité et le contexte peuvent changer le comportement logique de phrases apparemment simples.

Pour le langage naturel exécutable, la sémantique formelle contribue à deux exigences. Premièrement, la représentation intermédiaire doit préserver la structure de l'opérateur plutôt que seulement les entités et les relations. «Chaque», «certains», «non», «seulement», «à moins», «peut», «doit» et «avant» ne sont pas des annotations autour d'une proposition; ils déterminent l'espace des modèles. Deuxièmement, la construction sémantique doit être consciente du type. Un intervalle temporel, une personne, une organisation, une proposition et une obligation ne peuvent pas être librement substitués simplement parce que leurs étiquettes de surface se ressemblent. La vérification de type est l'un des moyens les moins chers de rejeter les formalisations incohérentes avant d'invoquer un solveur.

La théorie de la représentation du discours et les approches dynamiques-sémantiques connexes abordent les phénomènes que la logique phrase par phrase gère mal. Un discours introduit les référents, les transporte à travers les phrases, met à jour le contexte et crée des contraintes d'accessibilité pour les pronoms et les descriptions. Les analyseurs Boxer et DRS ont mis en œuvre ces idées pour l'analyse sémantique à grande couverture [Bos 2008; Liu et al. 2018]. La banque de signification parallèle et les tâches d'analyse DRS partagées ont permis d'évaluer les représentations de signification étendue entre les langues [Abzianidze et al. 2019]. Ces efforts montrent à la fois le progrès et la difficulté: la liaison variable, l'attribution de sens, la portée et les relations de discours sont beaucoup plus difficiles que l'extraction peu profonde.

Un système exécutable au niveau du document devrait apprendre de la sémantique dynamique sans nécessiter un ZX0000QXZ monolithique pour chaque source. Il a besoin d'objets de discours persistants, d'hypothèses d'entités étendues, d'événements, de mondes et de relations de mise à jour. Il doit distinguer une mention d'une entité et d'un lien d'identité candidat à partir d'un lien établi. Il doit préserver les contextes cités, hypothétiques, niés et fictifs. Un graphique de connaissance plat qui fusionne des chaînes égales en un seul nœud fera des erreurs systématiques dans les documents longs. Les noms se répètent, les alias se produisent, les rôles changent et les documents décrivent des mondes mutuellement incohérents.

Le coût pratique de l'analyse sémantique formelle complète explique pourquoi de nombreux systèmes se sont déplacés vers des représentations moins profondes. L'étiquetage des rôles sémantiques identifie la structure des arguments prédicatifs. La représentation abstraite signifie représente les concepts et les relations dans un graphique. L'extraction d'informations identifie les entités, les événements et les relations. Ces représentations sont utiles, mais leur sémantique est souvent incomplète pour l'exécution. Ils peuvent omettre la portée du quantificateur, la modalité, l'ancrage temporel et la différence entre le contenu affirmé et le contenu déclaré. Une exécution en langage exécutable devrait traiter de telles structures comme des observations porteuses de preuves plutôt que comme une forme logique complète. Un graphique peut prendre en charge de nombreuses règles locales tout en laissant les aspects non résolus explicites.

La logique naturelle offre un autre compromis important. Au lieu de traduire les phrases en un langage formel sans restriction, les systèmes de logique naturelle raisonnent par des transformations structurées près de la forme de surface. MacCartney et Manning ont développé des relations d'inférence basées sur la monotonie, l'implication lexicale et la projectivité [MacCartney and Manning 2007, 2009]. NaturalLI a ensuite utilisé des opérateurs de logique

naturelle pour une inférence efficace sur le texte [Angeli and Manning 2014]. LangPro combine une grammaire formelle, des connaissances lexicales et un théorème prouvant pour l'inférence en langage naturel [Abzianidze 2017].

La logique naturelle est attrayante parce que de nombreuses questions de documents impliquent des relations qui peuvent être capturées sans construire un grand modèle mondial. Le remplacement de «chien» par «animal» est valable dans un contexte ascendant mais pas nécessairement sous négation ou restrictions universelles. Ces calculs sont plus transparents qu'un score de similarité d'intégration. Ils peuvent soutenir les contrôles de contradiction, le raisonnement taxonomique et l'inférence contrôlée. Cependant, la logique naturelle n'est pas un exécuter universel. Il gère bien certaines monotonicités et relations lexicales, mais n'exprime pas naturellement de procédures arbitraires, d'arithmétique, de planification ou de comportement temporel complexe. Dans une architecture multiformalisme, il devrait être un moteur parmi plusieurs.

Les traditions sémantiques expliquent également pourquoi la «mise à l'échelle» a plusieurs niveaux. La sémantique formelle peut spécifier comment une expression contribue aux conditions de vérité, mais elle ne détermine pas automatiquement quel objet du monde réel un nom désigne ou si une source est crédible. DRS peut représenter un référent de discours, mais la résolution d'identité reste une tâche empirique. La logique naturelle peut propager l'implication lexicale, mais la relation lexicale peut dépendre du domaine. L'exécutabilité nécessite donc un partenariat entre la structure linguistique, l'ontologie de domaine et la gestion des données probantes.

Une architecture utile peut distinguer au moins trois produits sémantiques. Le premier est un modèle source contenant la structure exacte, les portées et les citations. La seconde est une couche d'interprétation contenant des mentions, des événements, des relations, des modalités et des alternatives typées. La troisième est une théorie exécutable qui utilise des interprétations sélectionnées pour répondre à une question particulière. Les techniques sémantiques formelles informent principalement la deuxième couche. Les langages contrôlés et les preuves de théorème connectent les deuxième et troisième. La provenance et la couverture se connectent toutes deux au premier.

La littérature antérieure met également en garde contre le fait de supposer qu'une représentation peut servir chaque tâche. DRS est expressif pour le discours mais coûteux à produire et à raisonner. La logique du premier ordre est bien comprise mais gênante pour les phénomènes temporels, probabilistes ou défaisibles. Les logiques de description offrent un raisonnement ontologique décidable mais restreignent l'expressivité. La logique naturelle reste proche du langage mais est spécialisée. L'anglais contrôlé améliore la prévisibilité en limitant les entrées. La bonne conclusion n'est pas de choisir un gagnant, mais de conserver suffisamment d'informations sémantiques pour réduire une tâche dans le formalisme qui l'intègre.

LLMs change l'économie de cette conception. Ils peuvent générer des analyses de candidats, réécrire un langage non contrôlé en formes contrôlées, suggérer des cartographies ontologiques et expliquer les alternatives formelles aux utilisateurs. Ils font qu'il est pratique de tenter une interprétation sémantique sur des documents que les grammaires manuscrites rejetteraient. Ils n'enlèvent pas la nécessité des distinctions sémantiques développées par les travaux antérieurs. Au contraire, ces distinctions deviennent les contrats par lesquels les propositions de LLM sont jugées.

Une erreur récurrente dans les systèmes contemporains est de traiter un triple magasin généré par LLM comme une représentation sémantique complète. Les triples sont pratiques pour la récupération et les requêtes graphiques, mais ils n'encodent pas en eux-mêmes la portée, la provenance, la modalité, le temps, la négation, les alternatives d'identité ou la couverture. Une couche sémantique exécutable doit être plus riche mais toujours finie et opérationnelle. Il peut stocker une affirmation en tant qu'objet typé avec les participants, le contexte temporel, la polarité, la modalité, le monde, les preuves, le producteur et le statut. Il peut représenter des fixations alternatives sans en choisir une. Il peut déclarer quelles dimensions sont absentes. C'est moins élégant qu'une seule forme logique, mais c'est plus fidèle à la façon dont les systèmes réels acquièrent du sens.

La contribution la plus profonde de la lignée pré-LLM est donc la discipline conceptuelle. La lisibilité humaine n'implique pas de détermination sémantique. La détermination formelle est obtenue par la restriction, la frappe, le contexte explicite et le rejet des constructions non soutenues. Le sens du document est dynamique et étendu. L'inférence près du langage de surface est possible mais spécialisée. Plusieurs représentations formelles sont légitimes car elles répondent à différentes questions. Ces leçons définissent l'architecture qu'un compilateur basé sur LLM doit récupérer plutôt que de contourner.

2.2 Analyse sémantique, requêtes exécutables, induction de programmes et premiers systèmes neuro-symboliques

L'analyse sémantique convertit les énoncés de langage naturel en objets formels qui peuvent être exécutés sur une base de données, une base de connaissances, un environnement ou un interprète. C'est le plus proche prédécesseur historique du pipeline en langage exécutable proposé ici. Le champ comprend la réponse aux questions sur les tableaux et les graphiques de connaissances, le texte à SQL, la suite d'instructions, la synthèse d'expression régulière, l'interprétation de la commande robotique et la génération de programmes. Ses succès démontrent le pouvoir de l'exécution extérieure; ses échecs révèlent pourquoi l'exécution seule n'établit pas la compréhension.

Les premiers analyseurs sémantiques reposaient sur des grammaires, des lexiques et des formes logiques supervisées. Ils pouvaient produire des représentations transparentes, mais nécessitaient des annotations coûteuses et souvent mal transférées dans tous les domaines. La faible supervision a réduit le fardeau de l'annotation par la formation à partir de dénnotations: une question est associée à une réponse, et le système recherche un programme qui le donne. Cette approche a permis des ensembles de données plus larges, mais elle a créé le problème du programme fallacieux. De nombreux programmes peuvent renvoyer accidentellement la même réponse, en particulier dans les petites tables ou les bases de connaissances. Le signal d'entraînement récompense alors le comportement sans identifier la sémantique [Pasupat and Liang 2016] prévue.

Le problème est structurel, pas seulement statistique. Supposons que la réponse à «Quelle ville a accueilli l'événement 2012?» est Londres. Un programme sélectionnant la ligne avec l'année 2012 est prévu. Un programme sélectionnant la première ligne peut également renvoyer Londres dans la table d'entraînement. Plus de données peuvent réduire les corrélations accidentelles, mais ne peuvent pas garantir que le programme appris reflète la langue. Des exemples abstraits, des modèles d'alignement, des contraintes de type et un filtrage basé sur l'exécution ont été développés pour supprimer les candidats pursifs [Goldman et al. 2018; Lee et al. 2023]. Ces techniques sont également importantes pour la formalisation LLM. Un résultat de solveur doit être traité comme un test d'une formalisation, et non comme son certificat sémantique.

Les machines symboliques neurales combinent la génération de programmes neuronaux avec un interprète Lisp symbolique [Liang et al. 2017]. La composante neuronale propose des programmes, tandis que l'exécuteur testamentaire rejette les candidats invalides et renvoie les notations. Mou et ses collègues ont explicitement couplé l'exécution neuronale distribuée avec l'exécution symbolique pour les requêtes en langage naturel, en signalant des améliorations de précision, d'efficacité et d'interprétabilité [Mou et al. 2017]. Ces systèmes ont établi un modèle durable: les modèles neuronaux gèrent la variabilité du langage; les interprètes symboliques imposent une signification opérationnelle.

Text-to-SQL a rendu le modèle commercialement important. Une question utilisateur est compilée dans une requête dont l'exécution renvoie des données. Le décodage guidé par l'exécution peut exécuter des candidats SQL partiels ou complets et rejeter ceux qui produisent des erreurs ou des résultats impossibles [Wang et al. 2018]. Le schéma liant limite les noms et les relations. Le moderne LLMs améliore considérablement la génération, mais la généralisation inter-domaines, les questions ambiguës, les conventions spécifiques à la base de données et

les demandes sans réponse restent difficiles. La principale leçon de langage naturel exécutable est que l'environnement formel lui-même fournit des contraintes utiles. Les types de colonnes, les clés étrangères et les erreurs de requête fournissent un retour d'information que la modélisation du langage pur manque.

Pourtant, text-to-ZX10000QXZ montre également les limites d'une seule métrique de précision finale. La correspondance de chaîne exacte peut rejeter les requêtes équivalentes. La précision de l'exécution peut accepter des requêtes sémantiquement erronées qui doivent renvoyer le même résultat. Une évaluation rigoureuse nécessite une équivalence dans des instances de base de données variées ou des contre-exemples soigneusement construits. Ceci est directement analogue à l'évaluation de la formalisation des règles: un programme formel généré devrait être testé sur des scénarios distinctifs, et pas seulement sur une seule réponse observée.

La synthèse du programme à partir du langage naturel généralise l'idée. Les systèmes génèrent des expressions régulières, des extraits de code, des transformations de données ou de petits programmes spécifiques au domaine. Les exemples peuvent restreindre l'espace candidat, et les systèmes de type peuvent éliminer les programmes absurdes. La génération de neurones d'expressions régulières a montré que la cartographie langue-programme peut fonctionner même avec des connaissances de domaine limitées, tout en illustrant la fragilité lorsque les descriptions sont sous-spécifiées [Locascio et al. 2018]. LLMs aidé par programme plus tard a déplacé le calcul en dehors du modèle, mais le problème de synthèse fondamental est resté: le modèle doit choisir un programme dont la sémantique correspond à la demande.

Un compilateur robuste devrait donc rechercher de multiples formes de preuves. Les descriptions en langage naturel limitent l'intention. Les exemples d'entrée-sortie limitent le comportement d'extension. Les types limitent les opérations admissibles. Les schémas de domaine limitent le vocabulaire. Tests sondez les cas de bord. Un programme généré qui satisfait à toutes ces contraintes est plus crédible qu'un programme qui ne fait qu'exécuter. Ce principe multi-preuves peut être appliqué aux règles du document: autorité textuelle, exemples positifs et négatifs, résultats attendus et mutations sémantiques devraient tous façonner un circuit avant la publication.

La programmation logique fournit une autre cible exécutable. Les règles Datalog calculent les conséquences d'un point de correction et prennent en charge l'évaluation incrémentielle. Prolog et les langues associées représentent les relations et la recherche. L'interprétation abstraite a longtemps été utilisée pour analyser les programmes logiques, montrant un pont conceptuel entre la sémantique déclarative et l'analyse de programme. Pour les applications en langage naturel, la programmation logique est bien adaptée aux fermetures monotones: relations transitoires, taxonomies, propagation de la dépendance et règles de politique sans défauts complexes. Il est moins approprié lorsque le raisonnement dépend fortement de l'incertitude, des sources incohérentes ou des exceptions non monotoniques, à moins d'être étendue avec des mécanismes explicites.

Les tableaux de décision et les systèmes de production sont souvent négligés dans les enquêtes universitaires, mais sont très pertinents. De nombreuses politiques ne sont pas des théories logiques profondes; ce sont des combinaisons finies de conditions et de résultats. Une table de décision rend visible la couverture, les conflits et les combinaisons manquantes. Il peut être compilé en prédicats exécutables et testés systématiquement. Le langage naturel peut être utilisé pour écrire ou expliquer le tableau, tandis que le tableau reste la sémantique faisant

autorité. Pour la linting de documents d'entreprise, les tables de décision peuvent être plus sûres et plus faciles à valider que la logique arbitraire de premier ordre.

Les instructions temporelles et procédurales exigent des cibles plus riches. Les machines à états finis, les arbres de comportement, les langages de planification et les logiques temporelles représentent des actions et des conditions ordonnées. La robotique a longtemps utilisé des analyseurs de langage qui composent des primitives exécutables. Les systèmes de manipulation neuro-symbolique récents poursuivent ce modèle: un analyseur de langage mappe une instruction à un programme dont les composants peuvent inclure la perception neuronale et le comptage symbolique, le raisonnement spatial ou le contrôle d'action. Le succès de ces systèmes dépend d'une bibliothèque de compétences restreintes et d'un environnement fondé, et non d'une couverture sémantique illimitée [Tziafas and Kasaei 2026].

La programmation logique probabiliste et les systèmes neuro-symboliques différentiables tentent une intégration plus étroite. DeepProbLog, Réseaux de tenseurs logiques et systèmes associés combinent des prédicats appris avec la structure logique [Manhaeve et al. 2018; Badreddine et al. 2022]. Ils sont précieux lorsque l'incertitude et l'apprentissage sont intrinsèques, et ils peuvent injecter des contraintes dans la formation. Toutefois, la différenciation n'est pas équivalente à l'assurance formelle. Les valeurs de vérité floues et les pénalités douces peuvent guider les modèles sans produire d'obligations de preuve discrètes. Pour le langage naturel exécutable, de tels systèmes peuvent estimer ou classer les candidats sémantiques, mais une limite d'acceptation de haute assurance bénéficie toujours d'un vérificateur symbolique distinct.

La revue neuro-symbolique 2026 NLP distingue les systèmes injectifs, qui compilent des informations symboliques dans les modèles neuronaux, des systèmes fédératifs, qui préservent les composants autonomes. Il rapporte que les systèmes injectifs dominent la littérature mais montrent souvent des gains modestes sur de larges tâches NLU, tandis que les systèmes fédératifs montrent des gains plus importants sur les tâches de raisonnement. Les auteurs avertissent explicitement que la comparaison est confondue par différents repères et que la généralisation hors domaine reste un problème sérieux [Chatzikyriakidis and Lappin 2026]. L'interprétation équilibrée est que la fédération est architecturalement attrayante pour l'auditabilité et la décomposition des tâches, mais pas encore empiriquement prouvée comme une stratégie de mise à l'échelle universelle.

La lignée sémantique clarifie également le rôle d'un langage intermédiaire. Une représentation doit être suffisamment expressive pour capturer une signification pertinente à la tâche, mais suffisamment contrainte pour compiler de manière fiable. S'il est trop proche du texte de surface, l'exécution reste implicite. Si c'est trop général, la traduction et la complexité du solveur deviennent ingérables. Beiser et Penz décrivent cela comme un problème de langage intermédiaire dans les systèmes neuro-symboliques contemporains: choisir un formalisme peut se déplacer plutôt que de résoudre la difficulté [Beiser and Penz 2025]. Une architecture multi-formalisme réduit la pression sur n'importe quelle langue, mais introduit une nouvelle exigence pour le routage typé et la cohérence du formalisme croisé.

Un pivot sémantique utile ne devrait donc pas prétendre être la logique finale. Il devrait préserver les preuves et les distinctions neutres – entités, mentions, événements, relations, temps, modalité, quantités, mondes, provenance, alternatives et couverture – tout en permettant des baisses dans les exécuteurs supérieurs spécialisés. Certaines requêtes peuvent être répondues directement sur le pivot en utilisant des opérations relationnelles. D'autres

nécessitent une formule temporelle compilée, un système de contrainte ou une machine d'état. Le pivot est précieux parce qu'il sépare la compréhension du langage de n'importe quel solveur particulier tout en conservant suffisamment d'informations pour l'exécution basée sur la source.

Le programme de recherche pré-LLM a obtenu un succès substantiel mais limité dans le domaine. Il a montré que le langage naturel peut être compilé en programmes exécutables, que les interprètes symboliques améliorent la systématisme, que les types et les schémas réduisent les erreurs et que la faible supervision crée des raccourcis sémantiques. Il a également montré qu'une large couverture et une sémantique exacte sont en tension. LLMs améliore considérablement la couverture et la génération de candidats, mais ils héritent de tous les problèmes structurels identifiés par l'analyse sémantique. La synthèse correcte est de ne pas rejeter la machinerie antérieure. Il s'agit de placer LLMs devant et autour de lui, avec des interfaces formelles qui rendent les anciennes leçons exécutoires à une nouvelle échelle.

3. Interprétation abstraite et exécution symbolique pour le langage

3.1 Interprétation abstraite comme une sémantique de l'incertitude bornée

L'interprétation abstraite a été développée comme une théorie générale pour calculer les approximations sûres du comportement du programme [Cousot and Cousot 1977]. Un programme peut avoir infiniment d'entrées et d'états, ce qui rend l'exécution exhaustive impossible. Au lieu d'énumérer chaque état, une analyse représente des ensembles d'états à travers des propriétés abstraites telles que des intervalles, des signes, des types d'objets possibles, des définitions ou des relations d'alias. Les opérations abstraites sont conçues de manière à ce que tout comportement concret pertinent pour la propriété reste représenté. L'analyse peut inclure des comportements impossibles et donc signaler de fausses alarmes, mais une sur-approximation sonore n'omet pas silencieusement un comportement que la sémantique concrète permet.

L'idée opérationnelle est simple. Une exécution concrète suit les valeurs exactes. Une analyse d'intervalle peut suivre qu'une variable se situe entre zéro et dix. Lorsque le programme en ajoute cinq, l'opération de transfert abstrait produit un intervalle compris entre cinq et quinze. Dans une branche, les états sont affinés selon la condition. Lors d'une fusion, l'information est jointe. Pour les boucles et les processus récursifs, l'analyse recherche un point de correction. L'élargissement peut forcer la convergence en sacrifiant la précision, tandis que le rétrécissement peut récupérer quelques détails par la suite. Le cadre mathématique fournit un vocabulaire discipliné pour la précision, la solidité, la convergence et le raffinement.

L'application du cadre au langage nécessite une sémantique concrète. L'objet approché ne peut pas être «signifiant» dans un sens indéfini. Pour une tâche bornée, le domaine concret peut être l'ensemble de structures sémantiques admissibles sous une grammaire, une ontologie, une politique de mise à la terre, une révision de source et une théorie de fond. Chaque structure contient des entités possibles, des affectations d'événements, des relations temporelles, des modalités et des liens sources. L'ambiguïté crée plusieurs structures. L'information manquante laisse les dimensions sans contrainte. Les sources contradictoires peuvent générer des mondes alternatifs plutôt qu'une union incohérente.

Un état sémantique abstrait résume cet ensemble. Il peut indiquer qu'un pronom peut désigner l'une ou l'autre des deux personnes, qu'un événement se produit dans un intervalle de temps, qu'une obligation est présente sous une interprétation et absente sous une autre, ou qu'une quantité se trouve dans une fourchette. Un fait incontournable dans chaque interprétation représentée. Un fait peut tenir dans au moins un. Un fait ne tient en aucun cas. Inconnu indique que le système n'a pas la représentation, la couverture ou les ressources de calcul nécessaires pour décider. Cette distinction à quatre est plus informative qu'une probabilité seule et plus honnête que de forcer chaque question à être vraie ou fausse.

Considérez un document de politique indiquant: «L'opérateur envoie l'avis après validation. Si le mode d'urgence est actif, le superviseur peut renoncer à la période d'attente.» Supposons qu'il ne soit pas clair si «la période d'attente» se réfère à la période avant validation ou à une période ultérieure. Une seule parse fait un choix fragile. Un état abstrait peut retenir deux

modèles temporels. Si les deux impliquent que cet avis doit suivre la validation, cette conclusion est nécessaire. Si un seul permet un préavis immédiat, l'envoi immédiat est peut-être. Une règle de conformité peut identifier quel verdict dépend de la phrase ambiguë et demander des éclaircissements seulement si nécessaire.

Le principal avantage n'est pas que l'interprétation abstraite comprend magiquement le langage. C'est qu'il donne une sémantique d'exécution de principe une fois qu'un ensemble limité de lectures a été construit. Il indique au système comment agréger les alternatives sans confondre la possibilité avec nécessité. Il prend également en charge les domaines modulaires. L'identité, le temps, la quantité, la modalité, la provenance et le statut normatif peuvent chacun avoir leur propre abstraction, puis être combinés par des produits réduits qui échangent des informations lorsque cela est utile.

Un domaine identitaire peut représenter des classes d'équivalence candidates avec des relations explicites identiques, différentes et non résolues. Il ne doit pas fusionner les entités simplement parce que les noms correspondent. Un domaine temporel peut utiliser des ordres partiels, des intervalles ou des contraintes de différence. Un domaine de modalité peut distinguer le contenu affirmé, rapporté, prévu, autorisé, requis, interdit, hypothétique et contre-factuel. Un domaine de quantité peut utiliser des valeurs exactes, des intervalles, des unités et une compatibilité dimensionnelle. Un domaine de provenance peut ordonner des statuts de preuve tels que extrait, proposé, supposé, dérivé, vérifié et confirmé par l'homme sans permettre la confiance d'effacer leur origine. Un domaine de couverture peut vérifier si une recherche est suffisamment complète pour supporter les réclamations d'absence.

Les domaines doivent être choisis par la propriété en cours de vérification. Un linter de terminologie n'a pas besoin d'un réseau déontique riche. Une règle temporelle n'a pas besoin d'une sémantique lexicale complète si les événements et les dates sont déjà matérialisés. Cette sélection axée sur les tâches réduit la complexité et reflète l'analyse statique axée sur la demande. Dans un long temps d'exécution de document, les circuits ou les règles déclarent les types d'observation et les dimensions abstraites dont ils ont besoin. Le compilateur ne matérialise que ces dimensions et rejette l'exécution lorsqu'un domaine critique ne peut pas être fourni.

Les fonctions de transfert sont la partie la plus difficile. Dans l'analyse de programme ordinaire, la sémantique d'une affectation ou d'une opération arithmétique est fixée par le langage de programmation. Dans l'analyse en langage naturel, l'opération qui met à jour l'état sémantique peut être proposée par un LLM. Une phrase pourrait introduire un événement, modifier un état antérieur, établir une relation temporelle, créer une exception ou signaler la croyance de quelqu'un d'autre. Si un traducteur savant devient directement la fonction de transfert, la solidité classique est perdue.

Une conception défendable sépare la génération de candidats du transfert admis. Le LLM propose un ensemble fini de mises à jour sémantiques typées avec des portées et des alternatives sources. Un matérialisateur déterministe valide l'existence de la preuve citée, les champs satisfont un schéma, les unités et les types sont compatibles, les références pointent vers des objets admissibles, et aucun statut n'est mis à niveau silencieusement. L'interpréteur abstrait applique alors des fonctions de transfert enregistrées aux dossiers admis. L'analyse qui en résulte est solide par rapport à l'ensemble de candidats admis, pas nécessairement par rapport à chaque interprétation qu'un humain pourrait reconnaître. Cette condition doit être énoncée explicitement.

Il existe plusieurs façons de renforcer l'hypothèse du candidat. Plusieurs invites ou modèles indépendants peuvent proposer des alternatives. Les générateurs à base de grammaire ou d'ontologie peuvent assurer la couverture pour les fragments contrôlés. Les variantes de paraphrase peuvent tester la stabilité. Des exemples contrastifs peuvent révéler des interprétations manquantes. L'examen humain peut se concentrer sur les ambiguïtés pertinentes pour les résultats. Un compilateur peut délibérément surgénérer au sein d'une grammaire bornée et laisser les contraintes ultérieures éliminer les candidats impossibles. Chaque méthode échange le coût par confiance que l'ensemble contient toutes les lectures matériellement plausibles.

L'interprétation abstraite est également pertinente pour la propagation de règles à l'échelle du document. De nombreux problèmes de peluche et de conformité ressemblent à l'analyse des flux de données. Les faits sont générés à des endroits sources, propagés par le biais de portées, joints à travers des sections et invalidés par des déclarations ultérieures. Une définition peut être disponible après sa déclaration mais pas avant. Un objet peut changer d'état à travers un récit. Une exigence peut s'appliquer dans une juridiction ou une version. Une réclamation peut obtenir des preuves plus tard. Un moteur de point de correction incrémental peut calculer les conséquences et mettre à jour uniquement les régions touchées lorsque le document change.

Cette perspective évite de réexécuter un LLM général sur un document entier après chaque édition. Les artefacts structurels et sémantiques stables peuvent être mis en cache par le contenu digest. Un paragraphe modifié invalide les observations locales et les états abstraits en aval qui en dépendent. Le graphique d'analyse identifie les circuits qui doivent être rejoués. Ceci est analogue aux cadres de compilation et d'analyse statique incrémentaux. Il rend également les explications précises: le système peut montrer quel changement de source a modifié quel invariant abstrait et pourquoi.

L'élargissement et la perte de précision ont des analogues sémantiques. Un long récit peut contenir des centaines d'identités d'objets possibles ou de relations temporelles. Conserver chaque combinaison est peu pratique. Le système peut résumer «l'un de ces objets», «un certain temps dans cet intervalle» ou «l'une de ces juridictions». Un tel élargissement peut introduire des avertissements fallacieux. Le rétrécissement ou le raffinement local peut retrouver la précision autour d'une violation présumée. L'objectif du design n'est pas d'éviter l'abstraction mais de rendre visible et réversible sa perte de précision.

La littérature établie NLP sur la robustesse certifiée utilise des idées d'interprétation abstraite d'une manière différente. Jia et ses collègues ont utilisé la propagation liée à l'intervalle pour entraîner des modèles dont les prédictions sont certifiées contre chaque substitution de mots dans une famille prédéfinie, rapportant une précision contradictoire de 75% sur IMDB et SNLI sous leur modèle de menace [Jia et al. 2019]. Plus tard, les travaux ont utilisé le lissage randomisé, les intégrations conscientes de la robustesse et les domaines de transformateur spécialisés pour améliorer les limites certifiées [Ye et al. 2020; Wang et al. 2023; Huang et al. 2026]. Ce sont de véritables garanties, mais le domaine concret est un ensemble d'entrées de modèle perturbées, pas un ensemble d'interprétations sémantiques. La famille de transformation est définie à l'extérieur et supposée préserver l'étiquette.

Cette distinction est importante. Certifier que la prédiction d'un classificateur est invariante sous toutes les substitutions dans une liste de synonymes ne prouve pas que chaque substitution préserve le sens de la phrase dans son contexte. Il ne prouve pas que le

classificateur est correct sur la phrase originale. Il certifie la robustesse par rapport à un ensemble de perturbations formelles. Cette littérature est toujours très pertinente car elle démontre comment les domaines abstraits peuvent évoluer sur les entrées de langage combinatoires et comment la précision de la vérification dépend de la représentation des corrélations. Il fournit également un avertissement méthodologique: la revendication formelle la plus forte est toujours relative à l'ensemble de béton réellement défini.

Une direction de recherche prometteuse est de connecter les deux utilisations. L'analyse neuronale certifiée pourrait lier le comportement du modèle de proposition sémantique sous des paraphrases contrôlées, tandis que l'interprétation abstraite sémantique pourrait lier les conséquences d'interprétations alternatives fondées. Le premier demande si l'extraction d'un modèle change en vertu de transformations de surface autorisées. Ce dernier demande si une conclusion au niveau du document change entre les structures sémantiques admissibles. Ensemble, ils pourraient déterminer si l'incertitude trouve son origine dans l'instabilité du modèle ou une véritable ambiguïté sémantique.

LLMs peut également aider à construire des interprètes abstraits, mais uniquement sous acceptation mécanique. SAIL démontre ce principe en utilisant LLMs pour rechercher des transformateurs abstraits tout en appliquant des contraintes syntaxiques et sémantiques et en optimisant un coût qui mesure la mésonacité. Les transformateurs acceptés sont globalement sains et le système peut synthétiser des transformateurs précis pour les opérateurs non linéaires qui n'étaient pas disponibles auparavant [Gu et al. 2026]. La leçon plus large est architecturale: les modèles génératifs peuvent explorer un immense espace de conception si un petit validateur formel contrôle l'admission.

Demander directement à un LLM d'effectuer une interprétation abstraite est moins fiable. Les expériences sur les repères d'analyse de programme montrent que les modèles peuvent suivre des styles d'interprétation abstraite, mais font des erreurs logiques sur des structures complexes et hallucinent des états [Mitchell et al. 2025]. Ce résultat devrait tempérer les propositions dans lesquelles un modèle raconte un invariant et la narration est acceptée comme analyse. Le modèle est précieux pour suggérer des invariants, des partitions de domaine ou des prédicats de raffinement. Le calcul du point de correction et les vérifications de solidité doivent rester symboliques.

Pour le langage naturel exécutable, l'interprétation abstraite n'est donc ni une métaphore ni une solution complète. C'est une théorie pour l'approximation disciplinée après que le système a défini ce qui compte comme une interprétation possible. Il prend en charge la sémantique peut/mus, la propagation incrémentale, la composition du domaine, le raisonnement d'absence conservateur et le raffinement ciblé. Sa garantie est conditionnelle à la couverture sémantique. Cette limitation n'est pas fatale; c'est le point exact où l'interprétation assistée par modèle doit être évaluée et régie.

3.2 Exécution symbolique, résolution de contraintes, vérification de modèle et raffinement sémantique

L'exécution symbolique classique exécute un programme avec des entrées symboliques plutôt que des valeurs spécifiques [King 1976]. Un état symbolique contient un emplacement de contrôle, des expressions symboliques pour les variables de programme et une condition de chemin décrivant quelles entrées atteignent cet état. Dans une branche, l'exécution forks. Un chemin ajoute la condition de la branche; l'autre ajoute sa négation. Un solveur de contraintes détermine si les conditions sont satisfaisantes et peut produire des intrants concrets qui réalisent un chemin réalisable. La méthode est puissante parce qu'elle produit des témoins: non seulement un avertissement qu'une erreur peut exister, mais une entrée et un chemin qui le démontrent.

La difficulté est l'explosion du chemin. Les branches se multiplient, les boucles et la récursion créent des chemins non liés, les théories complexes remettent en question les solveurs et les bibliothèques externes sont difficiles à modéliser. Les sondages d'exécution symbolique décrivent un grand espace de conception de stratégies de recherche, de fusion d'état, de résumés de composition, d'exécution concolique, de modèles d'environnement et d'optimisations de solveurs [Baldoni et al. 2018]. Aucun moteur n'explore tous les comportements de logiciels réels. Les allégations d'exécution symbolique comprennent donc les limites, les hypothèses de modélisation et les limitations de couverture.

Le langage naturel se connecte à l'exécution symbolique de plusieurs manières distinctes. Le plus simple est le langage-à-code: un LLM génère un programme, et un exécuteur symbolique conventionnel analyse ce programme. Un autre est la langue à la spécification: le modèle génère une propriété ou une contrainte, qui est vérifiée par rapport au code existant. Un troisième est l'exécution de programme sémantique: un document est compilé dans une machine d'état ou un programme de règles dont les interprétations alternatives deviennent des choix symboliques. Un quatrième est l'exploration assistée par LLM: un modèle priorise les chemins, propose des résumés, ou des raisons sur les contraintes que les solveurs conventionnels ne peuvent pas facilement exprimer.

La terminologie doit rester précise. L'exécution de SQL, Python ou d'une forme logique générée à partir d'une question est une exécution formelle, mais ce n'est pas nécessairement une exécution symbolique au sens sensible du chemin. La différence est importante parce que l'exécution symbolique offre des conditions de chemin et des témoins concrets, tandis que l'exécution ordinaire fournit un résultat pour un état concret. Les deux sont utiles. Une étude de synthèse qui les regroupe sans distinction fait que les garanties semblent plus fortes qu'elles ne le sont.

SymGPT fournit l'un des exemples de langue à spécification les plus clairs. Le système étudie 132 règles en langage naturel à partir de trois normes ERC, les traduit par un LLM en une grammaire restreinte, synthétise les contraintes qui décrivent les violations et utilise l'exécution symbolique pour rechercher des contrats Ethereum. Les auteurs font état de 5.783 violations dans 4.000 contrats, dont 1.375 avec des voies d'attaque explicites pour le vol, et rapportent des performances plus solides que six techniques automatisées et un service d'audit [Xia et al. 2026] d'experts en sécurité. La force de la conception réside dans la grammaire étroite, la synthèse systématique des contraintes et les trajectoires de programme concrètes.

Son risque sémantique restant réside dans la question de savoir si chaque règle ERC a été traduite fidèlement.

Ce modèle généralise. Une politique peut être compilée dans des conditions préalables, des transitions interdites et des exceptions. Une procédure peut être compilée en états et en actions. Une règle de continuité narrative peut modéliser des objets et des événements, puis rechercher un chemin dans lequel un objet est utilisé après avoir été laissé sans récupération. Un protocole clinique peut modéliser l'éligibilité et les séquences d'intervention. Un flux de travail scientifique peut modéliser les transformations de données et les exigences de provenance. Dans chaque cas, l'exécuteur testamentaire symbolique devrait fonctionner sur un programme typé plutôt que sur une prose brute.

L'exécution symbolique est particulièrement utile pour les contre-exemples. L'analyse abstraite peut indiquer qu'une violation est possible parce qu'elle a perdu une corrélation. Une recherche de chemin peut tenter de choisir des identités, des temps, des conditions et des exceptions concrètes qui réalisent la violation. Si le chemin est satisfaisant, le témoin peut être rendu comme un scénario source. S'il est insatisfaisant, le système apprend que l'avertissement abstrait était faux sous la sémantique actuelle.

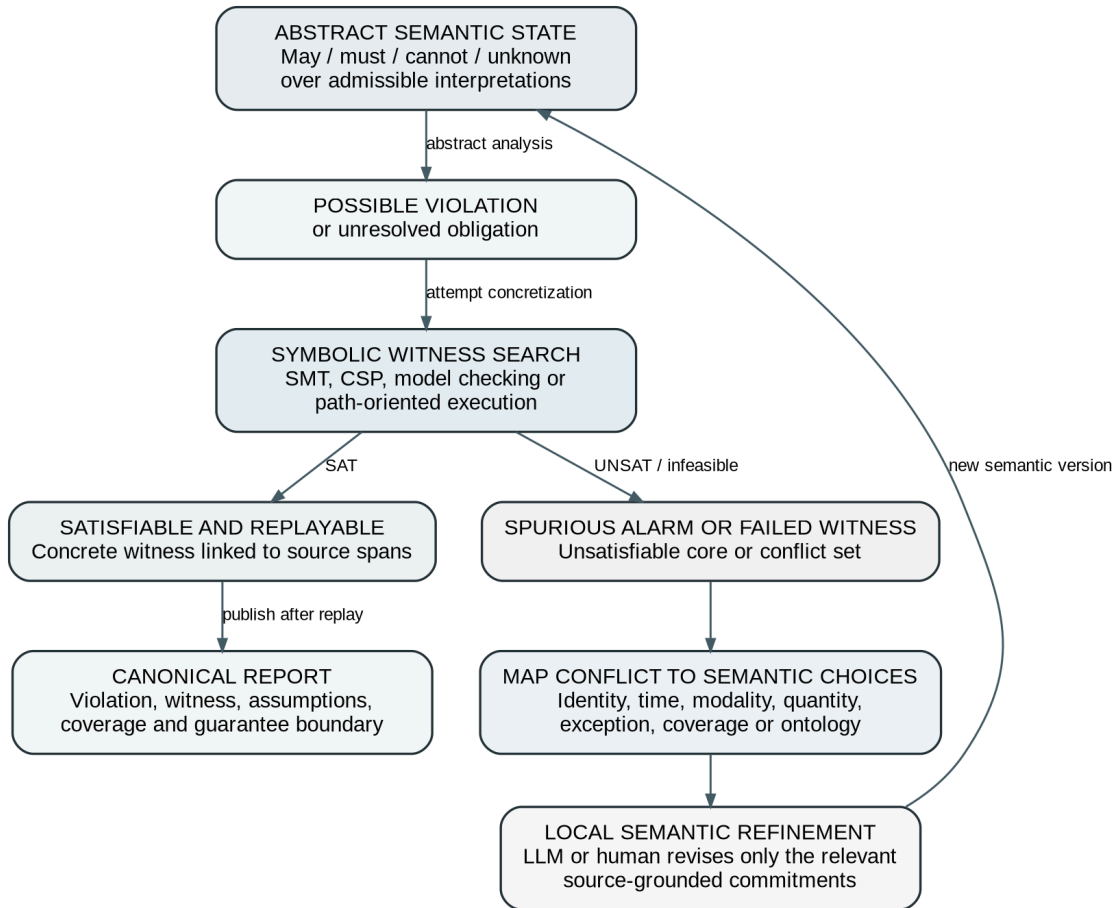


Figure 2. Affinement sémantique guidé par contre-exemples. L'analyse abstraite identifie une possible violation; la recherche symbolique de témoins confirme un cas rejouable ou expose les distinctions sémantiques qui doivent être affinées.

Cela conduit à un raffinement sémantique à contre-exemples. Le CEGAR traditionnel commence par une abstraction grossière du programme. La vérification du modèle trouve un contre-exemple abstrait. Une vérification de faisabilité détermine si le contre-exemple correspond à une exécution concrète. Si ce n'est pas le cas, des prédicats sont ajoutés pour

affiner l'abstraction, et l'analyse se répète. Dans le langage, l'abstraction comprend des choix sémantiques. Un contre-exemple fallacieux peut survenir parce que deux mentions ont été fusionnées, une exception a été ignorée, un intervalle temporel a été élargi ou une déclaration modale a été traitée comme un fait.

Une boucle de raffinement sémantique devrait préserver l'explication symbolique de l'échec. Supposons qu'un avertissement de conformité dépende de l'hypothèse que «le conseil» en deux sections désigne le même corps. Le replay symbolique montre que le chemin devient infaisable si les entités sont distinctes. Le système cartographie l'identité en fonction des mentions pertinentes et pose une question ciblée: ces références visent-elles à désigner le même conseil? Un LLM peut réexaminer le contexte local et proposer des alternatives; un humain peut juger si nécessaire. Le reste du modèle de document reste inchangé. C'est plus fiable et efficace que de demander à un modèle de tout relire et d'«essayer à nouveau».

Les noyaux insatisfaisables fournissent un mécanisme connexe. Lorsqu'un ensemble de contraintes est incohérent, un solveur peut souvent retourner un sous-ensemble responsable du conflit. La cartographie du noyau vers les portées sources identifie l'ensemble minimal de déclarations et d'interprétations qui ne peuvent pas coexister. Dans un examen de la politique, cela peut révéler des obligations contradictoires. Dans l'analyse sémantique, il peut révéler qu'un quantificateur, un lien d'identité et une hypothèse temporelle créent conjointement l'impossibilité. L'utilisateur reçoit une explication fondée sur le conflit formel plutôt que sur une critique générique de modèle.

Tous les problèmes ne sont pas mieux représentés comme l'exécution du chemin. La vérification du modèle analyse les systèmes de transition par rapport aux propriétés temporelles. La logique temporelle linéaire peut exprimer que quelque chose finit par se produire, tient toujours, ou se produit avant un autre événement. Les logiques de calcul et probabilistes expriment des changements de ralliement ou de comportement incertain. Les systèmes de langue naturelle à logique temporelle tels que NL2TL et nl2spec montrent que LLMs peut construire des spécifications temporelles lorsque le vocabulaire et le domaine sont contrôlés [Chen et al. 2023; Cosler et al. 2023]. L'exécuteur est alors un vérificateur ou un planificateur de modèle, pas un LLM.

La résolution de contraintes couvre un large juste milieu. SAT représente les choix booléens. SMT ajoute des théories telles que l'arithmétique, les tableaux, les chaînes et les fonctions non interprétées. CSP et CP-SAT gèrent les domaines finis, la planification et l'optimisation. MaxSAT et les contraintes souples expriment les préférences. De nombreuses tâches en langage naturel – planification, allocation des ressources, vérification de la cohérence, relations familiales, énigmes et combinaisons de politiques – sont mieux compilées en contraintes qu'en code de procédure. SatLM a montré que les spécifications déclaratives plus un solveur peuvent surpasser les approches assistées par programme sur les tâches arithmétiques et logiques [Ye et al. 2023]. Le résultat prend en charge un choix architectural central: le LLM devrait décrire le problème, tandis que le solveur effectue la recherche.

La planification formelle fournit une sémantique d'action et une recherche axée sur les objectifs. Les travaux récents positionnent LLMs comme des formalisateurs de planification plutôt que comme des planificateurs. Hao et ses collègues traduisent les contraintes de planification dans le monde réel en outils de vérification formels, utilisent des cœurs insatisfaisants pour la réparation et déclarent 93,9% de succès sur TravelPlannerrr dans leur cadre, par rapport à des niveaux de référence de modèle direct beaucoup plus faibles [Hao et

al. 2025]. Une étude de synthèse de 2025 conclut que la principale opportunité n'est pas de remplacer les planificateurs, mais d'utiliser LLMs pour construire et affiner des modèles de planification [Tantakoun et al. 2025]. C'est précisément la division préconisée ici.

Les moteurs Dataflow et Datalog fournissent l'exécution de points fixes plutôt que la recherche de chemin. Ils sont utiles pour les conséquences monotones et la propagation à l'échelle du document. Les automates et les moteurs à motifs gèrent les séquences d'événements ordonnées. Les tables de décision traitent des politiques limitées. L'architecture devrait acheminer chaque problème vers le formalisme le plus simple qui l'exprime. L'exécution symbolique ne doit pas devenir une étiquette universelle pour chaque composant symbolique.

LLMs peut aider l'exécution symbolique traditionnelle de manière plus sûre et plus risquée. Un rôle plus sûr est la priorisation de la voie. Un modèle peut identifier les régions de code susceptibles de contenir des vulnérabilités ou des chemins susceptibles d'atteindre une cible, tandis que KLEE ou un autre moteur conserve la décision de faisabilité finale. Des travaux récents tels que KLEECopilot font état d'améliorations substantielles de la couverture de la recherche guidée par LLM, bien que l'état de préimpression et la sensibilité du modèle nécessitent une interprétation prudente [Chen et al. 2026]. Le principe est sain: les heuristiques apprises décident où chercher, pas si un chemin est valide.

Un rôle plus risqué est de remplacer le solveur. AutoBug, présenté par ses auteurs sous le nom d'exécution symbolique à grand modèle de langage, décompose le code en tâches de raisonnement basées sur le chemin et demande à un modèle de raisonner directement à leur sujet sans tout traduire en SMT. L'approche est légère et agnostique du langage, et les expériences rapportées améliorent l'analyse de programme basée sur LLM [Li et al. 2025]. Les auteurs le caractérisent comme approximatif. Il peut analyser des constructions difficiles pour les solveurs conventionnels, mais il ne fournit pas de solidité classique. Pour une utilisation à haute assurance, un tel modèle est mieux traité comme un moteur dans un portefeuille dont les résultats nécessitent des tests, une relecture ou une confirmation indépendante.

LLMs peut également générer des tests, des résumés ou des modèles satisfaisants pour les API externes. Les études font état d'une couverture utile sur les programmes que les outils traditionnels ont du mal à gérer, mais les performances diminuent avec les longs chemins, les API complexes, la récursion et la classification des chemins infaisables. Ces méthodes sont prometteuses car les environnements logiciels contiennent une sémantique coûteuse à axiomatiser. Leur statut de résultat devrait rester «candidat» jusqu'à ce que l'exécution concrète confirme le chemin ou qu'un solveur prouve la condition.

La sécurité est une préoccupation majeure. Les règles et les documents sources en langue naturelle sont des contributions non fiables. Un document malveillant peut tenter d'injecter rapidement, fabriquer des références ou exploiter un traducteur générateur de code. Les artefacts formels doivent être des données, et non du code exécutable sans restriction. Lorsque des algorithmes personnalisés sont requis, ils doivent être examinés les extensions d'hôte avec des digests verrouillés, des entrées et des sorties tapées, des limites de ressources et aucune autorité ambiante. Les invocations de solveurs ont besoin de temps et de budgets de mémoire. Un témoin devrait inclure la version exacte de la théorie, du compilateur et du moteur qui l'a produit.

Le schéma d'analyse combiné est maintenant clair. Une analyse sémantique et de flux de données grossière identifie les problèmes possibles dans un long document. Les moteurs formels spécialisés ne testent que les tranches pertinentes. L'exécution symbolique construit

des témoins pour des revendications dépendantes de la voie. Le contrôle de modèle gère les comportements temporels. Les solveurs de contraintes gèrent les choix combinatoires. Les témoins ratés et les noyaux de solveurs conduisent le raffinement sémantique local. Chaque conclusion acceptée est retournée avec un statut et une provenance qui reflètent le maillon le plus faible de la chaîne.

Cette architecture ne promet pas la vérification totale du langage sans restriction. Il promet quelque chose de plus opérationnel: chaque revendication formelle a un sens exécutable; chaque symbole a une source ou une hypothèse déclarée; chaque contre-exemple peut être rejoué; chaque revendication d'absence est liée à la couverture; et chaque ambiguïté restante est représentée plutôt que cachée. L'interprétation abstraite fournit l'échelle et l'agrégation conservatrice. L'exécution symbolique fournit la précision et les témoins. Leur combinaison est l'un des fondements les plus plausibles du langage naturel exécutable.

4. Ce que l'ère LLM a réellement atteint

4.1 Autoformalisation, raisonnement assisté par programme, logique, spécifications temporelles et planification

L'arrivée de grands modèles de langage a changé la limite pratique de la compilation sémantique. Les systèmes antérieurs nécessitaient des grammaires spécifiques à la tâche, des formes logiques supervisées ou des lexiques soigneusement conçus. LLMs peut proposer des artefacts formels à partir de descriptions ouvertes avec peu ou pas de formation spécifique à la tâche. Cela élargit considérablement la couverture, mais cela crée également une tentation de déduire plus de gains de référence que ne le justifient les mécanismes. La question pertinente n'est pas simplement de savoir si une méthode améliore la précision de la réponse. C'est la mesure où les erreurs ont été transférées à une composante symbolique, quelles erreurs restent dans la formalisation, et quelles preuves indépendantes existent que l'artefact formel représente la source.

Les modèles de langage aidés par programme, ou PAL, fournissent un point de départ propre. Le LLM lit un problème, génère un programme Python et délègue le calcul à l'interpréteur [Gao et al. 2022]. Sur GSM8K, le système signalé s'est sensiblement amélioré par rapport à l'invite de la chaîne de pensée. Le mécanisme est convaincant parce que les modèles de langage sont souvent meilleurs pour décomposer un problème que pour réaliser une longue arithmétique exactement. Une fois la décomposition correcte, Python fournit des opérations et un état déterministes.

La limitation de PAL est tout aussi instructive. L'interprète certifie seulement qu'il a exécuté le programme généré. Il ne certifie pas que le programme modélise fidèlement la question en langue naturelle. Une variable erronée, une condition omise ou une mauvaise formule peut s'exécuter sans erreur. Le succès du système dépend donc d'une distribution où les programmes sont courts, les exemples contraignent fortement la structure et les erreurs d'exécution révèlent de nombreuses erreurs. Pour le raisonnement documentaire, l'analyse des politiques et l'interprétation scientifique, la cartographie est moins limitée et l'interprète fournit des commentaires plus faibles.

SatLM remplace le code de procédure par des contraintes déclaratives [Ye et al. 2023]. Le modèle indique des variables et des relations, et un moteur de satisfaction recherche une solution. Cela peut être plus facile que de générer un algorithme car le modèle n'a pas besoin de décider de l'ordre des opérations. Les représentations déclaratives permettent également plusieurs modèles, des preuves d'infaisabilité et un débogage au niveau de la contrainte. Les gains déclarés sur les tâches de raisonnement difficile soutiennent l'idée que le choix du solveur devrait suivre la structure du problème. Un calendrier est naturellement un problème de contrainte; un processus dynamique peut être une machine d'état; un problème de mot arithmétique peut être un programme ou des contraintes.

Logic-LM formalise les problèmes logiques en langage naturel et invoque des solveurs symboliques [Pan et al. 2023]. Il utilise également des erreurs de solveur pour accélérer l'auto-réaffinement. Dans plusieurs ensembles de données, les auteurs font état d'importantes améliorations moyennes par rapport à l'incitation standard et à la chaîne de pensée. L'architecture démontre deux principes utiles. Tout d'abord, le moteur symbolique élimine

l'incohérence déductive une fois qu'une formalisation valide existe. Deuxièmement, les messages d'erreur formels fournissent un meilleur signal de réparation qu'une demande générique de reconsidération.

Cependant, la réparation de syntaxe n'est pas une réparation sémantique. Une formule peut être bien formée, satisfaisante et erronée. Le pipeline de Logic-LM peut corriger les prédicats malformés ou les défaillances du solveur, mais il peut ne pas détecter que «certains» sont devenus «tous» ou qu'une exception a été omise. Des travaux ultérieurs tels que Logic-LM++ ajoutent un raffinement en plusieurs étapes, et LTRAG récupère des exemples de formalisation guidés par la pensée, en rapportant des améliorations par rapport à Logic-LM et LINC sur FOLIO et AR-LSAT [Kirtania et al. 2024; Hu et al. 2025]. Ces gains suggèrent que la formalisation elle-même bénéficie d'exemples et de critiques structurées. Ils n'éliminent pas la nécessité d'évaluer directement la fidélité sémantique.

LINC rend la modularité explicite: un LLM sert de parser sémantique, la logique du premier ordre représente le problème, et un teur de théorème effectue la déduction [Olausson et al. 2023]. Sur ProofWriter, la combinaison déclarée a produit de grands gains par rapport aux systèmes de chaîne de pensée, y compris les modèles sensiblement plus grands que l'analyseur. L'analyse des erreurs a montré des modes de défaillance complémentaires: le système symbolique évite de nombreuses erreurs de raisonnement, tandis que les défaillances de formalisation restent. C'est exactement la preuve attendue d'une architecture fédérée. Le système est le plus fort lorsque la tâche en langage naturel correspond déjà à une théorie compacte du premier ordre.

FOLIO fournit un indice de référence utile car il contient des problèmes de raisonnement en langage naturel créés par l'homme associés à la logique de premier ordre vérifiée [Han et al. 2024]. Il évalue à la fois le raisonnement et la traduction NL-to-FOL. L'ensemble de données reste difficile même pour les modèles forts, ce qui réduit l'idée que la formalisation du premier ordre est une étape de prétraitement résolue. FOLIO illustre également l'importance de la complexité d'auteur humain. Les ensembles de données de règles synthétiques peuvent rendre le langage inhabituellement régulier et permettre aux modèles d'exploiter des modèles. Les exemples humains introduisent la variation lexicale, la connaissance du monde et des choix de portée plus réalistes.

FoVer adopte une autre approche en vérifiant le raisonnement en langage naturel par le biais de la logique de premier ordre [Pei et al. 2025]. Au lieu de traiter le texte de raisonnement couramment comme auto-authentifant, il construit des obligations formelles et les vérifie. Il s'agit d'un modèle prometteur pour la génération à contrainte de preuve: un LLM peut proposer un argument, mais un vérificateur n'accepte que les étapes inférentielles qui correspondent à des conséquences formelles valides. Le problème restant est l'alignement entre chaque phrase et les prédicats formels utilisés pour la vérifier.

La recherche sur l'autoformalisation traite de plus en plus la traduction comme une discipline distincte plutôt que comme une astuce rapide. Les enquêtes publiées en 2025 dans la traduction du catalogue dans des proverbes de théorème, des langages logiques et des spécifications formelles, mettant l'accent sur la nécessité de boucles de rétroaction, de récupération, de représentations intermédiaires et de repères qui séparent la syntaxe de la sémantique [Weng et al. 2025]. Le travail direct de NL-to-FOL fait état d'un progrès significatif, mais aussi d'erreurs systématiques dans les connaissances implicites, la portée du quantificateur et la conception de l'ontologie [Yang et al. 2024; Lalwani et al. 2025]. L'ajout de

connaissances de base implicites peut améliorer la performance des tâches tout en augmentant le risque que le système formel prouve une prémisse non autorisée par la source.

Le traitement des connaissances de fond est un choix architectural décisif. Dans un benchmark de puzzle, on peut s'attendre implicitement à des locaux de bon sens. Lors de la vérification juridique ou scientifique, l'introduction de locaux non déclarés peut invalider le résultat. Un système rigoureux devrait distinguer les locaux sources, les locaux de l'autorité, les axiomes de domaine et les hypothèses suggérées par le modèle. Le solveur peut raisonner sur chacun d'eux, mais l'explication finale doit montrer quelles conclusions dépendent de quelle autorité. Si une hypothèse est essentielle et non vérifiée, le résultat devrait être conditionnel.

La traduction temporelle-logique apporte différents défis. NL2TL utilise des représentations levées pour apprendre la structure temporelle entre les domaines et rapporte une forte adaptation avec relativement peu de données de domaine cible [Chen et al. 2023]. La stratégie de levage cache des propositions atomiques concrètes tout en apprenant des schémas logiques réutilisables. Il s'agit d'un aperçu de conception précieux: séparez la structure de «toujours avant», «éventuellement après» ou «jusqu'à» de la base spécifique au domaine des événements.

nl2spec met l'accent sur l'interaction [Cosler et al. 2023]. Le système cartographie des parties de l'exigence de langage naturel à des parties de la formule temporelle générée, permettant aux utilisateurs de réparer les erreurs de traduction locales. Cette interface reconnaît que l'ambiguïté ne peut pas toujours être résolue automatiquement. Il suggère également comment le raffinement sémantique basé sur LLM devrait fonctionner: l'utilisateur devrait voir la décision linguistique, pas seulement une chaîne d'opérateurs temporels.

L'échouement reste le point fragile. Une formule temporelle peut avoir la forme correcte tandis que ses propositions atomiques font référence aux mauvais événements ou objets. Le travail sur l'équivalence logique fondée rapporte que les mécanismes de mise à la terre explicites améliorent la rectitude, renforçant la distinction entre la syntaxe de formule et l'attachement sémantique. Pour un temps d'exécution en langage exécutable, les atomes temporels doivent être des références à des observations typées avec des travées et des identités, et non des étiquettes de forme libre inventées par le traducteur.

La planification automatisée fournit un backend symbolique particulièrement mature. Les langages de planification définissent les types, les prédicats, les actions, les conditions préalables, les effets, les états initiaux et les objectifs. Les planificateurs recherchent ensuite une séquence d'action valide. LLMs sont de mauvais planificateurs à long horizon lorsqu'ils doivent maintenir l'état et les contraintes en interne, mais ils peuvent être efficaces pour formaliser un problème pour un planificateur. Une étude de synthèse de 2025 identifie la construction et le raffinement de modèles comme le rôle central pour LLMs dans cette zone [Tantakoun et al. 2025].

Hao et ses collègues démontrent une version rigoureuse du modèle sur la planification réelle [Hao et al. 2025]. Le système formalise les contraintes, invoque des outils de vérification et utilise des cœurs insatisfaisables pour réparer les problèmes. Les auteurs font état de 93,9% de succès sur TravelPlannerr dans leur cadre expérimental, bien au-dessus des lignes de base directes LLM qu'ils comparent. Le résultat est important car il montre que les outils formels peuvent transformer la fiabilité de la planification lorsque le problème est représentable. Cela ne signifie pas que le LLM est devenu un planificateur de son. Cela signifie que le planificateur et

le vérificateur portent l'état à long horizon tandis que le LLM construit et répare l'instance formelle.

La planification expose également des contraintes cachées. Les demandes en langage naturel comprennent des attentes de bon sens, des préférences et des conditions de faisabilité qui peuvent ne pas être énoncées comme des exigences formelles. Des systèmes tels que LRPlan tentent la collaboration entre les modèles de langage et de raisonnement pour des contraintes explicites et implicites, tandis que des repères tels que PlanningArena montrent que même les modèles forts luttent avec la sélection des outils, la mémoire de contexte et la cohérence logique [Karthikeyan et al. 2025; Zheng et al. 2025]. Un système de haute assurance doit soit formaliser les contraintes implicites à partir d'une source de confiance, soit les étiqueter comme des hypothèses. «Commonsense» n'est pas une catégorie de provenance suffisante.

RuleArena et les repères connexes testent LLMs par rapport à des systèmes de règles réalistes et trouvent que l'augmentation de l'outil aide mais ne rend pas la formalisation triviale. Les tâches de SemEval 2026 sur le contenu et le raisonnement formel montrent en outre que les modèles sont influencés par le contenu sémantique, même lorsque la forme logique devrait déterminer la réponse. Les entrées neuro-symboliques modulaires utilisent FOL et Z3 pour réduire le biais de contenu, mais la traduction formelle et le bruit multilingue restent difficiles [Grimaldi 2026; Kartáč et al. 2026; Wang et al. 2026]. Ces résultats renforcent la nécessité de démêler l'interprétation linguistique de l'inférence formelle.

L'une des observations les plus conséquentes dans cette littérature est que les systèmes réussis utilisent des langages intermédiaires spécifiques à la cible. PAL utilise Python. SatLM utilise des contraintes déclaratives. Logic-LM et LINC utilisent la logique. NL2TL utilise la logique temporelle. Les systèmes de planification utilisent des modèles de type PDDL. SymGPT utilise une grammaire adaptée aux règles ERC. Le domaine n'a pas découvert une représentation formelle universelle pour le langage naturel. Il a découvert que LLMs peut acheminer le langage vers plusieurs écosystèmes formels lorsque des exemples, des schémas et des commentaires sont disponibles.

Cela prend en charge une architecture de portefeuille plutôt qu'un «bytecode de langage» universel. Un pivot sémantique peut préserver les preuves, les entités, les événements, la portée et la modalité, tandis qu'un sélecteur de formalisme choisit la cible d'exécution. Le sélecteur lui-même peut être assisté par un LLM, mais sa décision doit être validée par rapport aux contrats de capacité. Un formalisme n'est compatible que s'il peut exprimer les distinctions requises et que les observations fondées nécessaires existent. Un planificateur ne devrait pas être sélectionné pour une requête purement taxonomique; la logique de premier ordre ne devrait pas approcher silencieusement la dynamique temporelle incertaine; un moteur relationnel simple ne devrait pas prétendre décider d'exceptions juridiques défaisables.

Les données empiriques soutiennent donc une revendication plus étroite mais substantielle. LLMs peut améliorer considérablement la construction d'objets formels exécutables. Les systèmes symboliques externes peuvent améliorer la correction, la cohérence et l'interprétabilité. La rétroaction du solveur peut guider la réparation. Les résultats les plus solides apparaissent dans des domaines bornés avec des langages explicites, des schémas clairs et des réponses évaluable. Le défi non résolu est la compilation sémantique: s'assurer que l'objet formel préserve la signification pertinente de la source et que chaque prémisse est fondée et autorisée.

4.2 Analyse de programmes assistée par LLM, exploration symbolique, transformateurs abstraits et NLP certifié

L'interaction entre LLMs et l'analyse de programme s'exécute dans les deux sens. LLMs peut aider les outils formels à gérer les spécifications, les modèles de code et les environnements difficiles à modéliser. Les outils formels peuvent vérifier le code généré par LLMs, vérifier les réseaux de neurones et limiter les composants appris. Cette littérature est particulièrement précieuse pour le langage naturel exécutable car elle teste la même question architecturale sous une sémantique plus claire: où peut contribuer un modèle probabiliste sans devenir l'oracle final?

L'analyse statique traditionnelle dépend d'abstractions conçues à la main, de fonctions de transfert, de résumés et de modèles d'environnement. L'exécution symbolique dépend de la recherche de chemin et des encodages de solveur. Ces outils offrent des garanties de principe, mais sont aux prises avec des fonctionnalités linguistiques dynamiques, le code de la bibliothèque, la manipulation de chaînes complexes, les services externes et l'évolutivité. LLMs peut reconnaître l'intention du code, proposer des invariants, résumer des fonctions, identifier les emplacements de bugs probables et générer des tests. Une étude de synthèse de 2025 sur l'analyse du programme assistée par LLM classe le travail en méthodes statiques, dynamiques et hybrides et met l'accent à la fois sur l'opportunité et les risques des résultats probabilistes [Wang et al. 2025].

Un rôle architectural est l'orientation heuristique. Un exécuteur testamentaire symbolique peut conserver sa sémantique et utiliser un LLM pour classer les chemins, identifier les régions vulnérables ou sélectionner les limites de boucle. Si l'heuristique est erronée, la couverture peut en souffrir, mais les chemins acceptés restent contrôlés par le solveur. Ceci est analogue à l'utilisation d'un modèle neuronal pour la commande de recherche dans le théorème prouvant. La distinction entre orientation et acceptation est essentielle. Un score appris peut déterminer où les ressources sont dépensées; il ne devrait pas déterminer si une condition de chemin est satisfaisante.

KLEECopilot représente cette direction. La pré-impression 2026 utilise les directives LLM pour diriger KLEE vers les vulnérabilités probables et introduit des heuristiques pour l'évasion en boucle, en signalant des améliorations substantielles dans la couverture de base-bloc et de ligne et des dizaines de violations uniques [Chen et al. 2026]. En tant que pré-impression récente, les résultats nécessitent une réplication indépendante et une sensibilité aux questions familiales modèles. Néanmoins, le modèle de conception est plausible: LLMs contribue à la priorisation sémantique que la recherche classique manque, tandis que KLEE conserve la sémantique d'exécution.

Un autre rôle est la génération de tests satisfaisants. Un modèle peut lire le code et proposer des entrées qui devraient traverser un chemin cible. L'exécution concrète valide alors si le chemin a été atteint. Cette approche peut gérer les conventions API ou les formats de données qui sont lourds pour SMT. Des études font état de taux de réussite utiles, mais aussi d'échecs sur de longs chemins, de récursion, de branches infaisables et de bibliothèques complexes. L'interprétation sûre est que le LLM est un générateur de test. Une course de béton, une trace d'instrumentation ou un solveur reste le validateur.

AutoBug adopte une approche plus radicale en utilisant LLMs pour effectuer un raisonnement décomposé par chemin directement [Li et al. 2025]. Au lieu de traduire toutes les

contraintes en une théorie du solveur, il présente des conditions de chemin au niveau du code au modèle. Cela peut être agnostique du langage et peut analyser la sémantique difficile à encoder. Les auteurs font état d'améliorations de plusieurs repères d'analyse de programmes, en particulier pour les modèles plus petits. Ils caractérisent également l'analyse comme approximative. Le système n'offre pas de solidité classique, et ses conclusions peuvent varier avec le modèle.

AutoBug est précieux car il montre où les méthodes symboliques échouent sur le plan opérationnel. De nombreux programmes réels appellent des bibliothèques opaques, manipulent des objets complexes ou dépendent de contrats informels. Un modèle de langage peut déduire le comportement probable à partir des noms, des commentaires et des modèles de code appris. Mais cette flexibilité est la même source d'incertitude. Dans un portefeuille orienté vers l'assurance, l'analyse approximative LLM devrait générer des candidats, des résumés ou des vérifications priorisées. Les résultats doivent être confirmés par des essais, des contrats, une surveillance de l'autonomie ou un moteur formel chaque fois qu'une demande de sécurité est faite.

LLMs peut proposer des invariants pour l'interprétation abstraite. Un candidat invariant peut accélérer la preuve si un vérificateur vérifie qu'il détient initialement, est préservé par des transitions, et implique la propriété. C'est une division puissante du travail: la découverte invariante est créative et combinatoire, tandis que la vérification invariante est mécanique. Des modèles similaires apparaissent dans la preuve du théorème et la synthèse du programme. Le modèle de langage recherche; le vérificateur accepte.

SAIL étend ce principe à la synthèse de transformateurs abstraits [Gu et al. 2026]. Il est difficile de construire un transformateur à la fois sonore et précis pour un opérateur non linéaire. SAIL demande à LLMs de générer des candidats mais impose des contraintes syntaxiques et sémantiques et utilise un coût mathématiquement fondé sur la malsaineté. Le système rapporté synthétise des transformateurs sonores à l'échelle mondiale dans plusieurs domaines, correspond à des conceptions manuelles et trouve des transformateurs précis qui ne sont pas auparavant dans la littérature. L'importance va au-delà de la vérification du réseau neuronal. C'est la preuve que LLMs peut automatiser la construction de méthodes formelles lorsque chaque artefact accepté est soumis à des obligations de preuve indépendantes.

Le contraste avec le fait de demander à un LLM d'«agir comme un interprète abstrait» est vif. Mitchell et ses collègues ont incité des modèles à raisonner par l'interprétation abstraite sur les programmes SV-COMP [Mitchell et al. 2025]. Les modèles pourraient suivre des styles de composition ou de point de correction sur certains cas, mais ont commis des erreurs logiques et halluciné sur des structures complexes. Le résultat est cohérent avec un modèle général: les modèles peuvent imiter les processus formels et générer des candidats utiles, mais la fiabilité diminue lorsque le processus nécessite un état exact persistant. Un véritable interprète abstrait devrait exécuter les opérations de réseau; le modèle peut suggérer des domaines, des prédicats ou des explications.

La vérification formelle des modèles Neural NLP est une autre branche majeure. Le travail de robustesse certifié demande si la prédiction d'un modèle reste inchangée sur toutes les entrées d'un ensemble de perturbations définie. Jia et ses collègues ont utilisé la propagation à bornes d'intervalle pour les substitutions de mots et ont signalé une précision contradictoire de 75% sur IMDB et SNLI sous les familles de substitution construites [Jia et al. 2019]. SAFER utilisé un lissage randomisé et pourrait s'appliquer aux modèles à boîte noire, y compris BERT

[Ye et al. 2020]. Les intégrations conscientes de la robustesse ont renforcé les limites de certification, améliorant la précision certifiée robuste sur IMDB de 76,73 à 84,78 pour cent dans le cadre déclaré de [Wang et al. 2023].

La vérification du transformateur est difficile parce que l'auto-attention contient des produits, des opérations softmax et des corrélations entre les positions. Les domaines spécialisés améliorent les limites. L'interprétation abstraite paramétrée pour la vérification du transformateur a introduit des domaines affinés paramétrés pour les produits intérieurs et a signalé des limites plus serrées et la vérification des instances que les méthodes antérieures ne pouvaient pas prouver [Huang et al. 2026]. Cette ligne de travail démontre que l'analyse formelle peut passer à des architectures non triviales NLP lorsque la propriété et l'ensemble d'entrée sont définis avec précision.

La portée de la garantie ne doit pas être floue. Un classificateur certifié peut être robuste à chaque substitution tirée d'un ensemble spécifié et toujours être faux. Une liste de substitution peut inclure des «synonymes» inappropriés dans le contexte ou omettre des paraphrases significatives. La certification dit que le modèle est invariant sur la famille de perturbations formelles. Elle n'établit pas l'équivalence sémantique dans le langage humain. Cette distinction devrait être explicite dans tout système de langage exécutable utilisant des modèles d'extraction certifiés.

Une intégration utile attacherait un certificat de robustesse à une observation sémantique. Supposons qu'un extracteur d'événements identifie qu'une phrase énonce une obligation. Un modèle certifié pourrait montrer que cette classification est stable sous une famille contrôlée de substitutions lexicales. L'observation est encore modèle proposée, mais elle a une propriété de stabilité supplémentaire. L'analyse symbolique en aval peut enregistrer cette preuve sans mettre à niveau l'observation en fait. Un tel statut en couches est plus instructif qu'un score de confiance.

Les méthodes formelles vérifient également le code généré par LLMs. La génération de tests, l'analyse statique, la vérification de type et les assistants de preuve peuvent détecter de nombreuses erreurs. Les systèmes de synthèse de programme utilisent de plus en plus le compilateur et la rétroaction de test pour la réparation itérative. Le même modèle devrait régir les circuits sémantiques générés et le code de formalisation. Un agent de codage peut créer une implémentation de règle, mais la publication devrait nécessiter une compilation, des tests de mutation, des cas de référence en langage naturel, des replays et des vérificateurs indépendants. Un LLM ne doit pas être autorisé à créer un comportement exécutable caché à l'intérieur d'un artefact théorique déclaratif.

La littérature sur la modélisation protocolaire du langage naturel offre un résultat dégrisant. Les travaux présentés à ICSE 2025 ont étudié la génération de modèles Tamarin ou ProVerif à partir de la documentation du protocole. L'utilisation naïve de LLM était inefficace; le pipeline réussi nécessitait une décomposition sémantique, une homonymisation humaine limitée et des transformations intermédiaires solides. Il a généré des modèles à échelle modérée corrects pour 10 de 18 protocoles réels [Mao et al. 2025]. Le résultat est significatif parce que les protocoles contiennent des rôles, des messages, des hypothèses cryptographiques, de l'état et de l'ordre temporel. Il montre également que même une documentation structurée forte ne se compile pas automatiquement dans des modèles de sécurité formels.

Les cas d'échec sont instructifs. La documentation omet les hypothèses que les experts en déduisent. Le même terme peut désigner un type de message, un état ou une instance. Les

propriétés de sécurité peuvent être indiquées de manière informelle ou réparties entre les sections. Un modèle de protocole formel doit rendre explicites les capacités adverses. Ce sont des analogues directs du raisonnement documentaire. La solution n'était pas une invite plus grande, mais une architecture qui sépare l'analyse sémantique, la désambiguïsation et la transformation vérifiée.

Les examens externes du domaine plus large convergent vers une conclusion similaire. L'examen neuro-symbolique 2026 NLP favorise une étude plus forte des architectures fédératives, mais avertit que les comparaisons actuelles ne sont pas contrôlées et qu'aucun système fédératif à l'échelle fondamentale n'a été établi [Chatzikyriakidis and Lappin 2026]. L'enquête de planification-formalisateur traite LLMs comme des constructeurs et des raffineurs de modèles formels, et non des remplacements pour les planificateurs [Tantakoun et al. 2025]. L'enquête d'analyse de programme met l'accent sur les systèmes hybrides et la nécessité de gérer l'hallucination et la reproductibilité [Wang et al. 2025]. Ce sont des lignes de preuve indépendantes pour la même division du travail.

Le champ a donc réalisé plus qu'un motif superficiel "LLM plus solveur". Il a démontré la réparation guidée par le solveur, les formalisations vérifiées par preuve, la planification formelle, le guidage de chemin, la proposition d'invariant, la synthèse du transformateur et la robustesse certifiée. Il a également exposé une limite stable. Lorsque le LLM contrôle l'acceptation finale, les garanties deviennent approximatives. Lorsque le LLM propose des artefacts qu'un système formel séparé vérifie, des garanties solides peuvent être conservées pour la propriété vérifiée. La question sans réponse est de savoir comment rendre la limite de langage naturel à artefacte comparablement disciplinée.

L'architecture proposée plus loin dans ce livre applique directement les leçons d'analyse de programme. Les transformations sémantiques candidates sont traitées comme du code synthétisé. Ils ont des contrats typés, des cartes sources, des limites de ressources et des tests. Les domaines abstraits sont enregistrés plutôt que improvisés pendant une course. Les prédicats de raffinement sont suggérés par LLMs mais validés par des distinctions concrètes. Les portefeuilles de solveurs exposent des objets ou des témoins de preuve. Le fabricant de langues consomme des résultats canoniques, pas des traces brutes. Dans cette conception, la flexibilité du LLM est un atout précisément parce qu'il est entouré de portes d'acceptation symboliques.

5. Pourquoi les systèmes de langage exécutable échouent

5.1 Ancrage, identité, formalisations fallacieuses, ontologies et connaissance du monde

L'erreur dominante dans les systèmes de langage exécutable n'est pas une déduction incorrecte des locaux acceptés. C'est l'admission des mauvais locaux, objets ou relations. La mise à la terre est le processus par lequel les symboles acquièrent une référence opérationnelle: un prédicat est relié à une phrase, une entité à une ou plusieurs mentions, un événement à la preuve, une expression temporelle à un intervalle, un champ de base de données à un concept, ou une action à une capacité d'environnement. Un programme formel sans mise à la terre fiable est exact dans le sens le moins utile. Il s'exécute de manière cohérente tout en décrivant le mauvais monde.

La mise à la terre commence par l'emplacement de la source. Chaque observation sémantique doit être liée à des preuves sources exactes ou explicitement marquée comme une hypothèse externe. Un devis doit être revalidé par rapport à la révision de la source immuable. Un résumé du document ne peut pas se substituer à la source parce qu'il peut omettre les qualifications. Les résultats de la récupération ne peuvent pas soutenir une réclamation concernant l'absence à moins que le processus de récupération ne soit terminé pour le domaine concerné. Cette discipline source empêche une défaillance commune dans les systèmes LLM: un modèle produit un texte de support plausible qui ne se produit pas dans le document, et la sortie structurée en aval donne à l'invention une apparence d'autorité.

La mise à la terre lexicale relie le langage à l'ontologie. L'expression «approbation» peut désigner une action, un état, un artefact signé, une permission ou une décision institutionnelle. Un schéma de domaine doit choisir et documenter l'interprétation nécessaire par une règle. LLMs sont utiles car ils peuvent déduire le contexte et proposer des cartographies, mais une cartographie doit être testée par rapport à des exemples et des alternatives. Une correspondance de chaîne entre une phrase et un nom de type n'est pas une compatibilité sémantique. La documentation nllAgent met explicitement en garde contre la cartographie de deux concepts simplement parce que leurs étiquettes correspondent; il nécessite un adaptateur ou une sonde sémantique lorsque les significations diffèrent.

L'ancrage de l'identité est plus difficile et plus conséquent. L'analyse de documents doit souvent savoir si deux mentions désignent la même personne, organisation, objet, ensemble de données, exigence ou événement. Les noms égaux n'impliquent pas l'identité, et les noms différents n'impliquent pas la différence. Les titres changent, des alias se produisent, les pronoms sont ambigus, les organisations ont des subdivisions, et les mentions fictives ou citées peuvent ne pas appartenir au monde source. Une couche d'identité sûre sépare les mentions des hypothèses de l'entité et des candidats à l'identité. Il enregistre le producteur, les preuves, la portée, le monde et les alternatives pour chaque lien.

La fusion d'identité prématurée crée des erreurs en cascade. Dans un récit, deux personnages portant le même nom de famille peuvent être fusionnés, produisant des violations de continuité impossibles. Dans un document réglementaire, «l'Autorité» peut se référer à différents organismes dans différentes juridictions. Dans la documentation logicielle, «client»

peut désigner un utilisateur, un composant d'application ou un point de terminaison HTTP. Une fois fusionnée, les règles ultérieures traitent toutes les propriétés comme appartenant à un objet. L'erreur devient difficile à diagnostiquer car le solveur voit un identifiant cohérent. Un système conservateur devrait permettre une identité non résolue et retourner inconnu lorsqu'une règle exige un lien qui n'a pas été établi.

La mise à la terre temporelle a des couches similaires. Les dates peuvent être explicites ou relatives. «Dans les trente jours» nécessite un événement d'ancrage et une politique de calendrier. «Avant la libération» peut se référer à une version planifiée ou réelle. «Après approbation» peut signifier immédiatement après ou à tout moment ultérieur. Les juridictions définissent les jours ouvrables différemment. Un solveur temporel peut gérer les intervalles et l'ordre exactement une fois ces choix effectués, mais il ne peut pas déduire l'ancre prévue d'une étiquette arbitraire. Les observations temporelles devraient donc inclure l'expression textuelle, la valeur ou la contrainte normalisée, l'ancrage, le calendrier, le fuseau horaire, la confiance et les alternatives.

L'échouement normatif est souvent mal géré. «Must», «doit», «devrait», «peut», «peut» et «devrait» avoir une force spécifique au domaine. Les conventions de rédaction juridique diffèrent du langage ordinaire. Une exception peut faire échec à une règle, en restreindre la portée ou créer une autorisation distincte. Les autorités en conflit peuvent avoir des priorités. Une simple représentation booléenne de la conformité perd ces distinctions. La logique déontique, les priorités de règles, les tableaux de décision ou les systèmes défaisibles peuvent les modéliser, mais la cartographie source-à-règle reste un jugement linguistique et institutionnel.

Des formalisations fallacieuses surviennent lorsque le mauvais programme produit le résultat attendu. L'analyse sémantique faiblement supervisée a rendu ce problème explicite, mais il est omniprésent dans l'évaluation de LLM. Une formule générée peut résoudre un benchmark car les exemples ont des modèles réguliers. Une requête text-to-SQL peut renvoyer les lignes actuelles correctes malgré une mauvaise jointure. Un modèle de planification peut trouver un plan parce qu'une contrainte omise n'est pas pertinente dans le cas testé. Un circuit de règles peut passer dix exemples tout en codant le mauvais concept. La précision de la réponse finale ne peut à elle seule distinguer ces cas.

La défense appropriée est la mutation sémantique. L'évaluateur modifie un aspect de la source ou de l'environnement tout en préservant les autres, puis vérifie si la formalisation change de la manière attendue. Si «tous» devient «certain», le quantificateur devrait changer. Si une exception est supprimée, le chemin précédemment exempté devrait devenir interdit. Si deux entités sont renommées à la même chaîne, l'identité ne devrait pas fusionner automatiquement. Si l'instance de la base de données change, une requête SQL correcte accidentellement devrait échouer. Le test de mutation évalue la signification du programme plutôt que son comportement sur un seul appareil.

La génération de contre-exemples peut comparer les formalisations concurrentes. Compte tenu de deux programmes logiques ou exécutables, un solveur peut rechercher un modèle dans lequel leurs sorties diffèrent. Le scénario qui en résulte est traduit en langage naturel et présenté à un examinateur. Cela transforme une comparaison de formule opaque en une question concrète: «Sous l'interprétation A, un superviseur qui n'a pas suivi de formation peut approuver; sous l'interprétation B, ils ne peuvent pas. Ce qui est prévu?» La méthode est

particulièrement utile lorsque plusieurs LLMs produisent des formes plausibles mais non équivalentes.

La conception d'ontologie crée un autre mode de défaillance. Un schéma universel est attrayant pour la réutilisation, mais il peut cacher les désaccords de domaine et forcer les concepts dans des catégories inappropriées. Un schéma extrêmement étroit, en revanche, produit de nombreuses langues locales incompatibles. Un design en couches est préférable. La couche supérieure définit la source, les preuves, les hypothèses d'identité, les mondes, le temps, la modalité, le statut, la couverture et les lacunes. Les paquets de domaine définissent les types et les relations de fond. Les paquets de cartographie déclarent les correspondances, les pertes et les hypothèses entre les domaines.

La compatibilité de l'ontologie doit être opérationnelle, et non rhétorique. Un circuit déclare les types et garantit qu'il consomme. Un producteur déclare quels types il peut se concrétiser, en vertu des domaines et des modes de couverture. La publication vérifie si un producteur compatible existe. Runtime vérifie si le document concret a donné les observations requises. Si le circuit a besoin d'identités confirmées par l'homme, mais que seuls les candidats à l'identité proposés par le modèle existent, il ne peut pas se présenter à la garantie revendiquée. Si une règle temporelle a besoin d'un calendrier juridictionnel absent, le résultat est un écart opérationnel, pas une date devinée.

Le savoir du monde est particulièrement dangereux. LLMs contiennent de larges connaissances, mais pas de provenance stable, de date d'entrée en vigueur ou de juridiction. Le traitement de la mémoire du modèle comme une base de fait formelle crée des dépendances invisibles et des conclusions viciées. Un système symbolique devrait charger des packs de connaissances explicites avec des identifiants de source, des digests, des intervalles efficaces, des juridictions, des politiques de conflit et des plafonds de garantie. Le paquet peut être organisé, récupéré ou assisté par modèle, mais ses revendications acceptées sont des artefacts externes. Un pack légal peut expirer. Un pack scientifique peut être remplacé. Un pack de monde fictif peut délibérément contredire la réalité actuelle.

Les conflits entre les sources ne doivent pas être effacés par le vote à la majorité, à moins que la tâche ne définisse une telle politique. Le système peut avoir besoin d'un raisonnement paraconsistant, d'une priorité de source, d'un versionnage temporel ou de mondes séparés. Une revendication peut être soutenue par une source et contredite par une autre. Le résultat canonique devrait préserver à la fois et énoncer quelle politique, le cas échéant, les résout. Ceci est essentiel pour les enquêtes scientifiques et l'analyse réglementaire, où le désaccord est l'information plutôt que le bruit.

Les hypothèses de fond doivent être visibles. Les repères logiques supposent souvent que les noms désignent des individus distincts ou que les catégories sont disjointes. Les documents réels énoncent rarement tous ces axiomes. Un LLM peut les insérer car ils rendent le puzzle résoluble. Dans un système de recherche ou de conformité, ces hypothèses devraient apparaître dans une couche d'autorité distincte. La réponse finale peut alors indiquer qu'une conclusion ne suit que si l'hypothèse tient. Cela empêche la connaissance implicite du monde de se faire passer pour des preuves de documents.

La couverture est la mise à la terre de l'absence. Pour conclure qu'un document ne contient pas l'approbation requise, le système doit savoir ce qui a été recherché, quelles sections et quels canaux ont été inclus, comment le concept a été reconnu et si la méthode de recherche a été complète pour la demande. Un balayage lexical peut être complet pour une phrase exacte,

mais pas pour les paraphrases. La récupération peut être un rappel élevé mais pas un monde fermé. Un registre organisé par l'homme peut être complet pour une libération. La couverture doit être un objet de première classe consommé par des règles négatives.

L'architecture nllAgent contient une version importante de ce principe. LongTextJS enregistre la couverture et les lacunes parallèlement aux observations. La compatibilité du circuit vérifie si le programme réel satisfait à l'état et à la couverture requis. Les connaissances manquantes ne sont pas déclarées comme étant conformes. Ceci est plus fort que le modèle commun de demander à un LLM de retourner true, false ou non trouvé, car non trouvé est interprété par rapport à un domaine de recherche documenté [AssistOS-AI 2026].

La difficulté restante est de savoir comment valider les observations produites par modèle. La validation du schéma est nécessaire mais faible. L'ancrage des preuves empêche les citations inventées mais pas l'interprétation erronée. Les scores d'étalonnage aident à classer les candidats mais n'établissent pas de signification. L'approche pratique la plus forte combine des signaux indépendants: analyseurs de langage contrôlé pour fragments tractables; ensembles modèles ou invites diverses; contraintes d'ontologie; cohérence de phrase croisée; mutations sémantiques; exemples négatifs; revue humaine pour les distinctions à fort impact; et contre-exemples en aval qui exposent les conséquences.

La mise à la terre doit également être axée sur les requêtes. Un système n'a pas besoin de résoudre chaque entité ou événement dans un document de 300 pages. Il devrait tirer une demande sémantique de la règle en cours d'exécution, récupérer les portées pertinentes et ne matérialiser que les types nécessaires. Cependant, le traitement de la requête d'abord ne doit pas confondre la pertinence avec l'exhaustivité. Si une règle demande si une exception existe n'importe où dans le texte de l'autorité, la récupération doit fournir un argument de couverture ou la réponse reste inconnue. L'exécution axée sur la demande réduit le travail; elle ne justifie pas une fermeture prématurée.

La conclusion fondamentale est sévère mais constructive. Il n'y a pas de remède purement symbolique pour un symbole attaché au mauvais référent. La mise à la terre n'est pas un détail préliminaire NLP; elle fait partie du calcul de confiance. Le système doit exposer la chaîne de la source au symbole, préserver les alternatives, suivre la couverture et refuser l'exécution incompatible. Une fois cette chaîne disciplinée, les outils formels matures peuvent ajouter une fiabilité substantielle. Sans elle, les outils ne font que certifier les conséquences d'une fiction plausible.

5.2 Pragmatique, non-monotonie, échelle, sécurité et échec de l'évaluation

Même une représentation bien fondée peut échouer parce que le langage naturel fait plus que des faits monotones d'état. Les orateurs impliquent, présupposent, révisent, citent, plaisantent, négocient et s'appuient sur le contexte institutionnel. Les documents contiennent des exceptions, des priorités, des définitions qui changent plus tard et des énoncés dont la force dépend du genre. Le langage naturel exécutable doit décider lequel de ces phénomènes il entend soutenir et doit revenir inconnu en dehors de cette portée.

La pragmatique est la première frontière. Une phrase peut communiquer plus que ses conditions de vérité littérale. «Le rapport est presque complet» peut impliquer qu'il n'est pas complet. «Certains tests passés» peuvent suggérer pragmatiquement que tous ne l'ont pas fait, bien que la logique classique n'implique pas cela. L'ironie inverse le sentiment apparent. Les requêtes indirectes utilisent les déclarations comme actions. Un formalisateur qui traite les inférences pragmatiques comme des faits peut surmonter; celui qui les ignore peut manquer l'intention pratique de l'auteur. Le traitement approprié dépend de l'application. Un moteur de conformité devrait généralement privilégier l'autorité explicite et étiqueter l'interprétation pragmatique comme proposé. Un agent conversationnel peut avoir besoin de modèles pragmatiques plus riches, mais ne devrait pas les représenter comme des preuves tangibles.

Le présupposé crée un défi connexe. «Le comité a cessé d'examiner la proposition» suppose qu'elle l'ait examinée. Les descriptions définitives supposent souvent l'existence ou l'unicité. Les documents peuvent exploiter de telles constructions sans affirmer explicitement le contexte. Un système formel doit décider si les présupposés entrent comme hypothèses, déductions défaisables ou exigences de clarification. Les affirmer automatiquement peut être dangereux; les ignorer peut rendre le discours incohérent. Le système de statut devrait préserver son origine.

Les règles de langage naturel sont souvent non monotones. De nouvelles informations peuvent rétracter une conclusion. «Les employés peuvent accéder au système» peuvent être vaincus par «à moins que leur certification n'ait expiré». La logique par défaut, la programmation de l'ensemble de réponses, les logiques défaisables et les règles priorisées fournissent des mécanismes formels pour les exceptions. La logique classique de premier ordre peut encoder explicitement les exceptions, mais les grands ensembles de règles deviennent difficiles à gérer et les informations incohérentes peuvent banaliser l'inférence si l'explosion ordinaire s'applique.

Une durée d'exécution pratique devrait supporter la sémantique des règles multiples. Monotone Datalog est approprié lorsque les faits ne s'accumulent que. Les règles défaisables sont appropriées pour les défauts et les exceptions. Un raisonnement paracohérent est approprié lorsque les sources sont en conflit et que le système doit éviter de tout dériver. Les tableaux de décision sont appropriés lorsque les combinaisons de politiques sont finies et explicites. La sémantique choisie doit être versionnée avec le paquet de règles; elle ne peut pas être déduite ad hoc par le LLM pendant l'exécution.

Les systèmes normatifs ajoutent autorité et priorité. Une règle ultérieure peut passer outre une règle antérieure. Une réglementation de niveau supérieur peut préempter une procédure locale. La compétence et la date d'entrée en vigueur sont importantes. Les autorisations et les obligations peuvent entrer en conflit. Les paradoxes déontiques montrent que les encodages

logiques naïfs peuvent produire des résultats contre-intuitifs. De nombreux cas pratiques sont mieux traités par des priorités de règles explicites, des exceptions et des conditions de violation que par un calcul déontique universel. Le langage naturel exécutable devrait favoriser la clarté opérationnelle par rapport à la grandeur théorique.

L'échelle crée plusieurs explosions en interaction. Les documents longs contiennent de nombreuses portées et des observations candidates. Les mentions ambiguës créent des devoirs d'identité combinatoires. Les relations temporelles créent des alternatives d'ordre partiel. Les règles créent des conséquences dérivées. L'exécution symbolique crée des chemins. Les solveurs sont confrontés à de grands systèmes de contraintes. Les fenêtres et les coûts de contexte LLM imposent des limites distinctes. Une architecture naïve qui formalise chaque phrase dans chaque logique possible est infaisable.

La matérialisation axée sur la demande est la première réponse. Les circuits déclarent ce qu'ils consomment. La structure d'exécution indexe et les informations lexicales stables, puis ne produisent une sémantique plus riche que pour les portées pertinentes. Le calcul incrémental réutilise des artefacts inchangés. L'interprétation abstraite résume les alternatives. Chemins de contrôle d'exploration symboliques de fusion et d'exploration bornés. Les portefeuilles Solver acheminent des cas simples vers des moteurs bon marché. Les budgets durs produisent des résultats explicites incomplets plutôt que de la troncature cachée.

La deuxième réponse est la modularité sémantique. Un document peut être analysé à travers des vues: sections, fenêtres temporelles, juridictions, scènes narratives, étapes expérimentales ou couches d'autorité. Les conséquences de l'identité et des règles peuvent être locales à un point de vue. Les relations transversales ne sont introduites que lorsque cela est nécessaire. Cela réduit la fusion planétaire accidentelle et soutient le parallélisme. Le système devrait encore enregistrer quand une conclusion dépend de l'hypothèse selon laquelle une vue est complète.

La troisième réponse est la précision de l'étape. Une passe structurale ou lexicale peu coûteuse élimine de nombreux cas. Une passe d'extraction assistée par modèle produit des candidats pour les autres. L'analyse abstraite identifie les violations potentielles. Le raffinement symbolique ne traite que les ambiguïtés pertinentes pour le résultat. L'examen humain est réservé aux choix non résolus à fort impact. Cela ressemble à des pipelines d'analyse statique qui utilisent des analyses bon marché avant que le théorème coûteux ne prouve.

Des menaces pour la sécurité surviennent à chaque étape. L'injection rapide dans les documents sources peut demander au LLM d'ignorer les schémas ou de fabriquer des résultats. Le traducteur devrait recevoir du contenu source en tant que données sous une invite spécifique au rôle, et sa production devrait être validée indépendamment. La source ne doit jamais obtenir l'autorisation de modifier l'exécution, le paquet de règles ou la politique de publication. Tout texte prétendant être une instruction système est toujours un contenu de document.

Le code généré est un risque encore plus grand. Un DSL qui permet arbitraire JavaScript, Python, importations ou accès réseau transforme l'apprentissage sémantique en exécution de code. Les circuits déclaratifs devraient se normaliser en données claires et appeler uniquement les opérateurs enregistrés. Les algorithmes personnalisés doivent être installés en tant qu'extensions examinées avec des contrats typés, des effets, des limites de ressources et des digests verrouillés. Le sandboxing reste nécessaire, mais la restriction de capacité est plus forte que de ne compter que sur une limite de processus.

Les attaques de solveurs incluent le déni de service par des contraintes pathologiques, des objets de preuve malformés et l'exploitation de bibliothèques indigènes. Les budgets, les restrictions théoriques, la vérification des preuves et l'épinglage de version sont requis. Un modèle peut construire délibérément ou accidentellement une formule techniquement valide mais informatiquement explosive. La sélection du formalisme devrait estimer le coût et rejeter les tâches en dehors de la politique.

L'empoisonnement des données affecte les producteurs sémantiques et les suites de référence. Si un agent de codage génère à la fois un circuit et les exemples qui le valident, les artefacts peuvent partager le même malentendu. Génération de tests indépendantes, mutations sémantiques, cas d'ancrage organisés manuellement et rôles de révision distincts réduisent ce risque. L'architecture d'apprentissage nllAgent empêche correctement un agent de codage de publier son propre candidat. La publication manuelle est un contrôle de la gouvernance, bien que son efficacité dépende de la qualité des repères et des contrôles indépendants.

La réalisation en langage naturel peut réintroduire des allégations non étayées. Un modèle peut convertir «possible sous une seule interprétation» en «le document viole la règle». Il peut omettre des hypothèses ou transformer une recherche incomplète en absence. La réalisation devrait donc être limitée par un schéma de résultats canoniques et vérifiée au niveau de la réclamation. Les sorties à impact élevé peuvent utiliser des modèles contrôlés. Les commentaires explicatifs devraient être clairement séparés des constatations vérifiées.

L'évaluation est peut-être l'échec le plus systémique. De nombreux benchmarks ne mesurent que la précision de la réponse finale. Cela confond l'extraction, la formalisation, le raisonnement et le rendu. Un modèle peut atteindre la bonne réponse pour la mauvaise raison. Un système formel peut être pénalisé pour être retourné inconnu lorsque l'indice de référence suppose une convention non déclarée. Les ensembles de données synthétiques peuvent récompenser la reconnaissance de modèles. Les métriques LLM-as-a-juge peuvent partager des biais avec les systèmes évalués.

Un benchmark rigoureux doit contenir plusieurs couches. La couche source inclut le texte et la structure exacts. La couche sémantique comprend une ou plusieurs interprétations admissibles avec des plages de preuves. La couche formelle comprend des programmes cibles ou des tests d'équivalence extensifs. La couche d'exécution comprend des modèles attendus, des épreuves, des témoins ou des noyaux insatisfaisants. La couche de résultat comprend les statuts canoniques et la couverture. La couche de réalisation comprend les revendications autorisées et les surestimations interdites. Les erreurs peuvent alors être attribuées plutôt que s'effondrer en un seul score.

Les paires minimales sont essentielles. Changer un mot ou une relation devrait produire une différence formelle prévisible. Les familles de paraphrase testent l'invariance. Les sections de distracteur testent la récupération. Les collisions d'identité testent la mise à la terre. Des preuves manquantes testent un comportement inconnu. Les sources contradictoires testent la politique de paraconsistance ou d'autorité. Les cas de limites temporelles testent les calendriers et les intervalles. Le texte rapide adverse teste l'isolement de sécurité. Les mutations sémantiques testent si un circuit met réellement en œuvre sa règle.

L'évaluation devrait également mesurer l'abstention. Un système digne de confiance doit savoir quand il manque une ontologie, un producteur, une couverture ou un budget. L'inconnu n'est pas un échec lorsque la question n'est pas étayée. Les métriques doivent distinguer l'abstention justifiée de l'incomplétude évitable. Les courbes de la qualité de la couverture

peuvent montrer comment la précision et la fausse assurance changent à mesure que le système admet plus de sémantique proposée par le modèle.

La mise en œuvre indépendante est importante. Un compilateur de circuit et son évaluateur «d'ombre» ne devraient pas partager toutes les fonctions critiques si des tests différentiels sont revendiqués. Les scores de mutation devraient provenir de mutations exécutables, et non de descriptions en prose. Les profils de modèles en direct doivent être évalués séparément des appareils déterministes. La planification et la réalisation nécessitent des repères spécifiques à la tâche plutôt que d'être déduites du rendement de la vérification. Ce sont précisément plusieurs des problèmes graves reconnus dans le dépôt nllAgent [AssistOS-AI 2026].

Les examens externes doivent être interprétés avec des soins similaires. L'enquête Frontiers 2026 est évaluée par les pairs et utile, mais son affirmation selon laquelle les systèmes fédératifs fonctionnent mieux est explicitement qualifiée par la confusion des tâches. Les enquêtes de planification et d'analyse de programme cartographient les tendances, mais ne peuvent pas établir qu'une architecture proposée fonctionne sur des documents sans restriction. Aucun avis indépendant sur nllAgent n'a été trouvé. Un livre responsable devrait donc utiliser les revues pour identifier le consensus et le désaccord, puis énoncer ses propres jugements en cas d'absence de preuves.

Le principe central de l'évaluation est l'attribution causale. Lorsqu'un système s'améliore, les chercheurs devraient déterminer si le gain provient d'une meilleure analyse sémantique, d'une meilleure correction du solveur, d'une récupération, d'un échantillonnage du temps d'essai ou d'une fuite de référence. Lorsqu'il échoue, ils doivent identifier la couche. Ce n'est pas seulement une hygiène scientifique. Les données d'erreur en couches sont ce qui permet à l'architecture de s'améliorer sans remplacer l'ensemble du système ou de masquer les régressions dans des scores globaux.

Le langage naturel exécutable restera une capacité limitée. La pragmatique, la connaissance du monde ouvert, le contexte social et les institutions en évolution garantissent qu'aucun noyau symbolique fini ne capture tout sens. Le but n'est pas de supprimer cette frontière mais de la rendre explicite et utile. Un système peut offrir des garanties solides pour les fragments structurels, relationnels, temporels, quantitatifs et régis par les règles tout en retournant les résultats conditionnels ailleurs. Le système le plus dangereux n'est pas celui qui admet les limites. C'est un langage qui convertit le langage non modélisé en sortie symbolique confiante et appelle le résultat vérifié.

6. Familles d'architectures pour le langage naturel exécutable

6.1 Conceptions concurrentes : représentations universelles, portefeuilles de formalismes, compilation contrôlée et systèmes query-first

Les architectures de langage exécutable peuvent être organisées en fonction de l'endroit où elles placent l'engagement sémantique. Certains tentent de traduire chaque entrée en une représentation universelle. D'autres conservent plusieurs formulaires candidats et sélectionnent un formalisme par tâche. Certains limitent le langage d'entrée jusqu'à ce que la compilation soit déterministe. D'autres préservent le langage ouvert mais ne formalisent que les observations demandées par une requête. Chaque conception résout un problème différent, et un système mature les combinera probablement plutôt que d'en sélectionner un exclusivement.

La représentation intermédiaire universelle est l'architecture la plus intuitive. Le texte est compilé en un graphique sémantique riche ou un langage logique, et chaque opération en aval interroge ou transforme cet objet commun. Les avantages sont l'unité conceptuelle, la réutilisation et la possibilité de contrôles de cohérence globaux. Une représentation unique peut prendre en charge plusieurs applications et créer une interface stable entre NLP et le raisonnement.

Sa faiblesse est une portée sémantique. Pour remplir chaque tâche, la représentation doit couvrir les entités, les événements, le discours, le temps, la modalité, la quantité, la causalité, les normes, l'incertitude, la provenance et les concepts spécifiques au domaine. Soit le langage devient extrêmement complexe, soit il simplifie les distinctions dont certaines tâches ont besoin. Une représentation universelle crée également un point d'échec unique: une mauvaise analyse globale contamine chaque analyse. La pression pour choisir une identité, une portée ou une ontologie précoce peut cacher l'ambiguïté.

Un pivot sémantique est une variante plus modeste. Il ne prétend pas être une forme logique complète. Il normalise les portées de source, les mentions, les hypothèses d'entité, les observations, les mondes, les valeurs temporelles, la modalité, la provenance, les alternatives, la couverture et les lacunes. Les compilateurs spécialisés réduisent les portions sélectionnées dans les cibles formelles. Cela préserve l'infrastructure commune sans prétendre qu'une sémantique d'exécution correspond à tout le sens. LongTextJS, tel que documenté dans nllAgent, est proche de cette idée: il s'agit d'un monde source adressable et d'un programme d'observation plutôt que d'un langage universel d'économie de théorème [AssistOS-AI 2026].

L'architecture à candidat multiple traite la formalisation comme une recherche. Un LLM produit plusieurs formes logiques, programmes ou graphes sémantiques. Les vérifications de type et les contraintes d'ontologie éliminent les candidats non valides. L'exécution, les exemples et les contre-exemples classent ou distinguent les survivants. Le système peut retourner une conclusion indispensable lorsque tous les candidats sont d'accord et une conclusion lorsque seuls certains le font. Cette architecture est appropriée lorsque l'ambiguïté est réelle ou lorsque la fiabilité de la formalisation est limitée.

Son coût est combinatoire. Les formulaires candidats se multiplient entre les phrases et les choix d'identité. Un produit croisé naïf devient infaisable. La solution est de représenter la

structure partagée et les alternatives locales de manière compacte, d'utiliser des domaines abstraits pour les résumer et d'affiner uniquement les choix pertinents pour les résultats. La confiance peut guider l'exploration, mais les candidats à faible probabilité ne doivent pas être retirés d'une analyse solide à moins que la politique n'accepte explicitement l'incomplétude probabiliste.

La compilation en langue contrôlée adopte l'approche opposée. L'entrée est réécrite dans un langage naturel restreint dont la grammaire et la sémantique sont déterministes ou fortement contraintes. Les utilisateurs peuvent créer directement dans la langue contrôlée, ou un LLM peut proposer une paraphrase contrôlée pour confirmation. Une fois acceptée, la compilation est fiable. Cette architecture est particulièrement forte pour les exigences, les contrats, les procédures et la création de règles.

Sa limitation est la couverture et l'adoption. Les documents ordinaires contiennent des constructions en dehors du fragment contrôlé. La réécriture peut changer de sens. Les utilisateurs peuvent approuver une paraphrase sans remarquer une différence subtile. L'architecture fonctionne mieux lorsque la forme contrôlée est affichée à côté de la source, les différences sont localisées et les contre-exemples formels font des choix importants concrets. Le langage contrôlé doit être un point de contrôle sémantique explicite, et non une normalisation invisible.

Les architectures de requête-première évitent la formalisation totale. Un circuit ou une question déclare les observations et les relations dont il a besoin. L'exécution préserve la source structurellement, récupère les régions pertinentes et ne matérialise que les types sémantiques demandés. Les requêtes relationnelles, les tableaux et les règles de décision fonctionnent sur le monde fini qui en résulte. Cette conception s'adapte mieux aux documents longs et aligne le calcul avec le but.

Une vision de 2026 étroitement liée aux systèmes de base de données distingue les opérateurs sémantiques interprétés des opérateurs compilés. Dans la conception interprétée, un LLM est invoqué dans la boucle de traitement des données pour chaque ligne, enregistrement ou paire de candidats. Dans la conception compilée, le LLM est invoqué lors d'une phase de construction distincte pour traduire l'opération sémantique en code local déterministe qui peut être exécuté à plusieurs reprises sans appel de modèle. Des expériences préliminaires ont rapporté des réductions substantielles de la latence et de l'utilisation du modèle tout en conservant une grande partie de la qualité de sortie [Dong and Wang 2026]. La proposition est importante car elle atteint indépendamment la même intuition architecturale développée dans ce livre: le modèle de langage est le plus utile en tant que compilateur ou constructeur de plan, tandis que le temps d'exécution répété devrait être stable et inspectable. Sa limitation est tout aussi instructive. Le code généré peut approximer efficacement l'opérateur défini par modèle sans fournir une preuve sémantique qu'il capture le concept prévu de l'utilisateur. La compilation élimine la stochasticité par article; elle ne résout pas en elle-même la mise à la terre ou la fidélité aux spécifications.

Le risque est une fausse fermeture. La récupération peut manquer des preuves pertinentes, et l'analyse de la demande peut omettre un concept nécessaire. Les systèmes de requêtes d'abord doivent suivre la couverture et les producteurs. Si une conclusion négative nécessite de rechercher dans l'ensemble du document une exception, quelques passages récupérés sont insuffisants. L'architecture doit distinguer un résultat de requête borné d'une garantie du monde

fermé. Les contrats de compatibilité et de couverture de nllAgent sont un mécanisme important dans cette famille.

Une architecture de formalisme unique réduit toutes les tâches dans une logique, souvent la logique de premier ordre, Datalog, ou SMT. Il simplifie les obligations de mise en œuvre et de preuve. Pour un domaine étroit, cela peut être le meilleur choix. Logic-LM et LINC gagnent une grande partie de leur valeur à partir d'une langue cible stable. Un système de planification peut raisonnablement utiliser un langage de contrainte. Un vérificateur de protocole peut utiliser un calcul de processus.

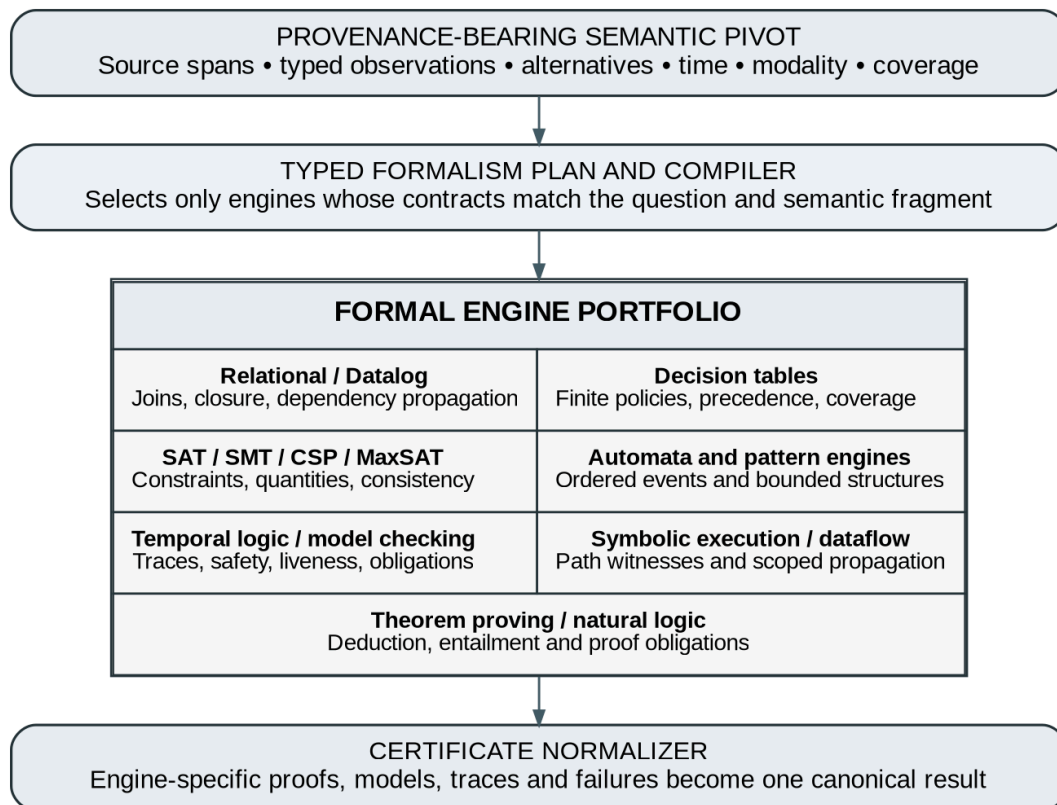


Figure 3. Un pivot sémantique porteur de provenance peut être compilé en un portefeuille de moteurs formels spécialisés et normalisés en un résultat canonique.

Pour les tâches en langage naturel, un portefeuille de solveurs est plus plausible. L'évaluation relationnelle traite les requêtes de documents. Datalog calcule la fermeture monotone. Les tableaux de décision expriment des politiques limitées. SAT et SMT gèrent les contraintes et l'arithmétique. Le contrôle de modèle gère le comportement temporel. L'exécution symbolique construit des témoins de chemin. La logique naturelle gère certaines relations d'implication près du langage de surface. Automata reconnaît les motifs ordonnés. Le pivot sémantique fournit des atomes à la terre, et un plan de formalisme sélectionne un ou plusieurs moteurs.

Le portefeuille crée de nouvelles obligations. Le routeur doit connaître les capacités expressives et opérationnelles de chaque moteur. Un compilateur doit préserver le sens à travers les abaissements. Les résultats de différents moteurs nécessitent un état commun et un modèle de provenance. Si deux moteurs raisonnent par rapport aux prédicats qui se chevauchent, la cohérence doit être vérifiée ou explicitement laissée ouverte. Les politiques de ressources doivent empêcher un modèle de choisir un solveur inutilement coûteux.

Un contrat de formalisme utile indique les types d'entrées, les opérateurs pris en charge, la sémantique, la couverture requise, la classe de garantie, le certificat attendu, le modèle de coût et les limitations connues. Un moteur logique temporel peut nécessiter des événements à la terre et un système de transition fini. Un moteur SMT peut supporter des fonctions réelles arithmétiques entières mais non illimitées. Un tableau de décision peut nécessiter des conditions énumérées finies. Un moteur de logique naturelle peut supporter la monotonie mais pas la co-référence de phrase croisée arbitraire. Le routeur peut alors rejeter un plan proposé dont les exigences dépassent le moteur.

La génération de la preuve est une autre famille d'architecture. L'artefact principal n'est pas la réponse en langage naturel, mais un résultat canonique plus une preuve, un témoin, une trace d'exécution ou une dérivation vérifiable. Le modèle de langage génère une explication à partir de cet artefact. Chaque revendication explicative est liée à un élément de résultat. Un vérificateur indépendant s'assure que l'explication ne dépasse pas le contenu formel.

Cette conception est précieuse car la génération et la vérification ont des points forts différents. Un LLM peut écrire une explication claire d'un contre-exemple ou d'une contrainte insatisfaite. Un résultat symbolique peut garantir que le scénario est réel sous le modèle accepté. L'explication peut encore être trompeuse si elle cache des hypothèses ou de l'incertitude, de sorte que le résultat canonique doit inclure ces dimensions et le contrat de rendu doit les exiger lorsque le matériel.

Les architectures abstraites-sémantiques placent une couche d'interprétation abstraite avant l'exécution spécialisée. Les observations des candidats peuvent et doivent être des domaines. La propagation bon marché détermine quelles propriétés sont définitivement satisfaites, définitivement impossibles ou potentiellement violées. Seules les violations potentielles sont réduites dans des problèmes symboliques coûteux. Cela réduit la charge du solveur et fournit des états inconnus de principe. Il crée également une architecture incrémentielle naturelle pour les modifications de documents.

Les architectures guidées par contre-exemples ferment la boucle. Une preuve ratée ou un témoin fallacieux n'est pas simplement une erreur; elle identifie quelle distinction sémantique est importante. Le système cartographie les faits de solveur à la source, demande à l'utilisateur LLM ou à l'utilisateur d'affiner l'interprétation pertinente, et rejoue l'analyse. Cette architecture est particulièrement prometteuse car elle transforme la rétroaction formelle en compréhension du langage ciblé plutôt qu'en auto-réflexion générique.

Les architectures d'apprentissage régissent la façon dont les nouvelles règles, ontologies et circuits entrent en production. Un extrême permet à un agent LLM de générer du code exécutable pendant chaque exécution. Cela maximise la flexibilité et minimise l'assurance. L'autre extrême opposé ne permet que des règles manuscrites et formellement vérifiées. Un milieu pratique utilise des agents de codage dans les espaces de travail de mise en scène, génère des repères et des mutations, restreint les artefacts aux formes déclaratives et nécessite une publication explicite. L'exécution de la production lit les versions immuables. C'est le modèle de gouvernance mis en œuvre dans nllAgent et devrait être généralisé.

La comparaison suivante résume le compromis de base sans prétendre qu'une conception domine chaque tâche.

Tableau 1. Principales familles d'architecture et leurs compromis essentiels.

Architecture	Force essentielle et risque principal
Représentation sémantique universelle	Réutiliser et cohérence globale; risques de surcomplexité et d'engagement sémantique prématuré.
Pivot sémantique plus des abaissments spécialisés	Préserve les preuves communes et prend en charge plusieurs moteurs; nécessite des contrats de compilateur et de moteurs croisés.
Formalisations multiples de candidats	Représente l'ambiguïté et permet des conclusions peut/doit être concluantes; risque la croissance combinatoire.
Compilation en langage contrôlé	Sémantique forte et formes inspectables par l'utilisateur; sacrifie la couverture et peut introduire des changements de sens lors de la réécriture.
La question-première matérialisation	Échelles aux documents longs et aux besoins spécifiques à la tâche; peut confondre la récupération avec l'exhaustivité, à moins que la couverture ne soit explicite.
Objectif formel unique	Base de confiance simple et évaluation dans un domaine borné; mauvais ajustement pour les phénomènes de langage hétérogène.
Portefeuille Solveur	Correspond à la méthode de calcul à la structure des problèmes; ajoute des obligations de routage, d'interopérabilité et de normalisation des certificats.
Génération de port de preuve	Limite les réponses en langage naturel au contenu vérifié; nécessite une vérification robuste de la réalisation au niveau des réclamations.
Analyse abstraite plus raffinement symbolique	Balance l'échelle et la précision; la solidité reste conditionnelle à la couverture d'interprétation.
Circuits savants gouvernés	Permet l'évolution sans auto-modification en période d'exécution; dépend de repères solides, de tests de mutation et de revue de publication.

L'architecture générale la plus crédible est un hybride des options intermédiaires. Il utilise un pivot sémantique provenant, préserve les alternatives où ils comptent, effectue une matérialisation axée sur les requêtes, des itinéraires vers un portefeuille de formalisme et n'accepte que des résultats soutenus par la preuve ou les témoins. Le langage contrôlé peut servir de point de contrôle interactif pour les formalisations à fort impact. L'interprétation abstraite contrôle l'échelle, tandis que le raffinement guidé par contre-exemples améliore la précision. L'apprentissage reste en dehors de la production.

Cette conclusion n'est pas seulement théorique. Elle se reflète dans la littérature empirique. PAL, SatLM, Logic-LM, LINC, traducteurs de logique temporelle, formalisateurs de planification, SymGPT, et les systèmes de vérification certifiés réussissent tous en réduisant la cible formelle. Aucun résultat ne démontre qu'une logique universelle ou un modèle de bout en bout peut remplacer le portefeuille. L'architecture devrait donc embrasser l'hétérogénéité tout en minimisant les interfaces de confiance entre les composants.

6.2 Un pipeline de référence du texte aux symboles, à l'exécution et aux réponses en langage naturel contraint

Un pipeline de langue exécutable de référence doit être compris comme un compilateur et une chaîne d'outils de vérification plutôt que comme une invite d'agent. Chaque étape matérialise un artefact immuable ou traité de contenu. Une course peut être rejouée sans demander au modèle de reconstruire son raisonnement précédent. Les erreurs sont assignées à des étapes. L'apprentissage peut améliorer un producteur ou une théorie sans changer silencieusement la signification des anciens résultats.

Le premier artefact est le paquet source. Il contient le texte ou les données canoniques, le résumé de révision, le langage, la structure, les canaux et les portées stables. Les paragraphes, les en-têtes, les tableaux, les devis et les blocs de code conservent l'ordre et l'identité. Les enregistrements externes ont des clés également stables. Le paquet source est l'autorité lexicale: il établit ce qui était présent, pas ce qu'il signifie.

Le second artefact est le contrat de tâche. Il énonce la question, la publication des règles, la portée de la source, la garantie souhaitée, les hypothèses autorisées, le monde et la juridiction, les budgets et la politique de production. La tâche détermine la demande sémantique. Une demande de recherche d'acronymes non définis nécessite différentes observations d'une demande de vérification de la conformité temporelle. Le contrat définit également si l'absence peut être conclue et quelles formes d'incertitude exigent un examen humain.

La troisième étape construit un graphique de preuve. Les producteurs déterministes ajoutent des observations structurelles. Les analyseurs en langage contrôlé ajoutent des observations pour les fragments qu'ils reconnaissent. Les producteurs soutenus par LLM ajoutent des événements candidats liés aux schémas, des relations, des identités, des modalités ou des quantités. Chaque observation enregistre les ancres, le producteur, la version du modèle ou de l'algorithme, l'état, les alternatives et la couverture. Le modèle n'écrit pas directement dans le graphique canonique; un matérialisateur revérifie les portées, les types et l'intégrité référentielle.

L'identité et la gestion du monde se produisent dans la même couche, mais ne sont pas résolues silencieusement. Les mentions sont des preuves locales. Les hypothèses d'entité recueillent les mentions dans un monde et une portée. Les candidats à l'identité énoncent des relations identiques, différentes ou non résolues avec le soutien. Les interprétations mutuellement exclusives appartiennent à des mondes séparés ou à des alternatives gardées. Le graphique peut donc représenter un désaccord sans corrompre la cohérence globale.

La quatrième étape calcule un état sémantique abstrait. Les abstractions spécifiques au domaine résument les identités possibles, les ordres temporels, les statuts normatifs, les quantités et les niveaux de preuves. Les faits doivent-ils tenir dans tous les mondes admis; les faits peuvent-ils tenir dans au moins un. Inconnu enregistre une lacune plutôt qu'une supposition probabiliste. L'état abstrait prend en charge la propagation de règles bon marché et identifie quelles questions nécessitent une plus grande précision.

La cinquième étape produit un plan de formalisme. Le planificateur n'est pas une décision de forme libre LLM. Il compare la tâche avec les contrats de formalisme enregistrés. Une simple requête de document peut rester relationnelle. Une politique peut se regrouper dans les tableaux de décision et les contraintes. Une procédure peut se regrouper dans un système de transition. Une violation dépendante du chemin peut invoquer une exécution symbolique. Une créance déductive peut invoquer un théorème. Les noms de plans nécessitaient des entrées,

des abaissements, des versions de moteur, des limites de ressources et des certificats attendus.

Le LLM peut proposer un plan car il reconnaît la structure du problème et le langage de domaine. Un vérificateur déterministe valide que le plan utilise des formalismes soutenus, que chaque observation requise a un producteur compatible, et qu'aucune distinction sémantique essentielle à la tâche n'est perdue. Par exemple, une règle impliquant des exceptions et une priorité ne peut pas être compilée dans un moteur monotone sans un codage explicite. Une règle impliquant l'identité ne peut pas se poursuivre si le domaine d'identité n'est pas résolu et que le verdict en dépend.

La sixième étape effectue la compilation typée. Ce compilateur doit être petit, déterministe et fortement testé. Il convertit les objets sémantiques en variables de solveur, relations, transitions ou prédicats. Les cartes sources relient chaque élément formel aux observations et aux travées. Les compilateurs devraient générer des obligations de preuve lorsqu'un abaissement est perdu ou dépend d'hypothèses. Différents compilateurs peuvent être comparés à travers des mutations sémantiques et des modèles distinctifs.

La septième étape est l'exécution formelle. Les moteurs Relationnels et Datalog calculent les fermetures. Les tableaux de décision identifient les lignes assorties et contradictoires. SAT/SMT/CSP solveurs trouver des modèles ou prouver l'infaisabilité sous leurs théories prises en charge. Les vérificateurs de modèles évaluent les propriétés temporelles. Les exécuteurs symboliques explorent les conditions du chemin et rendent des témoins concrets. Les assistants de preuve ou les preuves de théorème produisent des dérivations. La sortie du moteur n'est pas encore un verdict face à l'utilisateur; il s'agit d'un enregistrement d'exécution portant un certificat.

La huitième étape est la vérification du replay. Un vérificateur distinct vérifie le témoin ou la preuve contre le problème formel lié à la source et la théorie publiée. Pour une conclusion de document, il confirme que les observations citées existent, s'étend à la révision de la source, les statuts satisfont à la règle et la trace d'exécution soutient la réclamation. Pour un résultat négatif, il confirme que les hypothèses de couverture et d'exhaustivité sont suffisantes. Replay est précieux même lorsque le solveur sous-jacent est fiable car il valide la connexion entre la sortie candidate et la stratégie publiée.

Si l'analyse abstraite prédisait une violation mais que l'exécution symbolique ne peut pas la réaliser, le raffinement sémantique commence. Le système extrait le noyau insatisfaisant, les prédicats contradictoires ou l'échec de la preuve. Les cartes sources identifient les travées pertinentes et les choix sémantiques. Le LLM reçoit une tâche bornée: reconsidérez ce lien d'identité, la portée temporelle, la cartographie d'exception, ou l'alignement de prédicat. Il propose des candidats révisés; le matérialisateur et le compilateur les revalident; l'exécution se répète. Un examen humain est demandé lorsque la distinction a un impact élevé ou reste en suspens.

Cette boucle devrait être monotone en ce qui concerne la preuve, pas seulement la confiance. Un raffinement peut diviser une entité, réduire un intervalle de temps, ajouter une exception négligée ou marquer un prémisses non pris en charge. Il ne devrait pas mettre à niveau une observation proposée en fait parce que le nouveau résultat est pratique. Chaque changement est enregistré comme une nouvelle révision sémantique. Les chemins précédents restent reproductibles.

Le neuvième artefact est un résultat canonique. Il contient la règle ou l'identité de la requête, le statut accepté, les hypothèses, les must et peut être concluants, les candidats rejetés, les inconnues, la couverture, les preuves de la source, les preuves ou les références de témoins, et les limites d'exécution. Il peut également inclure un graphique d'explication concis à la machine. Cet artefact fait autorité sur tout rendu de prose.

La dixième étape réalise le langage naturel. Le renderer ne reçoit que le résultat canonique et les extraits de source sélectionnés. Il ne réexécute pas le raisonnement original. Chaque phrase générée est associée à une ou plusieurs revendications de résultat. Un vérificateur compare la phrase avec les réclamations autorisées, s'assure que la modalité et l'incertitude sont préservées et rejette les ajouts non pris en charge. Le produit peut distinguer les constatations, les hypothèses, les interprétations et les recommandations.

Par exemple, la bonne réponse peut être: «Sous l'interprétation que les deux références à la Commission désignent le même organe, la règle est violée parce que l'approbation précède l'examen de sécurité enregistré. Le document n'établit pas si les références désignent le même organisme; par conséquent, le résultat global est requis.» Un rendu fluide mais dangereux peut omettre la condition et annoncer une violation. Le résultat canonique empêche cette simplification.

Le pipeline prend également en charge la génération, pas seulement la peluche. Un utilisateur fournit une idée ou un objectif. Les circuits de planification construisent une spécification contrôlée avec des témoins de règle à planifier. Un LLM réalise un brouillon. Les circuits d'audit analysent le projet en utilisant la même voie de preuve et de vérification. La révision se poursuit jusqu'à ce que l'audit canonique réponde à la politique de libération ou que le budget soit épuisé. Le modèle de génération ne devient jamais l'oracle final pour savoir si son propre texte satisfait les règles.

La gouvernance entoure la chaîne d'outils. Les ontologies, les producteurs, les circuits, les compilateurs, les contrats de formalisme et les rendus ont des versions et des digests. Les agents d'apprentissage opèrent dans les zones de mise en scène jetables. La publication nécessite des repères, des mutations sémantiques, des vérifications de compatibilité et une approbation explicite. La production lit les rejets immuables. Les profils de modèles en direct sont évalués séparément parce que les appareils déterministes n'établissent pas la qualité du fournisseur. Les trousseaux de connaissances du monde actuel ont des dates et des juridictions effectives.

L'architecture doit exposer une hiérarchie de garanties. Les garanties structurelles concernent les portées exactes, l'ordre et la validité du schéma. L'ancrage garantit l'alignement des preuves et le statut d'identité. L'exécution garantit les conséquences d'un problème formel. Les garanties de bout en bout les combinent sous condition. L'utilisateur doit pouvoir voir le calque le plus faible. Un résultat dérivé d'événements proposés par modèle et d'une transition vérifiée mécaniquement ne doit pas être étiqueté simplement «vérifié»; il devrait indiquer que l'exécution est vérifiée par rapport aux observations proposées.

Un petit noyau de confiance est un objectif de conception majeur. Le noyau comprend la canonisation source, la vérification des schémas et des types, les compilateurs formels, les exécuteurs ou les vérificateurs de preuve enregistrés, les vérificateurs de replay et la validation du résultat canonique. Les invites LLM, les générateurs candidats, les heuristiques de récupération et les modèles d'explication restent à l'extérieur. Les composants de confiance peuvent toujours contenir des bugs, mais leur comportement est stable, testable et révisable. Il

s'agit d'une amélioration substantielle par rapport à la confiance d'un modèle de fondation en évolution en tant que raisonneur monolithique.

Le pipeline proposé n'est pas la seule mise en œuvre possible, mais il prend en compte les obligations révélées par la littérature. Il explique pourquoi un solveur est nécessaire mais insuffisant. Il donne une interprétation abstraite et une exécution symbolique des rôles distincts. Il prend en charge plusieurs langues formelles sans abandonner une couche de preuve commune. Il permet la créativité de LLM dans la découverte, la réparation et l'explication tout en préservant l'autorité symbolique sur l'ancrage et les résultats acceptés. Plus important encore, cela transforme l'échec en un artefact nommé – un type non pris en charge, une identité non résolue, une couverture incomplète, une perte de compilation, une formalisation insatisfaisable ou une phrase non autorisée – plutôt qu'une hallucination vague.

7. LongTextJS, CircuitJS et nllAgent: une étude de cas critique

Le dépôt public NaturalLanguageLinterAgent est l'une des rares tentatives de rendre explicite la frontière entre l'interprétation du langage et l'exécution des règles dans une implémentation exécutable. Le projet n'est pas un raisonneur général en langage naturel, et sa documentation ne le présente pas comme un seul. Il s'agit d'une bibliothèque Node.js 22+ ESM et d'un runtime en ligne de commande avec deux modes de produits en langue contrôlée sur la même architecture LongTextJS–CircuitJS. Le mode Audit applique une théorie versionnée à un document Markdown existant et émet un produit d'audit canonique. Le mode de spécification applique les théories de planification à une idée de haut niveau et émet un plan de génération vérifié; un modèle de langage optionnel peut réaliser ce plan en tant que prose, mais les circuits d'audit inchangés restent l'oracle d'acceptation [AssistOS-AI 2026].

Le dépôt est utile comme étude de cas car il incarne la thèse centrale de ce livre sous forme logicielle: les modèles peuvent proposer des observations sémantiques ou de la prose, mais l'autorité de production est affectée à des rejets immuables, à des vérifications de compatibilité explicites, à l'exécution restreinte de circuits, à des vérificateurs et à des artefacts canoniques. Le projet tient également un registre public des questions graves. Cette pratique est importante parce qu'une grande surface de documentation peut autrement faire ressembler les intentions architecturales à des garanties mises en œuvre. La discussion ci-dessous distingue ce qui est établi sur le plan opérationnel de ce qui reste une hypothèse de recherche.

Aucune évaluation indépendante évaluée par des pairs ou examen technique par des tiers de nllAgent, LongTextJS ou CircuitJS n'a été localisée par le biais de recherches des noms du projet, du nom du dépôt, de la documentation URL et des combinaisons avec «revue», «évaluation», «benchmark» et «réplication» au 2 août 2026. Les résultats de la recherche ont conduit au projet lui-même plutôt qu'à une évaluation externe. L'arrêt de ce chapitre est donc l'examen architectural indépendant de l'auteur, basé sur le dépôt public, la documentation, les exemples exécutables, le processus de libération déclaré et le dossier de questions graves en cours. L'absence d'un examen externe n'est ni l'éloge ni la critique; cela signifie que les revendications de projet devraient rester étroites jusqu'à ce qu'elles soient reproduites de manière indépendante.

7.1 Ce que l'architecture publique met en œuvre et pourquoi ses limites comptent

Le système définit trois activités de production qui sont délibérément séparées. L'audit de production lit une version publiée immuable, compile un document dans LongTextJS, exécute les circuits de validation de la version, vérifie les résultats des candidats et persiste un résultat canonique CNL/Audit-1 avec un rapport rendu. La spécification de production compile une idée dans les observations LongTextJS, applique des circuits de planification et produit un artefact canonique CNL/Plan-1; la réalisation optionnelle peut alors générer Markdown et la soumettre à la même machine d'audit. L'apprentissage se produit dans un espace de travail de mise en scène jetable. Un agent de codage peut générer des artefacts et des repères de création de candidats, mais les contrôles côté hôte admettent uniquement les fichiers en liste blanche, et un mainteneur doit explicitement publier et activer une version. Cela rend l'apprentissage analogue au développement logiciel régi plutôt qu'à l'auto-modification en temps d'exécution.

L'unité de travail la plus importante n'est pas le rapport final, mais le répertoire d'exécution. Une vérification préserve l'entrée copiée, le programme source, la base sélectionnée, les résultats de compatibilité et de couverture, la trace de circuit, les résultats du vérificateur, les résultats, les constatations, le rapport lisible par machine canonique et la vue lisible par l'homme. Les séries de planification préservent les artefacts de plan correspondants. Une course qui s'arrête parce que les connaissances manquent enregistrent toujours l'échec et peuvent créer un problème réutilisable. La règle de conception selon laquelle les connaissances manquantes ne sont jamais signalées comme conformité est fondamentale. Il empêche un échec commun des agents de documents: traduire «Je n'ai pas trouvé de preuves» en «le document satisfait à la règle».

LongTextJS ne doit être compris comme un programme côté source que dans un sens contraint. Il n'attribue pas de comportement impératif à des phrases arbitraires. Il compile une révision de source fixe dans un monde fini et interrogeable de structures et d'observations. Les propriétés précieuses sont l'identité source, les portées stables, les types d'observation versionnés, l'identité du producteur, la couverture, les lacunes explicites et la rejouabilité. Un artefact côté document peut être exécuté parce que les consommateurs enregistrés peuvent interroger et transformer ces relations sous la sémantique déclarée; il est plus proche d'une représentation intermédiaire sémantique ou d'un programme de chargement de base de données que d'un code d'application illimité.

Cette distinction est plus que la terminologie. Lorsqu'un modèle traduit une phrase, sa sortie ne devrait pas faire autorité simplement parce qu'elle est sérialisée dans un langage de programmation. Le texte source reste une preuve; l'observation traduite enregistre une interprétation de cette preuve, générée par un producteur désigné sous un schéma et un profil de modèle. Le circuit reste une théorie distincte. Une constatation ne devient publiable qu'après qu'un vérificateur a vérifié les conditions attendues par la version. Ces couches empêchent un analyseur de devenir un juge silencieusement et empêchent une explication de devenir le seul enregistrement d'une inférence.

Le pack de fondation actuel illustre la portée sémantique prévue. Les commandes d'audit, de plan et de référence ordinaires sélectionnent une base bornée-core@1.1.0, sauf invalidité explicite. Le paquet reconnaît les formes contrôlées-anglaises documentées pour l'état, le type, la préséance temporelle exacte exprimée par «avant», les affirmations arithmétiques, de quantité

et d'émotions littérales exactes. Cinq circuits vérifiés par replay ne vérifient que les invariants logiques, temporels, arithmétiques, physiques élémentaires et émotionnels/types publiés. Le pack sélectionné et le digest sont persistés. L'évolution des faits politiques, géographiques, sociaux, économiques et juridiques est intentionnellement exclue de la vérité intemporelle. C'est une frontière défendable du monde-modèle. Un système qui code en dur des faits actuels sans dates d'effet, juridictions, sources et politique de conflit ne peut pas offrir une exécution sémantique reproductible.

Le système prend en charge la matérialisation déterministe et assistée par modèle. La sélection de la traduction peut préférer un agent de modèle de langue `AchilleAgentLib` configuré, utiliser l'adaptateur d'agent de codage contraint actuel, ou exécuter sans modèle pour les agents déterministes. Chaque appel de modèle reçoit une compétence spécifique au rôle plutôt que l'ensemble de l'espace de travail de l'agent. Si une observation critique ne peut pas être matérialisée dans le schéma et le budget de ressources requis, le résultat est incompatible, incomplet ou épuisé. Le modèle ne contrôle pas la construction de la source, la politique de publication ou la signification d'un vérificateur. C'est précisément la bonne direction pour un système neuro-symbolique fédératif: le modèle est puissant à la limite, tandis que les artefacts symboliques et l'acceptation du contrôle de la politique hôte.

La couverture est considérée comme une obligation exécutable plutôt que comme un score de confiance informel. La compilation de circuits fonctionne en arrière à partir de nœuds atteignables en sortie pour déterminer quelles observations externes peuvent influencer un résultat. Les documents de publication établissent un lien entre ces demandes et les producteurs disponibles. La compatibilité du temps d'exécution demande ensuite si le document concret a effectivement donné des observations avec le type, le statut, la cardinalité, la structure, le canal et la couverture requis. La capacité déclarée d'un producteur n'est pas confondue avec la preuve que le document actuel a été traité avec succès. Lorsqu'une demande critique est non satisfaite, la course s'arrête à une limite nommée.

Cette conception axée sur la demande est bien adaptée aux documents longs. Il évite de poser un modèle pour produire une interprétation tout usage de chaque phrase avant qu'une question ne soit connue. Un circuit peut demander les dimensions sémantiques dont il a besoin: termes exacts, assertions d'état, quantités, relations temporelles, ou observations de domaine. Le compilateur ne peut matérialiser que les observations pertinentes à la sortie et persister ce qui était et n'était pas couvert. L'avantage scientifique n'est pas seulement un coût moindre. Elle limite les engagements sémantiques inutiles et rend visible la formalisation incomplète.

`CircuitJS` est la représentation côté théorie. Les modules de circuit modifiables par l'agent sont limités à une expression déclarative, normalisés aux données simples et transmis par le même compilateur utilisé pour les descriptions de circuits non-code. Ils ne peuvent invoquer que les opérateurs et les vérificateurs enregistrés; ils ne peuvent pas cacher le comportement exécutable arbitraire à l'intérieur d'une règle apprise. Une application hôte peut installer une extension d'exécution révisée contenant des algorithmes réels, mais les entrées et le contexte sont des copies de données simples gelées, les résultats doivent être des données simples, l'identité du cache comprend le condensé d'extension et les versions qui dépendent du verrouillage de l'extension qui digère. L'interface standard en ligne de commande n'accepte pas les chemins d'extension exécutables arbitraires.

Cette frontière aborde l'un des risques pratiques les plus graves dans les systèmes de règles «apprenants». Si un agent de codage peut générer JavaScript arbitraire dans chaque

circuit, l'apprentissage d'un linter devient une génération de code à distance et le comportement sémantique d'une libération devient difficile à inspecter. Limiter la création de circuit ordinaire aux opérations enregistrées crée un vocabulaire opérationnel fini. Les algorithmes spécialisés restent possibles, mais ils entrent par un chemin d'ingénierie, de révision et de versionnage distinct. L'architecture distingue donc l'évolution des théories de l'évolution du temps d'exécution de confiance.

La limite du vérificateur est tout aussi importante. L'exécution du circuit peut créer des conclusions des candidats, mais un vérificateur contrôle si ces candidats entrent dans l'audit ou le plan canonique. Dans les exemples bornés, le vérificateur peut relire les ancres sources, les observations, les métadonnées de libération et la politique appliquée. Il ne suffit pas qu'un nœud d'exécution émette un objet dont les champs ressemblent à un avertissement. Le vérificateur établit la propriété spécifique attendue par le produit. Cela ne prouve pas automatiquement que l'interprétation sémantique est correcte, mais il empêche la sortie accidentelle ou malveillante du circuit de se déguiser en une découverte acceptée.

Le mode de spécification montre comment la même architecture peut supporter la génération contrôlée sans accorder l'autorité au modèle d'auteur. Les circuits de planification élaborent un plan canonique et exigent des témoins de règles pour planifier les règles appliquées. La réalisation optionnelle transforme le plan en prose. La prose candidate passe alors par le mode d'audit, et le modèle de réalisation ne juge pas sa propre conformité. C'est plus fort que la génération limitée rapide parce que l'objet faisant autorité est le plan et la trace d'audit, et non l'adhésion autodéclarée du modèle.

Le sous-ensemble expérimental de requête-first est également significatif. Canonical LongTextJS est exposé comme un monde de requête en lecture seule fini. Les requêtes typées et les valeurs de la table de preuve sont abaissées dans l'exécution ordinaire de CircuitJS. Le compilateur vérifie les relations, les expressions, les anti-joints, la commande et les politiques de frappe; les documents de publication interrogent les contrats et les cartes sources; les benchmarks comparent l'évaluation directe des références avec l'exécution réduite. La version éditoriale de production actuelle reste délibérément plus simple, tandis que les agrégats, les modèles ordonnés, les index natifs et le raisonnement avancé sont gérés par des opérateurs directs ou des graphiques. Cette retenue est préférable à la revendication d'un langage de requête universel mature avant que la sémantique et l'optimisation ne soient stables.

Le modèle de publication du référentiel renforce la reproductibilité. Les agents nommés possèdent du matériel d'autorité distinct, des schémas, des profils d'extraction, des circuits, des suites de référence, des candidats, des communiqués, des exécutions, des problèmes et des commentaires. Un agent de codage peut proposer des candidats mais ne peut pas les rendre actifs. La production lit une version immuable. Cette structure permet de reproduire la théorie, le fond de teint, le profil du producteur, le digestion d'extension et le vérificateur qui ont généré un résultat. Il permet également à plusieurs agents de codage ou familles de modèles de construire des théories concurrentes sans partager silencieusement une ontologie ou une libération active.

La revendication la plus forte mise en œuvre est donc architecturale plutôt que sémantique: nllAgent a un chemin discipliné de la source immuable aux observations versionnées, de la demande d'observation à la compatibilité, des circuits restreints à la publication contrôlée par le vérificateur, et d'un produit canonique à un rapport lisible par l'homme. Il a des états d'arrêt explicites. Il n'est pas nécessaire de prétendre que chaque document est compatible

ontologiquement. Il traite les résultats des modèles comme des propositions liées au rôle. Il s'agit de réalisations substantielles par rapport aux pipelines conventionnels LLM-agent, où l'invite, l'interprétation, le raisonnement et l'explication n'existent souvent que dans un seul contexte transitoire.

L'architecture n'établit cependant pas que le langage naturel arbitraire a été fondé correctement. Les portées stables prouvent que des preuves existent à un endroit; elles ne prouvent pas qu'un acteur, un événement, une exception ou une modalité a été interprété correctement. Une observation typée peut encore se référer à la mauvaise entité. Une course complète ne peut être complète que pour le contrat d'observation borné, pas pour chaque signification pertinente qu'un humain pourrait déduire. La documentation et le registre de délivrance du projet sont suffisamment prudents sur ce point. La valeur de la mise en œuvre est que ces limitations ont des lieux à représenter plutôt que d'être cachés dans une réponse finale.

7.2 Évaluation indépendante: points forts, problèmes graves actuels et évolution requise

Mon évaluation globale est positive sur les limites de confiance et prudente sur les revendications sémantiques. nllAgent est plus qu'un emballage rapide, plus qu'un pipeline vectoriel-récupérateur, et plus qu'une collection de schémas JSON. Ses artefacts côté source et côté théorie, sa publication immuable, sa limite de création restreinte, sa vérification de replay, ses vérifications de couverture et ses produits canoniques constituent un substrat de recherche crédible. L'architecture est particulièrement bien alignée avec le modèle fédérateur identifié dans les revues neuro-symboliques récentes: la composante neuronale propose ou réalise le langage, tandis que la composante symbolique conserve sa propre identité, sémantique, et les critères d'acceptation [Chatzikyriakidis and Lappin 2026].

Les preuves actuelles du projet soutiennent néanmoins la faisabilité limitée, et non le langage naturel général exécutable. La fondation par défaut reconnaît un petit fragment contrôlé. La démonstration éditoriale utilise des opérateurs et des vérificateurs étroits. La première compilation de requête est expérimentale. La réalisation contrôlée est limitée. Aucune évaluation indépendante n'a établi de fidélité de formalisation, de résolution d'identité, de transfert entre les domaines, d'étalonnage d'erreurs ou de supériorité par rapport à des niveaux de référence plus simples. La bonne stratégie de publication consiste donc à indiquer exactement quelles propriétés sont mises en œuvre et à rattacher toutes les revendications plus fortes à une enveloppe de référence et de garantie.

Le registre actuel des questions graves identifie six limitations d'ingénierie et de sémantique. Le premier est l'incrémentalité de l'ordonnanceur borné. L'exécution a des mécanismes de dépendance et de mise en cache utiles, mais il ne prétend pas encore à un moteur de point de correction incrémental entièrement général pour les circuits arbitraires, les relations récursives, les mondes alternatifs et les mises à jour non monotones. Cela compte pour les documents longs, car une petite modification ne devrait invalider que les artefacts sémantiques et les états de circuit qui en dépendent. Un replay complet naïf est correct mais coûteux; un planificateur incrémental malsain est rapide mais dangereux. La solution nécessite des contrats de monotonie et d'effet explicites, une provenance de dépendance, une séparation d'époque pour les rétractations et des comparaisons de référence entre exécution incrémentale et pleine exécution.

Le deuxième problème est que le support d'interprétation ne circule pas encore uniformément à travers chaque chemin d'exécution. Un document peut contenir des lectures, des mondes, des contextes ou des conditions de présence concurrents, mais un opérateur, une clé de cache, une agrégation ou un vérificateur peut accidentellement les effondrer. C'est l'obstacle central pour traiter LongTextJS comme un état sémantique abstrait. Toute valeur qui dépend d'une interprétation doit porter son contexte. Les jointures doivent combiner des contextes compatibles. L'agrégation doit indiquer si elle se situe dans un monde ou à travers des alternatives. L'identité du cache doit inclure la formule de contexte. Les opérateurs qui ne peuvent pas préserver des alternatives doivent être déclarés incompatibles plutôt que de choisir silencieusement une.

Ce problème n'est pas un écart mineur. La promesse scientifique fondamentale du langage naturel exécutable est souvent que le système peut dire ce qui est vrai sous toutes les lectures admissibles, ce qui est possible sous au moins un, et quel résultat dépend d'une interprétation

contestée. Si des alternatives sont perdues dans une voie d'exécution interne, une interface polie peut/doit être trompeuse. Le projet devrait donc faire de la préservation de l'interprétation un invariant à temps entier et le tester à travers des mondes finis générés, des propriétés métamorphiques et une relecture différentielle.

Le troisième problème est la compatibilité conservatrice pour un comportement procédural opaque. Les extensions et les opérateurs enregistrés sont nécessaires car certains algorithmes utiles sont gênants à exprimer de manière déclarative. Mais un compilateur ne peut pas déduire des exigences sémantiques précises, des effets, de la monotonie, de l'utilisation des ressources ou des obligations de témoin de ZXX0000QXZ opaque. Le rejet conservateur est plus sûr que l'acceptation optimiste, mais l'opacité excessive réduit la composabilité et rend la matérialisation axée sur la demande moins efficace. Chaque primitive procédurale a besoin d'un contrat appartenant au registre déclarant les types d'entrées et de sorties, les statuts épistémiques acceptés, les exigences de couverture, les effets possibles, le déterminisme, la monotonie, le comportement de la cache, la classe de coût et le témoin lisible par vérificateur. Le contrat devrait faire plus autorité que les commentaires rédigés par l'auteur de l'extension.

Le quatrième problème est la planification minimale des requêtes et l'indexation temporelle. Un monde de requêtes fini peut s'exécuter correctement sans optimisation sophistiquée, mais l'utilisation à l'échelle du document nécessitera une planification de la jonction, des estimations de sélectivité, des index source-envergure et d'entité, des index d'intervalle temporel et une maintenance incrémentielle. Ces optimisations ne sont pas seulement des détails de performance. Un planificateur qui réécrit des requêtes doit préserver la multiplicité des preuves, les formules de contexte, l'ordonnancement, la sémantique anti-jointe et la couverture. Les index natifs doivent rester des accélérateurs jetables plutôt que des sources de vérité. Les tests différentiels devraient comparer l'évaluation optimisée et de référence sur les mondes générés, en particulier pour les questions d'absence, le chevauchement temporel, les interprétations alternatives et les preuves en double.

Le cinquième problème est une génération volontairement contrôlée étroite. La réalisation exacte ou vérifiée des conclusions et des plans bornés est précieuse, mais elle ne constitue pas encore une prose vérifiée de forme longue. La génération de forme longue introduit l'agrégation, les pronoms, l'ellipse, l'accent rhétorique, les relations de discours et les implications créées par l'ordre des phrases. Une réalisation ne peut contenir que des déclarations atomiques sous licence et implique encore un récit causal ou normatif non étayé. La prochaine étape devrait être la réalisation porteuse de preuves plutôt qu'un écrivain sans contrainte plus large. Un résultat canonique devrait fournir des identifiants de réclamation, des créneaux d'entité et de quantité, des modalités, de l'incertitude et des relations de discours autorisées. Le texte généré devrait être réétabli et comparé à ces allégations. Les inadéquations critiques doivent rejeter le texte; le matériel stylistique en dehors du fragment vérifié doit être étiqueté à un niveau d'assurance inférieur.

La sixième question est le support de mutation spécifique au scénario. L'exactitude de référence ordinaire peut cacher des règles qui sont inertes, redondantes ou mal étendues. Un circuit peut passer tous les exemples organisés même si une exception est ignorée car aucun cas ne l'active. Une évaluation sérieuse devrait générer des mutants sémantiques dactylographiés qui modifient les seuils, les comparateurs, les quantificateurs, la négation, la modalité, les liens d'identité, l'ordre temporel, l'état des preuves, les prédicats de couverture, la portée de l'exception, la priorité des règles et les voies de vérification. Une suite de test «tue»

un mutant lorsque le résultat canonique attendu change de la manière requise. Les mutants survivants révèlent des tests faibles ou des clauses sémantiquement redondantes. La mutation devrait couvrir les programmes sources, les théories des circuits, les abaisssements des compilateurs et les contraintes de réalisation, pas seulement les chaînes dans les exemples.

Ces six questions sont plus informatives qu'une étiquette générique «prototype» car elles identifient où l'architecture actuelle peut perdre de l'ampleur ou du sens. Ils suggèrent également un chemin d'évolution cohérent. Le premier jalon devrait être une sémantique formelle bornée pour le monde de la boutique et de la requête sémantique. Il n'a pas besoin de formaliser l'anglais sans restriction. Il devrait définir des mondes sources immuables, l'identité d'observation, le statut épistémique, la couverture, les alternatives, les transactions, les résultats de la requête, la publication et la rétractation. Les invariants clés peuvent être vérifiés sur des instances finies et testés en fonction de la propriété dans la mise en œuvre.

La deuxième étape devrait être l'exécution consciente de l'interprétation. Les conditions de présence ou les contextes factorisés doivent accompagner des observations alternatives dépendantes. Le planificateur, le moteur de requête, le registre primitif, le cache, le vérificateur et les produits canoniques doivent soit préserver ces contextes, soit déclarer l'incompatibilité. Le fait de pouvoir, de ne pas, de ne pas être, et d'inconnu doit être calculé à partir d'ensembles explicites ou d'abstractions de mondes admissibles, et non de la confiance du modèle. Cela créerait le pont concret de l'architecture existante à l'interprétation abstraite.

La troisième étape devrait être un registre du formalisme plutôt qu'un CircuitJS en expansion indéfinie. CircuitJS est mieux utilisé comme langage d'orchestration, de contrat de preuve et de publication. Les moteurs spécialisés devraient posséder une sémantique spécialisée: l'évaluation relationnelle et Datalog pour la fermeture et les jointures; les tableaux de décision pour la préséance de la politique; SAT, SMT, CSP, ou MaxSAT pour les contraintes; automates pour les modèles ordonnés; solveurs temporels et vérificateurs de modèles pour les traces; exécution symbolique pour les témoins de chemin; frameworks de flux de données pour la propagation. Chaque adaptateur doit déclarer un fragment d'entrée typé, des hypothèses de traduction, un certificat de sortie, un vérificateur de relecture et des modes de défaillance.

Le quatrième jalon devrait être le raffinement sémantique guidé par des exemples. Une analyse abstraite grossière peut identifier une éventuelle violation. Un solveur ou un exécuter symbolique cherche alors un témoin cohérent. Si le témoin est satisfaisant et rejouable, le système le signale avec des portées source. Si c'est faux, un noyau ou un conflit insatisfaisant devrait identifier quelle identité, quelle identité, quel moment, quel modal, quantité, exception ou distinction de couverture a été perdu. La demande de raffinement ne devrait cibler que la source et le producteur concernés. L'artefact sémantique réparé devient une nouvelle version et le contre-exemple devient un cas de régression. Cette boucle serait une synthèse véritablement nouvelle de la proposition sémantique basée sur LLM, de l'interprétation abstraite et de l'exécution symbolique.

Le cinquième jalon devrait être une couche de mise à la terre plus forte. Les ancrs de sécurité et de source de type sont nécessaires mais insuffisantes. L'exécution a besoin d'ensembles d'identité des candidats, de contraintes explicites identiques/différentes, de validité temporelle, de vérifications d'unité et de juridiction, d'autorités sources et de politiques de conflit. Les liens top-one générés par le modèle ne doivent pas fusionner de manière destructive des objets. Les entités critiques doivent utiliser des identifiants faisant autorité ou une confirmation humaine. Les connaissances externes devraient passer par des packs en

lecture seule, traités au contenu avec des sources, des intervalles efficaces, la juridiction, l'expiration, la résolution de conflits et des plafonds de garantie. La recherche Web peut proposer des mises à jour de pack, mais elle ne devrait pas devenir implicitement une vérité intemporelle.

La sixième étape devrait être une évaluation indépendante. Les appareils déterministes établissent la reproductibilité du logiciel, mais pas la qualité de la traduction sémantique en direct. Un profil d'évaluation opt-in devrait épingler le modèle, le fournisseur, les invites, les versions d'outils, la température et la politique de capture; répéter les cas pour mesurer la variance; enregistrer le coût et la latence; et interdire le repli silencieux. Un évaluateur de référence mis en œuvre séparément devrait comparer les résultats de requêtes et de circuit lorsque cela est possible. L'annotation humaine devrait se concentrer sur la fidélité de la source à la sémantique et les ambiguïtés pertinentes pour les résultats. Les lignes de base devraient inclure l'arrêt direct LLM, LLM plus la récupération, le programme de récupération, le LLM-to-one-formalism, la compilation en langage contrôlé et les systèmes de motifs déterministes. Le projet devrait démontrer quand son architecture supplémentaire produit une valeur mesurable.

Une évaluation utile peut être résumée sans gonfler les revendications. La préservation de la source, les rejets immuables, la persistance de la trace, la séparation des observations de documents de la théorie réutilisable, la couverture explicite, la création de circuits restreints, la publication contrôlée par le vérificateur et l'apprentissage régi sont des décisions de conception fortes. La compréhension sémantique générale, l'interprétation-exécution complète, la planification de requêtes à l'échelle de la production, la génération de formes longues porteuse de preuves, la mutation complète et la validation empirique indépendante restent ouvertes. La crédibilité de l'architecture vient en partie de l'admission de cette frontière.

La principale opportunité scientifique n'est pas d'ajouter plus de syntaxe DSL. Il s'agit de démontrer une solidité sémantique conditionnelle pour un fragment limité mais non trivial. Un résultat publiable pourrait spécifier un langage de document contenant des entités, des événements, des relations temporelles, des quantités, des modalités, des exceptions explicites et des références alternatives; définir l'ensemble d'interprétation admis; prouver ou tester mécaniquement que chaque résultat publié peut/doit être correct par rapport à cet ensemble; montrer que le raffinement symbolique élimine une classe mesurable d'avertissements parasites; et vérifier que les explications générées préservent le résultat canonique. Un tel résultat serait plus étroit que «l'anglais exécutable», mais beaucoup plus important.

La principale opportunité pratique est une nouvelle classe de systèmes de documents et d'agents qui sont prêts à arrêter. Un assistant LLM normal est optimisé pour continuer à produire une réponse. Une durée d'exécution symboliquement fondée peut renvoyer l'ontologie incompatible, l'identité non résolue, la couverture insuffisante, le formalisme non soutenu, l'épuisement budgétaire, le contre-exemple fallacieux ou la réalisation sans permis. Ces résultats peuvent sembler moins impressionnants dans une démonstration, mais ils sont des conditions préalables à une utilisation fiable dans la science, la réglementation, l'ingénierie et la prise de décision institutionnelle.

Mon jugement indépendant est donc que nllAgent a choisi bon nombre des bonnes limites architecturales et devrait résister à la pression pour élargir ses revendications plus rapidement qu'il n'en élargit les preuves sémantiques. Il s'agit déjà d'un prototype crédible d'analyse du langage gouverné, fondée sur la source et séparée par la théorie. Ce n'est pas encore la

preuve que le langage naturel sans restriction peut être exécuté avec des garanties formelles. Sa meilleure voie à suivre est de devenir une plate-forme expérimentale pour la revendication plus étroite et plus défendable développée tout au long de ce livre: LLMs découvrent des candidats formels, un noyau symbolique contrôle la mise à la terre et la conséquence, et la sortie en langage naturel reste subordonnée à un résultat canonique rejouable.

8. Construire et valider un runtime neuro-symbolique

8.1 Stratégie de mise en œuvre: un système de langage exécutable minimal mais scientifiquement honnête

L'architecture proposée dans ce livre n'est ambitieuse que si elle est décrite vaguement. Une fois ses obligations séparées, le premier système implémentable est relativement modeste. Il n'a pas besoin de comprendre le langage arbitraire, de construire une ontologie universelle ou de prouver chaque conclusion. Il doit accepter une classe de documents bornés, préserver des preuves exactes, matérialiser une petite famille d'observations typées, compiler une famille restreinte de règles en un ou deux moteurs formels, renvoyer des témoins ou des inconnues explicites, et empêcher la couche de langage naturel finale de changer le résultat symbolique. La valeur scientifique vient de rendre chaque limitation visible et mesurable.

La première décision de conception est la revendication cible. Un prototype de recherche ne devrait pas commencer par «le système comprend et exécute le langage naturel». Il devrait commencer par une allégation telle que: les documents donnés d'un genre déclaré, d'une ontologie versionnée, d'une famille finie de producteurs d'observation et d'un ensemble de règles publiées, le temps d'exécution détermine si chaque règle est satisfaite, violée, potentiellement violée, non étayée ou en dehors de la couverture sémantique de la version. Le résultat est rejouable et lié à des portées source. Cette allégation est suffisamment restreinte pour être testée et suffisamment large pour appuyer l'examen de la conformité, l'analyse des exigences, la peluche de documents scientifiques, l'inspection des protocoles et la planification contrôlée.

La couche source doit être délibérément ennuyeuse. Chaque document est stocké sous forme de révision immuable avec un digest stable. Les unités structurales – rubriques, paragraphes, phrases, éléments de liste, cellules de table, légendes, notes de bas de page et objets intégrés – reçoivent des identifiants persistants. La normalisation est enregistrée plutôt que silencieusement appliquée. Les décalages de caractères doivent faire référence à une représentation source canonique, tandis que les mappages préservent les positions dans le fichier original. Une déclaration dans un tableau et une phrase visuellement identique dans un paragraphe peuvent avoir des rôles structurels différents; cette distinction doit survivre à l'extraction. Lorsque OCR est inévitable, la sortie OCR et l'image de la page sont des objets de preuve distincts, et l'incertitude dans la transcription n'est pas effondrée dans le texte ordinaire.

Cette discipline source n'est pas une source administrative. C'est ce qui rend les conclusions symboliques contestables. Une conclusion qui ne cite qu'un résumé généré ne peut pas être vérifiée de manière indépendante. Une découverte liée à un texte exact, à des cellules de table et à des enregistrements de transformation peut être revue même si chaque modèle de langue impliqué a changé. Le paquet source devrait donc être considéré comme faisant partie du dossier d'audit de confiance, bien que les extracteurs qui le créent restent faillibles et nécessitent des tests spécifiques au format.

La deuxième couche est un graphique de preuve plutôt qu'une seule analyse. Le graphique contient des mentions, des entités, des événements, des états, des quantités, des expressions temporelles, de la modalité, des relations et des liens sources. Fondamentalement, il représente aussi des alternatives et un statut. Un lien d'entité proposé par un modèle n'est pas stocké

comme une égalité inconditionnelle. Il s'agit d'une relation d'identité candidate avec un producteur, d'un score, d'une portée de soutien, de preuves contradictoires et d'un statut épistémique tel que proposé, corroboré, accepté, rejeté ou non résolu. Une relation temporelle déduite de «bientôt après» ne devrait pas être normalisée à une durée exacte arbitraire; elle doit conserver la relation qualitative et toute interprétation déclarée du domaine.

Cette couche doit rester finie et liée à la tâche. Le système ne construit pas un modèle mondial complet. Il matérialise les distinctions exigées par les circuits publiés et par la requête actuelle. La matérialisation de la requête, déjà explorée dans nllAgent, est la bonne valeur par défaut. Un circuit déclare qu'il a besoin d'identité d'acteur, de relations document-ordre, de modalité déontique et de dates d'entrée en vigueur. Le compilateur tire ces demandes avant une interprétation coûteuse. Les producteurs ne génèrent alors que les observations requises pour décider de la compatibilité et exécuter la règle. Cela réduit les coûts et, plus important encore, limite la surface sémantique sur laquelle des erreurs peuvent se produire.

La troisième couche est un noyau symbolique de mise à la terre. «L'ancrage» est souvent utilisé de manière lâche pour signifier qu'une réponse cite un passage. Ici, il a une signification plus stricte: chaque symbole admis à l'exécution formelle a un référent, la provenance, la portée et le statut de type suffisants pour le formalisme cible. Un prédicat tel que approuvé (x, y, t) ne peut pas être introduit simplement parce qu'un LLM a produit ces jetons. Le noyau vérifie que x et y font référence aux types d'entités admissibles, qu'il s'agit d'une valeur temporelle ou d'un intervalle pris en charge, que le producteur d'observation est autorisé pour ce prédicat, que la portée de la source existe dans la révision déclarée et que le statut répond à l'exigence de garantie de la règle.

Les contrats de mise à la terre devraient distinguer les preuves de l'ontologie. La preuve dit qu'une travée soutient une observation. L'ontologie dit quel type de chose l'observation désigne et quelles relations sont admissibles. Les axiomes de domaine disent ce qui découle de ces relations. Le monde entier fournit des allégations provenant de sources externes. Ces couches ne doivent pas être fusionnées. Un registre de société peut établir que deux noms désignent une entité juridique au cours d'une période déterminée. Une ontologie de domaine peut déclarer que les entités juridiques peuvent signer des accords. Une théorie de la politique peut déclarer qu'une signature d'un représentant autorisé compte comme approbation. Un modèle peut suggérer les trois connexions, mais chaque connexion appartient à une autorité différente et devrait être acceptée ou rejetée séparément.

L'identité mérite son propre service car c'est la dépendance cachée la plus courante dans le raisonnement des documents longs. Le service doit gérer les clusters de mentions, les alias explicites, les identifiants, les rôles organisationnels, la validité temporelle et les mondes sélectionnés. Il devrait permettre à plusieurs candidats non résolus plutôt que de forcer l'unification précoce. Dans une première version, l'identité peut être intentionnellement conservatrice: les identifiants explicites et les alias déclarés exacts sont acceptés; les liens contextuels et fondés sur le rôle restent proposés; l'identité insupportée de l'identité croisée cesse les règles qui l'exigent. Cette politique conservatrice réduira la couverture apparente, mais améliorera considérablement la signification des résultats acceptés.

Le temps a besoin d'un domaine explicite similaire. Une représentation initiale pratique peut combiner des instants exacts, des intervalles fermés ou ouverts, un ordre qualitatif et des limites inconnues. L'exécution doit distinguer l'ordre du document de l'ordre de l'événement et du temps de publication à partir du moment effectif. Des phrases temporelles telles que «avant le

déploiement», «dans les trente jours», «alors que l'autorisation reste valide», et «après tout changement matériel» induisent différentes structures. Le compilateur devrait rejeter une règle temporelle si le graphique de preuve n'a que des observations d'ordre de document lorsque la sémantique événement-temps est nécessaire. La substitution silencieuse entre domaines temporels est une source classique de conclusions plausibles mais invalides.

La modalité et la force normative nécessitent un petit treillis plutôt qu'un booléen. Au minimum, le système devrait distinguer l'affirmation, le rapport, la croyance, la possibilité, la capacité, la permission, l'obligation, l'interdiction, la recommandation, l'intention, la condition et le contre-factuel. Il devrait également enregistrer le titulaire et le bénéficiaire d'une norme, le cas échéant. «Le fournisseur peut divulguer des données» et «le fournisseur doit divulguer des données» partager le vocabulaire, mais avoir des conséquences de conformité opposées. «Le document stipule que le fournisseur s'est conformé» est la preuve d'un rapport, pas nécessairement la preuve de l'événement sous-jacent. Un runtime formel devient utile précisément en refusant ces effondrements.

Une fois que des observations à la terre existent, la quatrième couche calcule un état sémantique abstrait. Un domaine minimal peut représenter, pour chaque proposition pertinente à la question, qu'elle soit vraie dans toutes les interprétations recevables, vraie dans certaines, fausses en tout, contredites ou non résolues en raison de la couverture manquante. La mise en œuvre n'a pas besoin de commencer par une bibliothèque en treillis sophistiquée. Il peut commencer par des alternatives finies explicites et un résumé dérivé mécaniquement peut/doit être résumé. Au fur et à mesure que l'échelle augmente, les domaines abstraits peuvent résumer les candidats à l'identité, les plages numériques, les intervalles temporels, les statuts modaux et les ensembles de provenance sans énumérer tous les mondes.

L'état abstrait effectue deux emplois. Tout d'abord, il fournit une propagation conservatrice à l'échelle du document. Si chaque interprétation acceptée lie une définition à un terme, cette relation est un fait incontournable. Si une seule des trois affectations d'identité plausibles crée un conflit d'intérêts, le conflit est une constatation de la question plutôt qu'une violation vérifiée. Deuxièmement, il décide où l'exécution plus coûteuse est justifiée. Un résultat incontournable peut se terminer tôt. Un résultat de violence peut déclencher une recherche de témoin. Une inconnue causée par une couverture de producteur absente déclenche un problème de compatibilité sémantique plutôt qu'un appel de solveur.

La cinquième couche est un plan de formalisme. Le plan n'est pas une prose de forme libre écrite par un agent. Il s'agit d'une décision de compilation dactylographiée qui indique quelle question sera posée quel moteur, quelles sont les observations, ce qui garantit les retours du moteur et comment ses résultats correspondent au vocabulaire des résultats canoniques. Le choix doit être motivé par la forme sémantique plutôt que par la mode.

L'évaluation relationnelle est le premier moteur à mettre en œuvre parce que de nombreuses vérifications de documents utiles réduisent les jointures, les comparaisons d'ordres, les regroupements et les différences. Des règles telles que «chaque terme technique défini est utilisé de manière cohérente», «chaque réclamation acceptée a au moins une source», «aucun identifiant d'exigence n'est dupliqué» ou «tous les risques énumérés ont un propriétaire d'atténuation» n'exigent pas de vérificateur de théorème. Un noyau relationnel est transparent, efficace et facile à tester. Datalog étend ce noyau lorsque des dépendances transitoires ou récursives sont requises. Les exemples incluent l'héritage, la propagation entière, les chaînes de dépendance et l'accessibilité par le biais de graphiques de citation ou d'exigence.

Les tableaux de décision sont le deuxième formalisme utile pour les politiques limitées. Un tableau de décision rend explicites les combinaisons de conditions et les résultats, expose les lignes qui se chevauchent ou manquent et prend en charge l'analyse de la couverture. Il est particulièrement approprié pour les règles d'admissibilité, d'acheminement, de classification et de politique dont la complexité provient des combinaisons plutôt que de la récursion. Un circuit peut compiler une politique en langage naturel dans une table candidate, mais la publication devrait nécessiter des exemples au niveau de la ligne, une détection de chevauchement, des vérifications d'exhaustivité et l'approbation humaine de la partition sémantique.

Les moteurs SAT, SMT, CSP et MaxSAT deviennent utiles lorsque la tâche contient des contraintes en interaction, de l'arithmétique, des horaires, des affectations ou de l'optimisation. Le compilateur doit choisir les théories prises en charge et éviter de cacher la sémantique non prise en charge dans des chaînes non interprétées. Un résultat SMT peut prouver l'infaisabilité sous des contraintes codées, produire un modèle satisfaisant ou exposer un noyau insatisfaisable. Le noyau devrait être cartographié aux obligations et aux hypothèses de la source. Ce mappage est plus instructif que de demander à un LLM d'expliquer une erreur de solveur générique; il identifie les plus petits engagements formels qui ne peuvent pas tenir conjointement.

La logique temporelle et la vérification du modèle sont appropriées pour les procédures, les flux de travail, les protocoles et les règles du cycle de vie. Le graphique de preuve fournit des événements et des propositions d'état; le compilateur construit un système de transition et une propriété temporelle; le vérificateur renvoie la satisfaction, une trace de contre-exemple ou un résultat non concluant sous des limites. La partie difficile n'est pas le vérificateur temporel. Il décide si la source définit un système de transition complète, si les transitions omises sont impossibles ou simplement sans papiers, et si les hypothèses d'équité ou d'environnement sont justifiées. Le résultat canonique doit donc exposer la limite du modèle.

L'exécution symbolique est appropriée lorsqu'une règle fait référence à des chemins réalisables par le biais de code exécutable, d'une procédure étatique ou d'un artefact sémantique de type programme. Son résultat devrait être un témoin concret: affectations, conditions de la branche, transitions d'état et liens sources. Dans les paramètres du document uniquement, l'exécution symbolique peut explorer d'autres liaisons ou branches procédurales, mais elle ne doit pas être invoquée simplement parce que le système contient des symboles. Le terme devrait conserver sa signification sensible au chemin. SymGPT est instructif parce que la règle du langage naturel est compilée dans une condition de violation, puis testée contre les chemins de contrat réels plutôt que contre plus de prose générée [Xia et al. 2026].

La logique naturelle, les automates finis, l'analyse des graphiques, le calcul de semi-ring et les preuves de théorème spécialisé appartiennent au portefeuille lorsque leurs hypothèses structurelles correspondent. La logique naturelle peut gérer efficacement la monotonie et certaines implications lexicales près du langage de surface. Les automates sont excellents pour les modèles locaux commandés. Les méthodes de graphique et de semi-ring peuvent maintenir plusieurs analyses avec un calcul partagé. Les assistants de preuve sont appropriés lorsqu'un petit nombre d'allégations de grande valeur justifient une formalisation manuelle et des preuves vérifiées par le noyau. L'architecture devrait faire de l'ajout d'un moteur une extension typée, et non une invitation à intégrer du code exécutable arbitraire dans des circuits.

La sixième couche est un raffinement sémantique à contre-exemple. Une constatation peut être compilée dans un problème de faisabilité. Si le solveur renvoie un témoin, l'exécution le

rejoue par rapport au graphique de preuve et signale les choix d'interprétation exacts requis. Si le chemin est infaisable, le moteur retourne un noyau insatisfaisant, une condition préalable échouée ou un jeu de conflits. Le système trace ce qui remonte à travers la compilation aux observations sources et aux cartographies candidates qui l'ont produit. Ce n'est qu'alors qu'un LLM est demandé pour aider.

L'invite de réparation doit être locale et dactylographiée. Il peut se demander si deux mentions ont été incorrectement unifiées, si une condition a été conçue comme nécessaire ou suffisante, si une exception s'applique, ou si une relation temporelle a le mauvais critère d'évaluation. Il devrait présenter les travées de source pertinentes, les contrats d'ontologie, les alternatives actuelles et le conflit symbolique. Le LLM propose un ou plusieurs artefacts sémantiques révisés. Ces propositions passent par les mêmes vérifications de l'échographie et de la compilation que les originaux. Un humain peut être inséré lorsque l'ambiguïté est normativement ou scientifiquement matérielle.

Ce processus est fondamentalement différent de l'auto-réflexion générique. On ne dit pas au modèle que sa réponse était erronée et invitée à improviser une autre chaîne de pensée. Il reçoit un conflit mécaniquement identifié dans un artefact sémantique persistant. La réparation peut être testée en régression. Si la réparation change une ontologie ou un producteur, la publication nécessite de nouveaux cas de référence et un diff sémantique. Le contre-exemple devient une partie permanente du corpus d'évaluation.

La septième couche est le résultat canonique. Il devrait s'agir d'un produit stable et indépendant du solveur qui sépare le verdict, les preuves, les hypothèses, les interprétations, l'exécution et les limitations. Pour une règle d'audit, il enregistre la version de la règle, la portée applicable, l'état de compatibilité, la couverture, les observations acceptées, les observations rejetées, le résultat d'exécution, la référence de témoin ou de preuve, le résultat du vérificateur et les questions non résolues. Pour un plan, il enregistre les objectifs, les contraintes, les étapes, les dépendances, les ressources, les alternatives et les vérifications exactes que le plan satisfait. Le produit canonique est l'autorité pour toute réponse ultérieure en langage naturel.

Le chemin de retour vers le langage doit être traité comme une compilation, pas comme une génération libre. Un remodeur reçoit un ensemble fini de réclamations sous licence. Chaque revendication a modalité, polarité, quantité, références d'entité, incertitude et provenance. Le rendeur peut choisir l'ordre de discours et le phrasé explicatif, mais il ne peut pas introduire de nouvelles entités, se renforcer peut à doit, convertir les preuves manquantes en absence, supprimer une hypothèse matérielle ou inventer une explication causale. Les identifiants de réclamation au niveau de la peine doivent rester disponibles pour vérification. Un deuxième vérificateur devrait comparer le texte rendu avec le résultat canonique, en utilisant des tests déterministes pour les noms, les quantités, les dates, la négation et la modalité et, le cas échéant, un modèle distinct limité à l'implication textuelle plutôt que l'évaluation ouverte.

La réalisation à contre-courant est particulièrement importante car les utilisateurs jugeront le système par sa prose. Une trace symbolique parfaite peut être mise à mal par un résumé élégant mais inexact. L'architecture devrait donc rendre la verbosité optionnelle mais la fidélité obligatoire. En mode haute assurance, le système peut générer un langage clair et répétitif car chaque phrase suit un modèle sous licence par le type de résultat. En mode explicatif, un LLM peut améliorer la lisibilité, mais les phrases non prises en charge sont rejetées et régénérées. L'artefact canonique, et non la prose, reste le compte rendu de la décision.

La sécurité et la gouvernance doivent être intégrées dès le début. Les documents en langage naturel sont des entrées non fiables et peuvent contenir des injections rapides, des exemples contradictoires, un formatage trompeur ou des fragments de code. Les producteurs sémantiques devraient recevoir des données sources par le biais d'interfaces typées plutôt que d'une invite de système privilégiée qui peut être écrasée par le contenu des documents. Les artefacts formels générés doivent être analysés par des grammaires restrictives. Les solveurs devraient fonctionner sous des limites de ressources. Les extensions d'exécution doivent être revues, versionnées et digestées. Les espaces de travail d'apprentissage ne doivent pas être publiés directement sur la production. Ces limites sont déjà partiellement présentes dans nllAgent et devraient être renforcées plutôt que contournées pour plus de commodité [AssistOS-AI 2026].

Un programme de mise en œuvre réaliste peut se dérouler dans cinq communiqués. La première version devrait prendre en charge l'ingestion immuable de Markdown ou ZX20000QXZ, les portées de source, les observations structurelles, un moteur de requête relationnel, les observations contrôlées-anglaises restreintes, les états de couverture, les circuits réutilisables, la vérification de la relecture et les résultats canoniques. Il devrait résoudre des tâches limitées telles que l'exhaustivité de la provenance, la cohérence de la définition, la couverture des exigences, les règles de section ordonnées et les distinctions modales simples. Aucune identité générale, inférence temporelle ou récupération du monde actuel ne devrait être revendiquée.

La deuxième version devrait ajouter l'identité et l'heure typées. L'identité commence par des identifiants explicites, des alias et des rôles déclarés par document; la sémantique temporelle commence par des dates exactes, des intervalles, un ordre de document et un petit ensemble de modèles d'ordre d'événement. Les repères devraient contenir des alias ambigus, des rôles changeants, des intervalles qui se chevauchent et des liens délibérément manquants. Le système devrait s'arrêter plutôt que de deviner quand un circuit nécessite une identité plus forte que la version ne le fournit.

La troisième version devrait ajouter des tableaux de décision et l'exécution de SMT/CSP, avec une cartographie insatisfaisable et un raffinement sémantique guidé par contre-exemple. Ce communiqué peut porter sur les politiques d'admissibilité, les calendriers, les contraintes d'affectation, la cohérence quantitative et la faisabilité de la procédure limitée. Chaque résultat du solveur doit être rejoué par un vérificateur indépendant ou un damier plus petit lorsque cela est possible. Les tests de mutation devraient modifier les contraintes et les observations pour démontrer que les suites de référence détectent les régressions sémantiques.

La quatrième version devrait ajouter un adaptateur de vérification temporelle/modèle et un adaptateur d'exécution symbolique sensible au chemin pour un domaine en béton, tel que les spécifications de flux de travail ou les règles de contrat intelligent. L'objectif n'est pas une large couverture, mais la preuve que le registre du formalisme et l'interface de certificat canonique soutiennent une sémantique vraiment différente. Les tests de cohérence du formalisme croisé devraient vérifier que les faits partagés conservent l'identité, le temps, la modalité et la provenance entre les diminutions.

La cinquième version devrait ajouter la planification et la réalisation contrôlée. Une idée est compilée dans un plan canonique, vérifiée par rapport aux contraintes formelles, réalisée en prose, et audité par le même système de règles. L'évaluation doit distinguer la validité du plan, la fidélité à la réalisation, la qualité linguistique et l'efficacité de la réparation. Le plan ne devient

pas digne de confiance simplement parce que la prose a été générée à partir d'un objet structuré; il devient plus auditable parce que l'objet et ses vérifications sont persistants.

Artifact stack for a replayable executable-language runtime

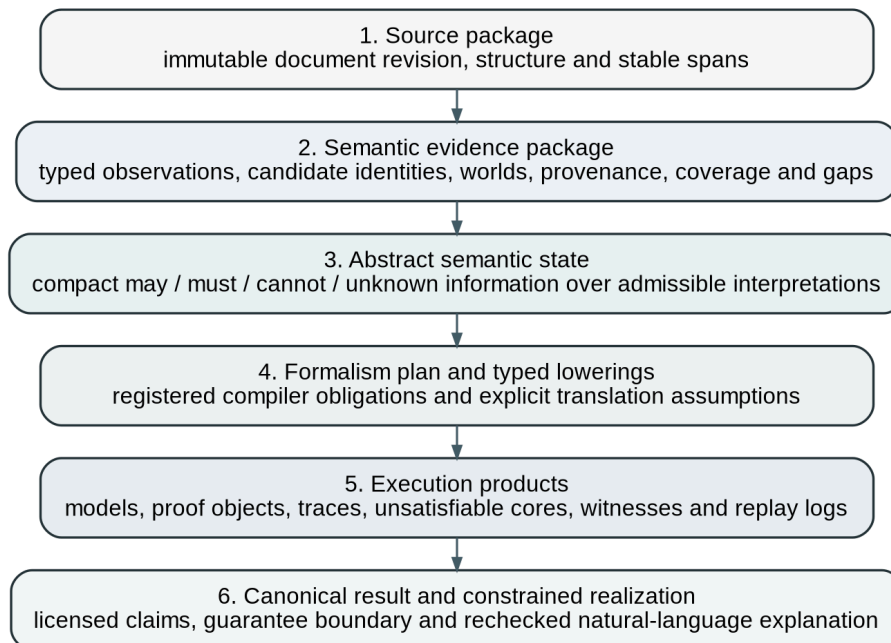


Figure 4. Pile d'artefact pour un runtime exécutable-naturel rejouable. Chaque couche est versionnée, mappée à la source et inspectable de manière indépendante.

Cette mise en œuvre mise en scène incarne un principe méthodologique important: l'ambition sémantique ne devrait croître que lorsque le système peut détecter que sa représentation antérieure est inadéquate. Le runtime ne devrait pas cacher le langage non pris en charge derrière la confiance du modèle. Il devrait exposer les lacunes de compatibilité comme des résultats de première classe et transformer les lacunes récurrentes en nouveaux producteurs, ontologie, moteur et travail de référence. Le langage naturel exécutable progresse par l'expansion gouvernée d'une enveloppe de solidité, et non par un saut à la compréhension universelle.

8.2 Questions de recherche, repères et programme de publication

L'architecture ne devient une contribution scientifique que lorsque ses revendications sont réfutables. Une démonstration dans laquelle un LLM génère un artefact formel plausible et un solveur renvoie une réponse est insuffisante. Le même modèle peut générer la référence, la traduction et l'explication, créant une boucle fermée d'erreur corrélée. Une évaluation sérieuse doit décomposer le pipeline et tester chaque obligation de manière indépendante. Il doit également inclure des conditions de défaillance dans lesquelles l'approche proposée devrait être rejetée ou réduite.

La première question de recherche concerne la représentation: le maintien de plusieurs interprétations sémantiques admissibles améliore-t-elle la fiabilité par rapport à la sélection d'une traduction? L'hypothèse correspondante est que la sémantique candidate, combinée à l'analyse *may/must*, réduit la fausse assurance sur les entrées ambiguës et sous-spécifiées. Il peut réduire le taux de réponse apparente parce que plus de cas deviennent conditionnels ou inconnus. Ce n'est pas forcément un défaut. Les mesures de comparaison pertinentes pour savoir si les conclusions acceptées doivent être plus souvent correctes et si les constatations peuvent saisir des solutions de rechange sensiblement plausibles sans écrasants examinateurs.

Un point de référence pour cette question devrait contenir des textes avec des ensembles d'interprétation annotés par des experts plutôt qu'une forme logique en or. Les paires minimales devraient varier la portée du quantificateur, la négation, l'attachement d'exception, la coordination, la coréférence, les limites temporelles, la force modale et la juridiction. Certains éléments devraient avoir une lecture clairement voulue, quelques lectures légitimes et quelques preuves insuffisantes pour toute réponse déterminée. Les systèmes sont évalués sur la couverture des lectures admissibles, l'exclusion des lectures inadmissibles, l'étalonnage du statut de *may/must* et les verdicts en aval. La correspondance exacte des graphes est trop fragile; l'équivalence sémantique doit être testée par le biais de suites de requêtes ou d'une implication bidirectionnelle sous l'ontologie de référence.

La deuxième question de recherche concerne le raffinement: les conflits dérivés du solveur sont-ils plus utiles pour réparer la formalisation que le modèle générique autocritique? L'hypothèse est que les contre-exemples, les noyaux insatisfaits et les obligations de preuve ratées produisent des réparations plus localisées, stables et plus efficaces par l'échantillon parce qu'ils identifient la dépendance responsable du résultat. Une expérience contrôlée devrait comparer quatre conditions: réessai direct, auto-réflexion verbale, rétroaction d'exécution-erreur et raffinement sémantique à contre-exemple avec cartographie de la source. Le même modèle, la même température et le même budget doivent être utilisés lorsque cela est possible.

La qualité de réparation doit être mesurée non seulement par la question de savoir si l'exemple cible devient correct. La réparation doit préserver les cas non liés, réduire la différence sémantique et éviter d'introduire des hypothèses non étayées. Les suites de régression basées sur la mutation sont essentielles. Une réparation qui résout un exemple en affaiblissant la règle à l'échelle mondiale n'est pas un succès. L'expérience devrait enregistrer le nombre d'itérations, de jetons, d'interventions humaines, de mutants survivants, d'observations modifiées et de savoir si l'artefact final était rejouable indépendamment.

La troisième question de recherche concerne le contrat de preuve. Une séparation de la source, des observations, des affirmations d'ontologie, des axiomes de domaine et des locaux

de connaissance du monde améliore-t-elle le transfert et l'auditabilité par rapport à un graphique de connaissances plats ou à une chaîne de pensée libre? L'hypothèse est que les contrats typés réduisent l'introduction silencieuse de prémisses et rendent la réutilisation inter-domaines plus prévisible. Le coût probable est la création de frais généraux et d'autres courses arrêtées.

L'indice de référence devrait inclure la même règle appliquée à travers les documents de plusieurs domaines, avec une variation contrôlée du vocabulaire et de l'ontologie. Par exemple, une exigence générale selon laquelle chaque revendication acceptée a des preuves traçables peut être mise à l'essai sur des documents de recherche, des documents de politique, des spécifications techniques et des plans. Les observations structurelles peuvent être transférées directement; les producteurs d'entités et de relations spécifiques à un domaine ne peuvent pas le faire. L'évaluation enregistre les vérifications de compatibilité qui s'arrêtent, si la raison d'arrêt est correcte et combien de nouveau matériel sémantique est nécessaire pour reprendre l'exécution. Le transfert réussi n'est pas simplement obtenir une réponse; il reconnaît avec précision ce qui peut et ne peut pas être réutilisé.

La quatrième question de recherche concerne le choix du formalisme. Le routage vers un portefeuille de moteurs spécialisés surpasse-t-il un langage intermédiaire universel sous une surveillance sémantique égale? Beiser et Penz montrent que le choix de langage intermédiaire peut modifier la précision globale et d'exécution de manière substantielle sur les tâches de raisonnement neuro-symbolique [Beiser and Penz 2025]. L'hypothèse ici est plus spécifique: un routeur dactylographié qui sélectionne le formalisme adéquat le plus simple améliorera le succès de compilation, la qualité d'explication et l'efficacité sans sacrifier la cohérence des tâches croisées.

Une comparaison significative ne devrait pas permettre au système de portefeuille de disposer d'informations plus cachées. Chaque tâche reçoit les mêmes preuves et ontologies sources. La base de référence du formalisme unique peut utiliser la logique du premier ordre, ASP, SMT, ou un langage général de requête graphique. Le système de portefeuille peut choisir des requêtes relationnelles, Datalog, des tables de décision, SMT, une logique temporelle ou une exécution symbolique. Les métriques comprennent la validité de la formalisation, l'équivalence sémantique, le succès du solveur, l'exécution, la qualité des témoins et le coût d'entretien. Les cas de formalisme croisé devraient nécessiter deux moteurs et vérifier si les symboles partagés restent cohérents.

L'expérience devrait également chercher des contre-preuves. Une représentation universelle peut surpasser le portefeuille dans des domaines étroits parce que le routage introduit des erreurs et plusieurs compilateurs augmentent la base de confiance. La conclusion attendue n'est pas que les portefeuilles sont toujours supérieurs. C'est que la pluralité du formalisme n'est justifiée que lorsque la distribution de la tâche contient véritablement des structures de calcul différentes et lorsque les interfaces préservent la sémantique.

La cinquième question de recherche concerne la couche linguistique finale. La réalisation à contre-courant réduit-elle les allégations non étayées ou renforcées par la sémantique tout en préservant une lisibilité acceptable? L'hypothèse est qu'un résultat canonique plus une génération sous licence d'allégations abaisse considérablement l'hallucination et la dérive de la modalité par rapport à la génération de réponse directe ou à la génération de réponse à partir d'une trace de solveur brut. Le coût peut être moins de prose naturelle et plus de régénération.

L'indice de référence devrait inclure des résultats canoniques avec une incertitude subtile, des preuves contradictoires, des conditions imbriquées, des quantités et des dates. Les évaluateurs humains jugent la fidélité factuelle, l'exhaustivité des limitations matérielles, la lisibilité et l'utilité. Les vérificateurs déterministes mesurent l'entité, le nombre, la date, la polarité et la préservation de la modalité. Un ensemble contradictoire devrait tenter le rendu de produire une conclusion plus forte et plus simple que le résultat supporte. La métrique centrale n'est pas une préférence stylistique mais la vitesse à laquelle la prose dépasse l'autorité du produit symbolique.

La sixième question de recherche concerne l'architecture fédérative elle-même. La revue systématique 2026 de NLP neuro-symbolique signale des gains prometteurs pour les systèmes fédératifs, mais prévient que les preuves sont confondues par les différences de tâches [Chatzikyriakidis and Lappin 2026]. Une expérience directe devrait comparer un pipeline fédérateur avec une alternative injective ou apprise de bout en bout sur la même échelle de tâches, de données et de modèles. Les avantages pertinents peuvent apparaître moins dans l'exactitude brute que dans l'auditabilité, l'abstention, la localisation des erreurs et la réparation.

L'étude devrait donc faire état de plusieurs dimensions. La précision de la tâche brute reste nécessaire. Il devrait être accompagné d'une précision de forme sémantique, d'une précision conditionnée par le solveur, d'un étalonnage inconnu, d'une reproductibilité entre les versions du modèle, d'une fidélité d'explication, d'un temps de révision humain et de la fraction d'erreurs attribuables à l'extraction, à la mise à la terre, à la compilation, à l'exécution ou à la réalisation. Un système modulaire qui correspond plutôt que bat la précision de bout en bout peut toujours être préférable pour un travail d'assurance élevée s'il transforme les erreurs silencieuses en artefacts diagnostiques. À l'inverse, la modularité ne doit pas être défendue comme une vertu intrinsèque si elle ne fait qu'ajouter de la bureaucratie sans améliorer les preuves.

Une septième question de recherche concerne l'interprétation abstraite par rapport aux alternatives sémantiques. Peut-on définir un domaine abstrait utile qui est conservateur par rapport à un ensemble d'interprétation de référence et plus évolutif que l'énumération explicite? Les premières expériences peuvent utiliser des ensembles finis comme référence concrète. Les domaines abstraits pour l'identité, les intervalles temporels, la modalité, les quantités et la couverture sont mis en œuvre séparément, puis combinés. La solidité est vérifiée mécaniquement contre l'énumération sur les petits boîtiers. La précision est mesurée par le taux de constatations de may qui peuvent être éliminées par la recherche ou le raffinement des témoins.

Ce programme doit être prudent avec le mot sonorité. Si l'annotation d'expert définit un ensemble d'interprétation finie, l'interpréteur abstrait peut être sonore par rapport à cet ensemble. Il ne peut donc pas prouver que l'ensemble épuise le sens humain. La contribution est une sémantique conditionnelle exécutable et des preuves de son utilité. Un article théorique ultérieur peut définir la relation d'admissibilité, les fonctions d'abstraction et de concrétisation, les conditions de transfert et les opérateurs de raffinement. Le système empirique ne devrait pas attendre une sémantique complète du langage, mais il devrait rendre explicite la nature conditionnelle de ses revendications.

Une huitième question de recherche concerne l'exécution symbolique sur les programmes sémantiques. La construction de témoins sensibles au chemin améliore-t-elle la compréhension de l'examineur et la découverte de défauts par rapport aux résultats des règles statiques? Dans un domaine de flux de travail ou de stratégie, l'exécution peut comparer un évaluateur de

règles simple avec l'exploration symbolique du chemin. L'évaluateur dit qu'une violation est possible; l'exécuteur testamentaire retourne une séquence concrète d'événements et d'affectations. Les examinateurs humains jugent si le témoin est intelligible et utile. Les défauts de vérité au sol et les mutants ensemencés déterminent si le témoin correspond à un cas réel.

L'explosion de la voie doit faire partie de l'évaluation plutôt que de l'être comme un inconvénient de mise en œuvre. L'étude devrait varier la ramification, les limites de boucle, les alternatives identitaires et les choix temporels. Le guidage LLM peut prioriser les chemins, comme le suggère les travaux d'exécution symbolique guidés par LLM, mais le moteur doit distinguer la couverture heuristique de l'exhaustivité formelle. Les mesures de couverture devraient rapporter les états explorés, les états élagués, les délais de résolution et le comportement externe non pris en charge. Un témoin est une preuve solide de l'existence; le fait de ne pas en trouver une n'est pas une preuve d'absence à moins que la borne d'exploration et le modèle ne justifient cette conclusion.

L'architecture de référence doit être superposée. La couche source teste l'ingestion et l'ancrage. La couche d'observation teste l'extraction et l'état. La couche de mise à la terre teste l'identité, le temps, la modalité, les quantités et l'autorité. La couche de formalisation teste l'équivalence sémantique et la validité de la langue cible. La couche d'exécution teste les sorties et les certificats du solveur. La couche de résultat teste la composition du verdict et l'incertitude. La couche de réalisation teste la fidélité des réponses en langage naturel. Chaque exemple doit identifier la couche la plus précoce à laquelle un défaut injecté doit être détecté.

Les cas de référence devraient être suffisamment petits pour être validés manuellement et composables en documents plus longs. De petits exemples fournissent une vérité fondamentale digne de confiance pour une logique subtile. Les composites longs testent la récupération, l'indexation, la continuité d'identité et l'évolutivité. La même règle sous-jacente peut apparaître dans plusieurs formes linguistiques et structures de documents. Les familles de paraphrase testent l'invariance; les ensembles de contraste changent une caractéristique sémantique et la sensibilité du test. Le point de référence doit éviter la défaillance commune dans laquelle un système fonctionne bien en mémorisant des motifs de surface tout en ignorant la distinction prévue.

Les tests de mutation doivent être centraux. Les mutations peuvent changer un quantificateur, annuler une condition, déplacer une exception, modifier une limite de date, fusionner des identités, supprimer des preuves, modifier la force modale, échanger l'acteur et l'objet, dupliquer une exigence ou modifier un seuil de circuit. Une suite robuste devrait détecter le changement attendu et laisser les résultats non liés stables. Le score de mutation n'est pas une mesure de qualité complète, mais il révèle des tests qui ne font que confirmer un chemin heureux. Pour nllAgent, les mutations exécutables permettraient de combler directement l'une des lacunes reconnues du dépôt [AssistOS-AI 2026].

L'évaluation devrait inclure une injection rapide et des adversaires sémantiques. Les documents peuvent contenir des instructions dirigées vers le modèle, des en-têtes trompeurs, un texte de politique cité, un Unicode contradictoire, un contenu caché ou des déclarations qui ressemblent à des déclarations d'ontologie. L'analyseur source doit les conserver en tant que données. Les producteurs sémantiques devraient fonctionner sous l'autorité du système fixe. L'indice de référence devrait vérifier si le texte du document peut modifier le circuit, supprimer les résultats ou provoquer des appels d'outils privilégiés. Les résultats en matière de sécurité doivent être consignés séparément de l'exactitude sémantique.

L'évaluation en direct doit être suffisamment reproductible pour soutenir les affirmations scientifiques. Chaque exécution devrait épingle le fournisseur, l'identifiant du modèle, demander le schéma, la température, les semences lorsqu'elles sont prises en charge, la construction du contexte et la politique de réessai. Les entrées et les extraits doivent être saisis sous réserve de contraintes de confidentialité. Les essais répétés estiment la variance. Les doubles déterministes restent utiles pour les tests logiciels, mais ne peuvent pas établir la qualité de la traduction. Au moins deux familles de modèles devraient être testées lorsque cela est possible pour distinguer les effets architecturaux du comportement d'un fournisseur.

L'évaluation humaine est nécessaire, mais doit être conçue de manière étroite. Il ne faut pas demander aux experts si une réponse «semble bonne». Ils devraient juger des obligations spécifiques: si la formalisation préserve la source, si les identités sont justifiées, si un témoin est faisable, si l'explication énonce toutes les hypothèses matérielles et si une inconnue est correctement retournée. Le désaccord inter-évaluateurs est lui-même une preuve d'ambiguïté sémantique et ne devrait pas être forcé dans une seule étiquette d'or sans analyse.

Les métriques quantitatives de base devraient inclure la précision de l'ancrage source; la précision de l'observation et le rappel par statut; l'ontologie et la validité du type; la précision, le rappel et l'étalonnage de lien d'identité; la précision de la relation temporelle et modale; la couverture de la réponse des candidats; la prudence des candidats; la solidité peut/doit être par rapport aux interprétations annotées; la syntaxe de formalisation et l'équivalence sémantique; la réussite du témoin; le succès de vérification; le faux taux d'assurance; la latence; la latence; la latence; Un seul score global masquerait le mécanisme et devrait être secondaire.

Une fausse assurance mérite une attention particulière. Il mesure les cas dans lesquels le système présente une conclusion déterminée, bien que la source, l'ancrage ou l'interprétation ne le justifie pas. Dans les demandes d'assurance élevée, réduire la fausse assurance peut avoir plus d'importance que de maximiser les questions de réponse. Le système devrait recevoir un crédit pour un état arrêté précis lorsqu'un LLM direct invente en toute confiance un pont. Une inconnue appropriée est un résultat positif.

Les lignes de base doivent inclure la réponse directe LLM, l'invite de raisonnement de la chaîne de pensée ou équivalente, l'exécution LLM augmentée par outil sans artefacts formels persistants, la traduction monoformalisme, la traduction de l'ensemble des candidats sans interprétation abstraite, l'exécution symbolique sans raffinement sémantique et l'architecture complète. Pour l'étude de cas nllAgent, la version actuelle de la fondation bornée peut servir de base de référence mise en œuvre, tandis que les extensions d'identité, de sémantique abstraite et de multiformalisme proposées sont évaluées de manière progressive. Cela évite de comparer une vision de recherche complète uniquement à une vision externe plus faible.

Le programme de publication peut être organisé autour de cinq contributions plutôt que d'une réclamation surdimensionnée. Le premier article devrait être un document de position et d'architecture qui définit le langage naturel exécutable, la limite de confiance, l'écart de formalisation, l'enveloppe de solidité et la conception multi-formalisme. Sa contribution est la clarté conceptuelle et une architecture de référence, soutenue par une étude de synthèse systématique et un petit démonstrateur de travail.

Le deuxième article devrait formaliser le pivot sémantique et la solidité conditionnelle de la provenance. Il définit les objets sources, les observations typées, les interprétations de candidats, la sémantique may/must et la compatibilité. Il prouve la solidité du calcul abstrait par rapport à un ensemble d'interprétations limitées admissibles et rapporte des expériences sur

l'identité, le temps, la modalité et la couverture. Il s'agit du noyau des langues de programmation et de la formation-sémantique.

Le troisième document devrait introduire un raffinement sémantique à contre-exemple. Il définit les cartographies des conflits de solveurs aux engagements sémantiques liés à la source et compare le raffinement avec l'auto-réflexion et la rétroaction d'exécution générique. Le résultat le plus fort serait une fausse assurance plus faible, des réparations plus petites, une meilleure préservation de la régression et une réduction de l'effort humain. Même un résultat négatif serait précieux s'il identifie la rétroaction du solveur trop faible pour guider la réparation sémantique.

Le quatrième document devrait présenter le portefeuille de formalisme et les contrats de moteurs typés. Il évalue les adaptateurs relationnels, Datalog, décision, SMT, adaptateurs d'exécution temporelle et symbolique sur un modèle d'artefact commun. Le document devrait mettre l'accent sur la conservation sémantique et les certificats plutôt que sur le nombre d'intégrations. Un petit ensemble de moteurs vraiment différents est plus convaincant que de nombreuses emballages peu profonds.

Le cinquième document devrait être le système nllAgent et le document d'évaluation. Il devrait distinguer clairement la version mise en œuvre de l'exécution plus large. Il peut documenter les révisions de source immuables, LongTextJS, CircuitJS, les observations axées sur la demande, la compatibilité, la couverture, l'exécution restreinte, la gouvernance de l'apprentissage et de la publication, la vérification de la relecture, les produits d'audit et de plan canoniques, et le processus de problèmes graves. L'évaluation devrait inclure la mutation exécutable, les dossiers de modèle en direct, les cas d'identité, la gouvernance du monde actuel, les repères de planification et un vérificateur indépendant ou mis en œuvre séparément lorsque cela est possible.

Un sixième article ultérieur pourrait étudier la réalisation à l'épreuve. Il comparerait la génération directe, le rendu de modèle, le rendu LLM contraint et la vérification post-hoc. La contribution centrale serait une représentation et une preuve de licence de revendication que le langage lisible peut être généré sans modifier la modalité, l'incertitude ou le contenu factuel. Ce travail relie l'exécution formelle au besoin pratique d'une communication humaine accessible.

Le projet devrait définir des critères de défaillance explicites. L'approche de l'ensemble des candidats devrait être réexaminée si elle produit tant d'alternatives qui peuvent/doivent être des résultats peu instructifs et que le raffinement ne peut pas retrouver la précision. Le portefeuille de formalisme devrait être réduit si le routage et les bugs de formalisme croisé dépassent les avantages de la spécialisation. La formalisation assistée par LLM devrait être limitée à l'appui à la création si l'équivalence sémantique reste médiocre dans les documents réalistes. La réalisation à contre-courant devrait retomber en arrière sur des modèles si la prose générée par le modèle dépasse à plusieurs reprises sa licence. Le raisonnement actuel du monde devrait rester en dehors du noyau de confiance si les contrôles de source, de temps, de juridiction et de conflit ne peuvent pas être appliqués.

L'honnêteté scientifique nécessite également une comparaison avec des alternatives plus simples. De nombreuses vérifications de documents peuvent être résolues par des analyseurs déterministes, des expressions régulières, la validation de schémas, des requêtes de base de données ou des règles d'auteurs humains. Un LLM est justifié lorsque la variation de langue, la cartographie ontologique ou la proposition sémantique réduisent matériellement le coût de création ou augmente la couverture. Un noyau symbolique est justifié lorsque l'état exact, le

replay, les témoins ou la question de vérification indépendante. Un document de recherche devrait rapporter quand un système plus simple fonctionne aussi bien. Le langage naturel exécutable n'est pas une raison pour formaliser chaque préférence éditoriale ou insérer un solveur dans des tâches qui n'ont besoin que d'un analyseur fiable.

Le résultat pratique probable n'est pas une machine qui exécute tout le langage naturel. Il s'agit d'une famille de compilateurs sémantiques gouvernés pour les domaines de documents et d'agents, partageant la source, l'ancrage, l'exécution et l'infrastructure de certificat. Chaque compilateur possède une enveloppe sémantique déclarée. LLMs rendent ces compilateurs plus faciles à écrire, adapter et expliquer. Les méthodes formelles rendent leurs résultats acceptés reproductibles et contestables. La combinaison est utile précisément parce qu'aucun composant n'est demandé pour fournir ce qu'il ne peut pas fournir de manière fiable seul.

Le programme de recherche est donc à la fois plus modeste et plus conséquent que le slogan «le langage naturel comme code». Le langage naturel reste un moyen ouvert, social, dépendant du contexte. L'exécution crée des projections exécutables de celui-ci pour les questions déclarées. Ces projections peuvent être suffisamment rigoureuses pour appuyer des audits, des plans, des protocoles et des vérifications scientifiques réels, à condition que le système enregistre la distance entre la source et l'objet formel au lieu de prétendre que la distance a disparu.

Conclusion : de la forme probabiliste à la conséquence symbolique

La renaissance récente de AI neuro-symbolique a produit un motif architectural reconnaissable. De grands modèles de langage sont utilisés où l'espace de recherche est linguistique, hétérogène ou difficile à énumérer. Les systèmes formels sont utilisés lorsque l'état exact, la cohérence, la recherche à destination exhaustive, les témoins ou les preuves sont importants. La question intéressante n'est plus de savoir si les méthodes neuronales et symboliques peuvent être combinées. Ils le peuvent clairement. La question importante est de savoir où se trouve l'autorité et ce qui est vérifié de manière indépendante.

L'enquête montre que les systèmes les plus crédibles n'accordent pas l'autorité finale du modèle de langue sur le résultat. Les langues contrôlées traduisent une surface restreinte en sémantique formelle. Les analyseurs sémantiques génèrent des requêtes ou des programmes exécutables. PAL délègue le calcul à Python. SatLM délègue des contraintes à un solveur. Logic-LM et LINC délèguent la déduction aux moteurs logiques. NL2TL et nl2spec traduisent les exigences aux spécifications temporelles. Les systèmes de planification utilisent des modèles formels et des outils de vérification pour maintenir la cohérence à long horizon. SymGPT compile les règles de langage naturel ERC en contraintes et utilise l'exécution symbolique pour construire des chemins de contrat concrets. SAIL utilise LLMs pour rechercher des transformateurs abstraits, mais admet seulement les candidats qui remplissent les obligations mécaniques. Ces systèmes diffèrent grandement, mais ils convergent vers le principe que la génération propose et un mécanisme indépendant décide.

Cette convergence ne doit pas être surestimée. L'utilisation de solveur ne vérifie pas la compréhension. L'écart de formalisation reste le problème central. Un théorème peut tirer une conclusion valable des prédicats qui faussent le texte. Un programme peut renvoyer la réponse attendue par un chemin fallacieux. Un modèle SMT peut satisfaire les contraintes qui omettent l'exigence décisive de l'utilisateur. Un exécuteur testamentaire symbolique peut exposer un véritable chemin logiciel pour une règle mal traduite. Les systèmes actuels les plus solides réduisent l'erreur déductive et de calcul tout en délocalisant le risque de parsing sémantique, de mise à la terre, d'ontologie et de portée.

La réponse appropriée n'est pas d'abandonner les outils formels parce que la traduction est imparfaite. C'est pour faire de la traduction un artefact persistant, dactylographié et révisable. La source doit rester disponible. Chaque observation sémantique doit identifier son producteur, ses preuves, son statut et son ontologie. L'identité, le temps, la modalité, les quantités et l'autorité doivent être représentés plutôt que de se faire subir à nouveau pendant chaque réponse. Les interprétations alternatives doivent rester des alternatives lorsque la distinction affecte le résultat. Un moteur symbolique ne devrait exécuter que des objets à la terre qui répondent à des contrats explicites.

L'interprétation abstraite fournit une théorie utile pour le côté global de cette architecture. Il demande ce qui peut être conclu de manière conservatrice à partir d'un grand nombre d'États possibles. Appliqués au langage naturel, les États concrets ne sont pas des «significations» indéfinies, mais des structures sémantiques bornées admissibles en vertu d'une politique de grammaire, d'ontologie, de révision de source et de mise à la terre déclarée. Un état abstrait peut alors représenter ce qui doit tenir, tenir, ne peut pas tenir, les conflits ou reste inconnu. La

garantie est conditionnelle à l'ensemble d'interprétation, mais la solidité conditionnelle est toujours significative lorsque les conditions sont explicites et testées.

L'exécution symbolique fournit le complément local. Lorsqu'une analyse abstraite signale une éventuelle violation, l'exécution symbolique ou un autre moteur de contrainte peut rechercher une interprétation et un chemin concrets. Une recherche réussie renvoie un témoin qu'un examinateur peut inspecter. Une recherche ratée expose une abstraction trop grossière ou un choix sémantique incompatible. La cartographie de l'échec à la portée de la source crée un raffinement sémantique guidé par contre-exemple: le modèle ou l'humain révise le plus petit engagement formel pertinent, et l'analyse est rejouée. Il s'agit d'une utilisation plus disciplinée de la rétroaction LLM que de l'autocritique générique parce que le signal d'erreur est lié à une dépendance formelle.

Aucune langue cible unique n'est susceptible de soutenir toutes les formes utiles de prose exécutable. La littérature démontre déjà la dépendance au choix du langage intermédiaire, et les systèmes contemporains réussissent en utilisant Python, SQL, logique de premier ordre, logique temporelle, PDDL, langages de contrainte, ou grammaires spécifiques au domaine selon la tâche. Une durée d'exécution pratique devrait donc utiliser un pivot sémantique provenant et un portefeuille de moteurs formels. L'évaluation Relational et Datalog traitent les requêtes et la fermeture du document; les tables de décision traitent les politiques finies; SAT/SMT/CSP traitent les contraintes d'interaction; le modèle vérifie les comportements temporels; les traitements d'exécution symboliques détectent les témoins; les automates manipulent les modèles ordonnés; les poignées logiques naturelles sélectionnées impliquent; les assistants de preuve traitent les revendications formelles de haute valeur.

Plurality présente ses propres risques d'ingénierie. Chaque compilateur peut déformer la sémantique, chaque moteur élargit la base de confiance, et le routage peut échouer. Le portefeuille n'est justifié que si les interfaces sont tapées, si les symboles partagés conservent l'identité et la provenance, que les garanties du moteur sont explicites et que les résultats sont normalisés en un produit canonique portant un certificat. Une logique universelle peut rester préférable dans un domaine étroit. La revendication architecturale n'est pas que la pluralité est toujours meilleure; c'est que le langage exécutable large nécessite des structures de calcul différentes et ne doit pas les déguiser derrière un vague DSL.

Le retour des symboles au langage fait partie de la frontière de confiance. Un dernier LLM ne doit pas être autorisé à transformer un résultat conditionnel en une recommandation inconditionnelle, une éventuelle violation en une violation confirmée ou une preuve manquante en preuve d'absence. Le résultat canonique devrait définir un ensemble fini de claims sous licence avec modalité, quantités, entités, hypothèses et provenance. Le rendu peut améliorer la lisibilité, mais doit rester subordonné à cette autorité. La réalisation à contre-courant est l'achèvement naturel de l'architecture.

La mise en œuvre publique nllAgent est importante car elle incarne déjà plusieurs de ces disciplines. LongTextJS préserve un programme de documents à la base de la source; CircuitJS représente les théories réutilisables; la demande d'observation est dérivée de circuits; la compatibilité et la couverture sont explicites; les modules restreints et les extensions examinées limitent l'exécution cachée; l'apprentissage a lieu dans l'étape jetable; la publication est manuelle et immuable; replay verification contrôle les résultats canoniques; et les connaissances manquantes ne sont pas traitées comme la conformité. Ces décisions font du

projet une plate-forme crédible pour la recherche, même si elles n'établissent pas de compréhension sémantique générale.

Les mêmes matériaux publics identifient l'important travail inachevé: matérialisation générale de l'identité, contrats primitifs de type complets, mutation sémantique exécutable, évaluation indépendante de l'ombre, benchmarking en mode direct, repères de planification et de réalisation, et paquets de connaissances actuels du monde. Aucune évaluation indépendante évaluée par des pairs de nllAgent n'a été trouvée pour cette édition. Le jugement proposé ici est donc architectural: la mise en œuvre a choisi des limites inhabituellement solides, mais ses revendications défendables les plus fortes restent une analyse structurelle et linguistique contrôlée bornée. Sa valeur augmentera si l'ampleur sémantique n'est étendue que par le biais de producteurs explicites, de contrats, de points de référence et de preuves.

L'étape suivante proposée n'est pas de remplacer LongTextJS ou CircuitJS par un langage universel plus grand. LongTextJS devrait mûrir en tant que monde sémantique fini, interrogeable et porteur de provenance. CircuitJS devrait rester une couche restreinte d'orchestration et de preuve-contrat. Les moteurs enregistrés spécialisés doivent effectuer une exécution relationnelle, de contrainte, temporelle, de flux de données, d'automates ou sensible au chemin. La sémantique de mai/mus doit devenir opérationnelle. Les contre-exemples devraient conduire au raffinement sémantique concentré. Les produits canoniques CNL doivent contrôler la sortie de la langue finale.

Un programme de recherche rigoureux peut tester chaque partie. La sémantique candidate peut être comparée à des systèmes de traduction unique. Le raffinement dérivé du solveur peut être comparé à l'auto-réflexion. Les contrats de preuves de type peuvent être testés pour le transfert et la fausse assurance. Un portefeuille de formalisme peut être comparé à un langage intermédiaire universel. Les domaines abstraits peuvent être vérifiés par rapport à des ensembles d'interprétation explicites. Les témoins symboliques peuvent être évalués pour la découverte de défauts et l'utilité de l'examineur. La réalisation à contre-teneur peut être testée pour la fidélité. Les architectures modulaires et de bout en bout peuvent être comparées sur les mêmes tâches plutôt que sur différentes familles de référence.

La métrique décisive devrait être une fausse assurance, et non seulement une réponse. Un système qui retourne «inconnu parce que l'identité n'est pas résolue» peut avoir plus de valeur qu'un système qui produit un verdict confiant en fusionnant silencieusement deux personnes. Un système qui s'arrête parce que les preuves du monde actuel sont obsolètes est plus digne de confiance que celui qui récupère un fait non étayé. Un système qui expose trois interprétations vaut mieux qu'un système qui choisit le plus fluide. L'abstention appropriée fait partie de la sémantique d'exécution.

Le langage naturel exécutable devrait donc être compris comme une relation de compilation régie, et non comme une propriété intrinsèque de la prose. Le texte source reste plus riche que toute projection exécutable. Le système sélectionne une tâche, matérialise la sémantique pertinente, préserve l'incertitude, les symboles de base, compile à un formalisme approprié, exécute sous des limites explicites, vérifie le résultat et renvoie une explication limitée. Ce qui devient exécutable n'est pas le langage dans son intégralité mais une projection sémantique déclarée dont les hypothèses et les limites restent visibles.

Cette conception plus étroite est suffisante pour un impact pratique important. Les allégations scientifiques peuvent être vérifiées pour la provenance et la cohérence. Les exigences peuvent être liées à des tests et à des états de cycle de vie. Les politiques peuvent

être rassemblées en structures de décision. Les plans peuvent être vérifiés avant la réalisation. Les règles du protocole peuvent être cartographiées pour les analyses d'état et de chemin. Les documents longs peuvent être colportés par des théories réutilisables sans demander à un modèle de se souvenir de chaque détail. Les agents peuvent utiliser le langage naturel comme couche de création et de coordination tout en déléguant des conséquences exactes aux exécutions symboliques.

Le principe central peut être énoncé sans symbolisme formel: laissez le LLM découvrir la forme possible; laissez le noyau de mise à la terre décider ce que les symboles sont autorisés à désigner; laissez les moteurs formels déterminer ce qui suit; et laissez le LLM final expliquer seulement ce que les licences d'enregistrement symboliques. Ce principe ne résout pas tous les problèmes de langage. Il crée une frontière défendable entre l'interprétation et la conséquence, ce qui est la base nécessaire pour que le langage naturel participe en toute sécurité à des systèmes exécutables.

Références

- Abzianidze, Lasha, Rik van Noord, Hessel Haagsma et Johan Bos. 2019. « La première tâche partagée sur la représentation du discours Structure d'analyse et d'évaluation ». Actes de la tâche partagée IWCS sur l'analyse sémantique. Association pour la linguistique computationnelle. <https://aclanthology.org/W19-1201/>
- Abzianidze, Lasha. 2017. « LangPro: Prover de théorème du langage naturel ». Actes de la conférence 2017 sur les méthodes empiriques dans le traitement du langage naturel: démonstrations de système. Association pour la linguistique computationnelle. <https://aclanthology.org/D17-2022/>
- Angeli, Gabor et Christopher D. Manning. 2014. « NaturalLI: Inférence logique naturelle pour le raisonnement de bon sens ». Actes de EMNLP 2014. Association pour la linguistique computationnelle. <https://aclanthology.org/D14-1209/>
- AssistOS-AI. 2026. « NaturalLanguageLinterAgent (nllAgent): LongTextJS–CircuitJS Architecture, documentation et problèmes graves ». Référentiel public et documentation générée, inspecté 2 août 2026. <https://github.com/assistos-ai/nllAgent>
- Badreddine, Samy, Artur d'Avila Garcez, Luciano Serafini et Michael Spranger. 2022. « Réseaux de tenseur logique ». Intelligence Artificielle 303: 103649. <https://doi.org/10.1016/j.artint.2021.103649>
- Baldoni, Roberto, Emilio Coppa, Daniele Cono D'Elia, Camil Demetrescu et Irene Finocchi. 2018. « Une étude de synthèse sur les techniques d'exécution symbolique ». ACM Enquêtes de calcul 51(3): 1–39. <https://doi.org/10.1145/3182657>
- Beg, Arshad, Diarmuid P. O'Donoghue et Rosemary Monahan. 2025. « Une courte enquête sur la formalisation des exigences logicielles à l'aide de grands modèles de langage ». arXiv:2506.11874. <https://doi.org/10.48550/arXiv.2506.11874>
- Beiser, Alexander et David Penz. 2025. « Faire LLMs Raison? Le problème de langage intermédiaire dans les approches neurosymboliques ». arXiv:2502.17216. <https://arxiv.org/abs/2502.17216>
- Bos, Johan. 2008. « Analyse sémantique à large couverture avec Boxer ». Actes de la conférence 2008 sur la sémantique dans le traitement de texte. <https://aclanthology.org/W08-2222/>
- Chatzikyriakidis, Stergios et Shalom Lappin. 2026. « Neuro-Symbolique NLP: Taxonomie, évaluation et directions ». Frontières de l'intelligence artificielle 9. <https://doi.org/10.3389/frai.2026.1797587>
- Chen, Lingfeng, Tao Xiao, Masanari Kondo et Yasutaka Kamei. 2026. « Exécution symbolique dirigée pour la découverte de vulnérabilité: une approche guidée par LLM dans KLEE ». arXiv:2607.21676. <https://arxiv.org/abs/2607.21676>
- Chen, Yongchao, Rujul Gandhi, Yang Zhang et Chuchu Fan. 2023. « NL2TL: Transformer les langages naturels en logiques temporelles en utilisant de grands modèles de langage ». Actes de la conférence 2023 sur les méthodes empiriques dans le traitement du langage naturel. Association pour la linguistique computationnelle. <https://aclanthology.org/2023.emnlp-main.985/>
- Clarke, Edmund, Orna Grumberg, Somesh Jha, Yuan Lu et Helmut Veith. 2000. « Affinement d'abstraction guidé par contre-exemple ». Vérification Assistée Par Ordinateur, LNCS 1855, 154–169. https://doi.org/10.1007/10722167_15
- Cosler, Matthias, Christopher Hahn, Daniel Mendoza, Frederik Schmitt et Caroline Trippel. 2023. « nl2spec: Traduire de manière interactive un langage naturel non structuré aux logiques temporelles avec de grands modèles de langage ». Vérification Assistée Par Ordinateur, LNCS 13964, 383–396. https://doi.org/10.1007/978-3-031-37703-7_18
- Cousot, Patrick et Radhia Cousot. 1977. « Un modèle de réseau unifié pour l'analyse statique des programmes par la construction ou l'approximation des points de correction ». Actes de POPL 1977, 238–252. <https://doi.org/10.1145/512950.512973>
- Delvecchio, Giovanni Pio, Lorenzo Molfetta et Gianluca Moro. 2025. « Intelligence artificielle neuro-symbolique: une étude de synthèse dirigée par des tâches à l'ère des modèles de boîte noire ». Résultats de IJCAI 2025, piste d'enquête, 10418–10426. <https://doi.org/10.24963/ijcai.2025/1157>
- Dong, Wenkai et Yifan Wang. 2026. « De l'interprétation à la compilation: exécution basée sur la compilation d'opérateurs sémantiques ». arXiv:2607.13407. <https://doi.org/10.48550/arXiv.2607.13407>

- Ferrari, Alessio et Paola Spoletini. 2025. « Exigences formelles Ingénierie et grands modèles de langage: une feuille de route bidirectionnelle ». *Technologies de l'information et du logiciel* 181: 107697. <https://doi.org/10.1016/j.infsof.2025.107697>
- Fuchs, Norbert E. et Rolf Schwitter. 1996. « Tentative contrôlée en anglais (ACE) ». Actes du premier atelier international sur les applications linguistiques contrôlées. <https://attempto.ifi.uzh.ch/site/pubs/papers/ace96.pdf>
- Fuchs, Norbert E., Kaarel Kaljurand et Tobias Kuhn. 2008. « Tentative d'anglais contrôlé pour la représentation des connaissances ». *Raisonnement Web, LNCS 5224*, 104–124. https://doi.org/10.1007/978-3-540-85658-0_3
- Gao, Luyu, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan et Graham Neubig. 2022. « PAL: Modèles de langage aidés par programme ». *arXiv:2211.10435*. <https://arxiv.org/abs/2211.10435>
- Goldman, Omer, Veronica Latcinnik, Udi Naveh, Amir Globerson et Jonathan Berant. 2018. « Parsage sémantique faiblement supervisé avec des exemples abstraits ». Actes de ACL 2018. Association pour la linguistique computationnelle. <https://aclanthology.org/P18-1168/>
- Grimaldi, Pasquale. 2026. « pamaldi à SemEval-2026 Tâche 11: Raisonnement syllogistique neuro-symbolique via l'extraction de structure guidée LLM et la validation déterministe ». Actes du 20e atelier international sur l'évaluation sémantique. Association pour la linguistique computationnelle. <https://doi.org/10.18653/v1/2026.semeval-1.186>
- Gu, Qiuhan, Avaljot Singh et Gagandeep Singh. 2026. « SAIL: Interprètes abstraits sonores avec LLMs ». Actes du ACM sur les langages de programmation 10 (PLDI): 1534–1559. <https://doi.org/10.1145/3808308>
- Guu, Kelvin, Panupong Pasupat, Evan Liu et Percy Liang. 2017. « Du langage aux programmes: l'apprentissage par renforcement de pont et la probabilité maximale de marginal ». Actes de ACL 2017. Association pour la linguistique computationnelle. <https://aclanthology.org/P17-1016/>
- Hamilton, Kyle, Aparna Nayak, Bojan Božić et Luca Longo. 2024. « Le neuro-symbole AI répond-il à ses promesses en matière de traitement du langage naturel? Un examen structuré ». *Web sémantique* 15(4): 1265–1306. <https://doi.org/10.3233/SW-223228>
- Han, Simeng, Hailey Schoelkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Wenfei Zhou, James Coady, David Peng, Yujie Qiao, Luke Benson, Lucy Sun, Alex Wardle-Solano, Hannah Szabó, Ekaterina Zubova, Matthew Burtell, Jonathan Fan, Yixin Liu, Brian Wong, Malcolm Sailor, Ansong Ni, Linyong Nan, Jungo Kasai, Tao Yu, Rui Zhang, Alexander Fabbri, Wojciech Kryściński, Semih Yavuz, Ye Liu, Xi Victoria Lin, Shafiq Joty, Yingbo Zhou, Caiming Xiong, Rex Ying et Dragomir Radev. 2024. « FOLIO: Raisonnement du langage naturel avec la logique de première commande ». Actes de EMNLP 2024. Association pour la linguistique computationnelle. <https://aclanthology.org/2024.emnlp-main.1229/>
- Hao, Yilun, Yongchao Chen, Yang Zhang et Chuchu Fan. 2025. « Les grands modèles de langage peuvent résoudre la planification du monde réel avec des outils de vérification formels ». Actes de NAACL 2025, 3434–3483. <https://aclanthology.org/2025.naacl-long.176/>
- Hu, Ruikang, Shaoyu Lin, Yeliang Xiu et Yongmei Liu. 2025. « LTRAG: Améliorer l'autoformalisation et l'auto-raffinement pour le raisonnement logique avec RAG guidé par la pensée ». Résultats de ACL 2025, 2483–2493. <https://aclanthology.org/2025.findings-acl.126/>
- Huang, Pei, Jiayu Li, Yuhao Zhang, Changliu Liu et Shih-hsi Alex Liu. 2026. « Interprétation abstraite paramétrée pour la vérification du transformateur ». Actes de AAAI 2026. <https://doi.org/10.1609/aaai.v40i42.40860>
- Jia, Robin, Aditi Raghunathan, Kerem Göksel et Percy Liang. 2019. « Robustesse certifiée aux substitutions verbales contradictoires ». Actes de EMNLP-IJCNLP 2019. Association pour la linguistique computationnelle. <https://aclanthology.org/D19-1423/>
- Karthikeyan, T, Om Dehlan, Mausam et Manish Gupta. 2025. « LRPLAN: Une collaboration multi-agents de grands modèles de langage et de raisonnement pour la planification avec des contraintes implicites et explicites ». Résultats de EMNLP 2025, 8280–8310. <https://doi.org/10.18653/v1/2025.findings-emnlp.440>
- Kartáč, Ivan, Kristýna Onderková, Jan Bronec, Zdeněk Kasner, Mateusz Lango et Ondřej Dušek. 2026. « UFAL-CUNI à SemEval-2026 Tâche 11: Une méthode neuro-symbolique modulaire efficace pour le raisonnement syllogistique ». Actes du 20e atelier international sur l'évaluation sémantique. Association pour la linguistique computationnelle. <https://aclanthology.org/2026.semeval-1.418/>
- King, James C. 1976. « Exécution symbolique et tests de programme ». *Communications du ACM* 19(7): 385–394. <https://doi.org/10.1145/360248.360252>

- Kirtania, Shashank, Priyanshu Gupta et Arjun Radhakrishna. 2024. « LOGIC-LM++: Raffinement multi-étapes pour les formulations symboliques ». Actes du 2e atelier sur le raisonnement du langage naturel et les explications structurées, 56–63. <https://aclanthology.org/2024.nlrse-1.6/>
- Kuhn, Tobias. 2014. « Une étude de synthèse et une classification des langues naturelles contrôlées ». *Linguistique computationnelle* 40(1): 121–170. https://doi.org/10.1162/COLI_a_00168
- Lalwani, Vivek, et al. 2025. « Autoformalisation du langage naturel à la logique de premier ordre: une étude de cas dans la détection logique de l'erreur ». Résultats de IJCNLP-AACL 2025, 132–147. <https://aclanthology.org/2025.findings-ijcnlp.8/>
- Lee, Kang-il, Segwang Kim et Kyomin Jung. 2023. « Parsage sémantique faiblement supervisé avec filtrage de programme faux basé sur l'exécution ». Actes de EMNLP 2023, 6884–6894. <https://aclanthology.org/2023.emnlp-main.425/>
- Li, Yihe, Ruijie Meng et Gregory J. Duck. 2025. « Exécution Symbolique Alimentée Par Le Grand Modèle De Langage ». Actes du ACM sur les langages de programmation 9 (OOPSLA2), Article 385, 3148–3176. <https://doi.org/10.1145/3763163>
- Liang, Chen, Jonathan Berant, Quoc Le, Kenneth D. Forbus et Ni Lao. 2017. « Machines symboliques neurales: Apprentissage des parsers sémantiques sur Freebase avec une surveillance faible ». Actes de ACL 2017. Association pour la linguistique computationnelle. <https://aclanthology.org/P17-1003/>
- Liu, Jiangming, Shay B. Cohen et Mirella Lapata. 2018. « Discours Représentation Structure Parsing ». Actes de ACL 2018. Association pour la linguistique computationnelle. <https://aclanthology.org/P18-1040/>
- Locascio, Nicholas, Karthik Narasimhan, Eduardo DeLeon, Nate Kushman et Regina Barzilay. 2018. « Génération neuronale d'expressions régulières du langage naturel avec une connaissance minimale du domaine ». Actes de EMNLP 2018. Association pour la linguistique computationnelle. <https://aclanthology.org/D18-1436/>
- MacCartney, Bill et Christopher D. Manning. 2007. « Logique naturelle pour l'inférence textuelle ». Actes de l'atelier ACL-PASCAL sur l'entraînement textuel et le paraphrasage. <https://aclanthology.org/W07-1401/>
- MacCartney, Bill et Christopher D. Manning. 2009. « Un modèle étendu de logique naturelle ». Actes de IWCS 2009. <https://aclanthology.org/W09-3714/>
- Manhaeve, Robin, Sebastijan Dumančić, Angelika Kimmig, Thomas Demeester et Luc De Raedt. 2018. « DeepProbLog: Programmation logique probabiliste neuronale ». Progrès dans les systèmes de traitement de l'information neuronale 31. <https://proceedings.neurips.cc/paper/2018/hash/dc5d637ed5e62c36ecb73b654b05ba2a-Abstract.html>
- Mao, Ziyu, Jingyi Wang, Jun Sun, Shengchao Qin et Jiawen Xiong. 2025. « LLM-Aided Automatic Modeling for Security Protocole Vérification ». Actes de la 47ème conférence internationale IEEE/ACM sur le génie logiciel, 2025. https://ink.library.smu.edu.sg/sis_research/10297/
- Mitchell, Jacqueline L., Brian Hyeongseok Kim, Chenyu Zhou et Chao Wang. 2025. « LLMs peut-il officiellement raisonner en tant qu'interprètes abstraits pour l'analyse de programme? ». arXiv:2503.12686. <https://arxiv.org/abs/2503.12686>
- Mou, Lili, Zhengdong Lu, Hang Li et Zhi Jin. 2017. « Couplage Exécution distribuée et symbolique pour les requêtes en langage naturel ». Actes de ICML 2017. <https://proceedings.mlr.press/v70/mou17a.html>
- Olausson, Theo X., Alex Gu, Benjamin Lipkin, Cedegao E. Zhang, Armando Solar-Lezama, Joshua B. Tenenbaum et Roger Levy. 2023. « LINC: Une approche neurosymbolique pour le raisonnement logique en combinant des modèles de langage avec des proverbes logiques de premier ordre ». Actes de EMNLP 2023. Association pour la linguistique computationnelle. <https://aclanthology.org/2023.emnlp-main.313/>
- Pan, Liangming, Alon Albalak, Xinyi Wang et William Yang Wang. 2023. « Logic-LM: Autonomiser les grands modèles de langage avec des solveurs symboliques pour un raisonnement logique fidèle ». Résultats de EMNLP 2023. Association pour la linguistique computationnelle. <https://aclanthology.org/2023.findings-emnlp.248/>
- Pasupat, Panupong et Percy Liang. 2016. « Inférer des formes logiques à partir des dénnotations ». Actes de ACL 2016. Association pour la linguistique computationnelle. <https://aclanthology.org/P16-1004/>
- Pei, Yu, Yongping Du et Xingnan Jin. 2025. « FoVer: Vérification logique de première commande pour le raisonnement du langage naturel ». Transactions de l'Association pour la linguistique computationnelle 13: 1340–1359. <https://aclanthology.org/2025.tacl-1.61/>

- Tafjord, Oyvind, Bhavana Dalvi Mishra et Peter Clark. 2021. « ProofWriter: Génération d'implications, de preuves et d'énoncés abductifs sur le langage naturel ». Résultats de ACL-IJCNLP 2021. Association pour la linguistique computationnelle. <https://aclanthology.org/2021.findings-acl.317/>
- Tantakoun, Nat, Christian Muise et Yilun Zhu. 2025. « Grands modèles de langage comme formalisateurs de planification: une étude de synthèse ». Résultats de ACL 2025. <https://aclanthology.org/2025.findings-acl.1291/>
- Tziifas, Georgios et Hamidreza Kasaei. 2026. « Planification de tâches neuro-symbolique et manipulation d'objets avec de grands modèles de langage ». Robots autonomes 50. <https://doi.org/10.1007/s10514-026-10230-6>
- Wang, Bailin, Richard Shin, Xiaodong Liu, Oleksandr Polozov et Matthew Richardson. 2018. « Décodage de programme de neurones guidés par exécution ». Actes du premier atelier sur la synthèse du programme appris. <https://arxiv.org/abs/1807.03100>
- Wang, Boxin, Chejian Xu, Xiangyu Liu, Yu Cheng et Bo Li. 2023. « L'intégration de mots conscients de la robustesse améliore la robustesse certifiée par rapport aux substitutions de mots contradictoires ». Résultats de ACL 2023. Association pour la linguistique computationnelle. <https://aclanthology.org/2023.findings-acl.620/>
- Wang, Jiayimei, Tao Ni, Wei-Bin Lee et Qingchuan Zhao. 2025. « Une étude de synthèse contemporaine sur l'analyse de programme assisté par grand modèle de langage ». Tendances de l'intelligence artificielle. <https://arxiv.org/abs/2502.18474>
- Wang, Wenguan, Yi Yang et Fei Wu. 2025. « Vers une étude de synthèse sur l'informatique neuro-symbolique axée sur les données et la connaissance ». IEEE Transactions sur l'analyse de motifs et l'intelligence machine 47(2): 878–899. <https://doi.org/10.1109/TPAMI.2024.3483273>
- Wang, Yu, You Zhang, Hao Zhang et Dan Xu. 2026. « YNU-NLP à SemEval-2026 Tâche 11: Une approche neuro-symbolique avec mécanisme de réflexion démêlant le contenu et le raisonnement formel dans les modèles de langage ». Actes du 20e atelier international sur l'évaluation sémantique. Association pour la linguistique computationnelle. <https://doi.org/10.18653/v1/2026.semeval-1.126>
- Weng, Ke, Lun Du, Sirui Li, Wangyue Lu, Haozhe Sun, Hengyu Liu et Tiancheng Zhang. 2025. « Autoformalisation à l'ère des grands modèles de langage: une étude de synthèse ». arXiv:2505.23486. <https://arxiv.org/abs/2505.23486>
- Xia, Shihao, Mengting He, Shuai Shao, Tingting Yu, Yiyi Zhang, Nobuko Yoshida et Linhai Song. 2026. « SymGPT: Audit de contrats intelligents via la combinaison d'exécution symbolique avec de grands modèles de langage ». Actes du ACM sur les langages de programmation 10 (OOPSLA1), Article 109. <https://doi.org/10.1145/3798217>
- Yang, Zonglin, et al. 2024. « Autoformalisation des problèmes de raisonnement du langage naturel à la logique de premier ordre ». Actes de EMNLP 2024. Association pour la linguistique computationnelle. <https://aclanthology.org/2024.emnlp-main.1132/>
- Ye, Mao, Chengyue Gong et Qiang Liu. 2020. « SAFER: Une approche sans structure pour la robustesse certifiée des substitutions de mots contradictoires ». Actes de ACL 2020. Association pour la linguistique computationnelle. <https://aclanthology.org/2020.acl-main.317/>
- Ye, Xi, Qiaochu Chen, Isil Dillig et Greg Durrett. 2023. « SatLM : modèles de langage assistés par la satisfaisabilité au moyen d'un prompting déclaratif ». Progrès dans les systèmes de traitement de l'information neuronale 36. <https://arxiv.org/abs/2305.09656>
- Zheng, Zihan, Tianle Cui, Chuwen Xie, Jiahui Pan, Qianglong Chen et Lewei He. 2025. « PlanningArena: Un repère modulaire pour l'évaluation multidimensionnelle de la planification et de l'apprentissage des outils ». Actes de ACL 2025, 31047–31086. <https://doi.org/10.18653/v1/2025.acl-long.1499>